▼ More on Before-or-After atomicity

1 ✦ More on Before-or-After atomicity

▼ OCC: executing a transaction without locks

2 ✦ Optimistic concurrency control

3 ✦ OCC Executes a Transaction in 3 Phases

4 ✦ OCC in details: The startup

5 ✦ OCC in details: Phase 1

6 ✦ OCC in details: Phase 2 & 3

7 ✦ The critical section requirement of executing phase 2 + 3

8 ✦ Why must phase 2 & phase 3 be in a critical section?

9 ✦ How to check whether a value has changed?

10 ✦ The ABA problem

11 ✦ How to implement the critical section for phase 2 & 3?

12 ✦ Re-run the two TXs with the global lock

13 ✦ Improvement: enabling more concurrency for OCC

14 ✦ How to implement the critical section for phase 2 & 3?

15 ✦ Problem: read-write conflict

16 ✦ How to implement the critical section for phase 2 & 3?

▼ Analysis of OCC

17 ✦ OCC Advantages

18 ✦ OCC Advantages (More)

19 ✦ Locking preliminary

20 ✦ Locking preliminary: atomic compare and swap

21 ✦ Locking (Spin Lock) using Compare-and-swap

22 ✦ Atomic operations are costly for spin lock

23 ✦ Put it all together: OCC in an example

24 ✦ OCC's Problem: False Aborts

25 ✦ OCC's Problem: False Aborts & Livelock

26 ✦ 2PL vs. OCC: in a nutshell

▼ Hardware transactional memory

27 ✦ Hardware Transactional Memory

28 ✦ A little introduction to RTM's ISA

29 ✦ Programming with RTM

30 ✦ The concurrent write aborts the RTM region

31 ✦ Challenge: how to handle aborts?

32 ✦ Can we use a simple retry? No

33 ✦ Why can't RTM guarantee success? Fun facts about the hardware implementation of RTM

34 ✦ Implementation of RTM: OCC!

35 ✦ The issue behind the efficient implementation: Limited working set

36 ✦ Issue: External events break the RTM execution

37 ✦ Hardware vs. Software-based OCC

38 ✦ Short summary of HTM

▼ Put it all together: transactions

39 ✦ Widely used transaction (TX)s

40 ✦ What is a transaction (TX)?

41 ✦ Why TX is good at managing the data? It provides ACID

42 ✦ Atomicity: All-or-nothing

43 ✦ Isolation & Durability

44 ✦ Consistency

45 ✦ Enforcing ACID properties (in the single-machine case)

46 ✦ Why Isolation (serializability) is ideal for consistency?

47 ✦ Review: 2PL vs. OCC: in a nutshell

▼ Multi-versioning: isolation for read-mostly transactions

48 ✦ Motivation: TX usage in real-world application

49 ✦ An analysis of read aborts in OCC

50 ✦ Abort in OCC case (2)

51 ✦ Idea: multi-versioning concurrency control

52 ✦ Time: a key method to determine the snapshot

53 ✦ Try #1: Optimize OCC w/ MV (incomplete)

54 ✦ Why incomplete? Partial snapshot (An example)

55 ✦ Enforcing the write atomicity of reading a snapshot w/ read checks

56 ✦ Example revisit

57 ✦ What about writes?

58 ✦ MVCC ensures no race for reads, but not writes

59 ✦ Put it together, MVCC so far

60 ✦ Write skew anomaly

61 ✦ Fixing the anomaly

62 ✦ Summary

63 ✦ 参考文献

More on Before-or-After atomicity

Xingda Wei

Optimistic concurrency control

Executing TXs optimistically w/o acquiring locks

- No deadlock, less blocking

Checks the results of a TX before it commits

- If it violates serializability, then aborts & retries

First proposed in 1981, widely used today







OCC Executes a Transaction in 3 Phases

Phase 1: Concurrent local processing of the transaction

- Reads data into a read set
- Buffers writes into a write set

Phase 2: Validate serializability in a (typically short) critical section

- Has any data in the read set been modified?

Phase 3: Commit the results in the same critical section as above or abort

- Aborts: aborts the transaction if validation fails
- Commits: installs the buffered write set to commit the transaction

OCC in details: The startup

Before phase one, TX needs to allocate the execution context

- Read set & write set

```
...
tx.begin();
...
tx.read(A)
...
tx.commit();
...
```

```
Init read_set
Init write_set
```

OCC in details: Phase 1

Phase 1: execute the program in the transaction

- Reads data into a read set
- Buffers writes into a write set

```
...
tx.begin();
...
tx.read(A)
...
tx.commit();
...
```

This step should be atomic!

```
val_a = read(A)
read_set.add(val_a)
```

OCC in details: Phase 1

Phase 1: execute the program in the transaction

- Reads data into a read set
- Buffers writes into a write set

```
...
tx.begin();
...
tx.read(A)
tx.write(A)
tx.commit();
...
```

```
Write_set[A] = ..
if A in read_set:
    read_set[A] = ..
```

Why must the read be atomic? Another transaction may concurrently do the commit!

OCC in details: Phase 2 & 3

Phase 2: validates whether serializability is guaranteed

- Has any data in the read set been modified?

```
...
tx.begin();
...
tx.read(A)
...
tx.commit();
...
```

```
for d in read_set:
    if d has changed:
        abort()
```

OCC in details: Phase 2 & 3

Phase 2: validates whether serializability is guaranteed

- Has any data in the read set been modified?

```
...
tx.begin();
...
tx.read(A)
...
tx.commit();
...
```

```
for d in read_set:
    if d has changed:
        abort()
for d in write_set:
    write(d)
```

Phase 3: aborts if the validation fails, otherwise installs the write set

- Phase 2 & 3 must run as one critical section

The critical section requirement of executing phase 2 + 3

Think: what if a value has changed during validation?

```
...
tx.begin();
...
tx.read(A)
...
tx.commit();
...
```

Critical section

```
for d in read_set:
    if d has changed:
        abort()
for d in write_set:
    write(d)
```

Why must phase 2 & phase 3 be in a critical section?

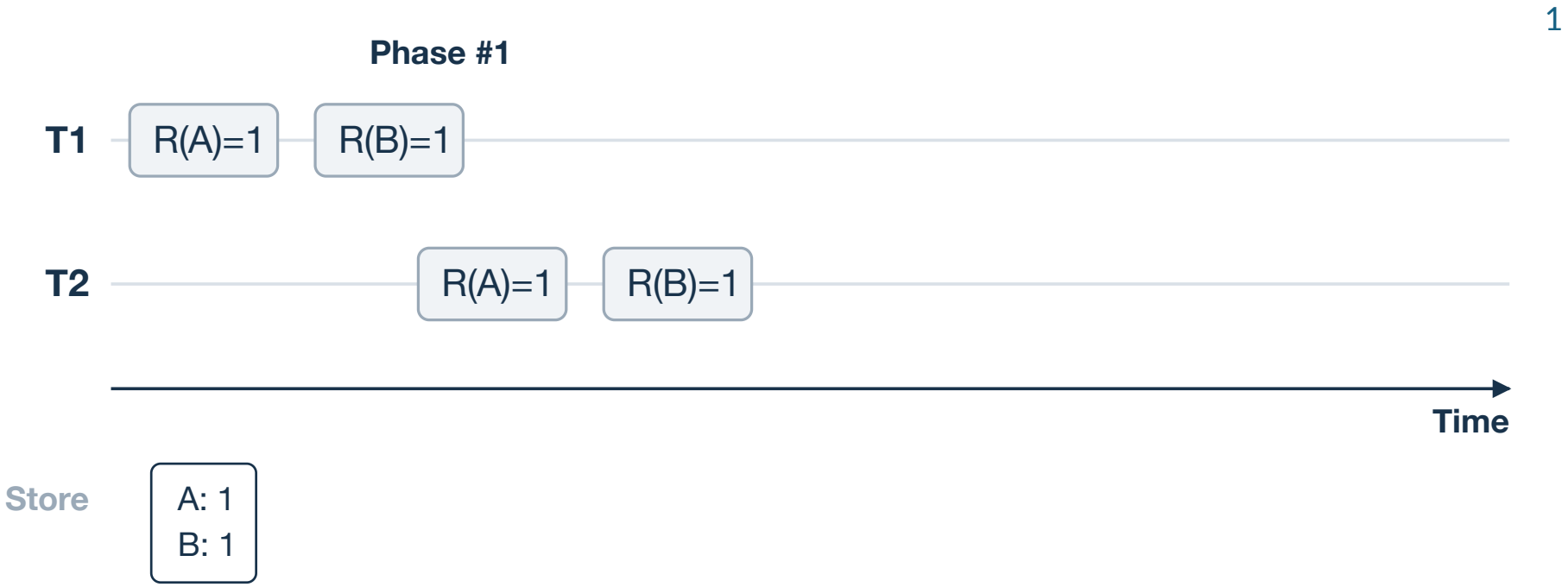
Both TXs run `A += 1; B += 1`



[1] R = read, W = write, V = validate, C = commit

Why must phase 2 & phase 3 be in a critical section?

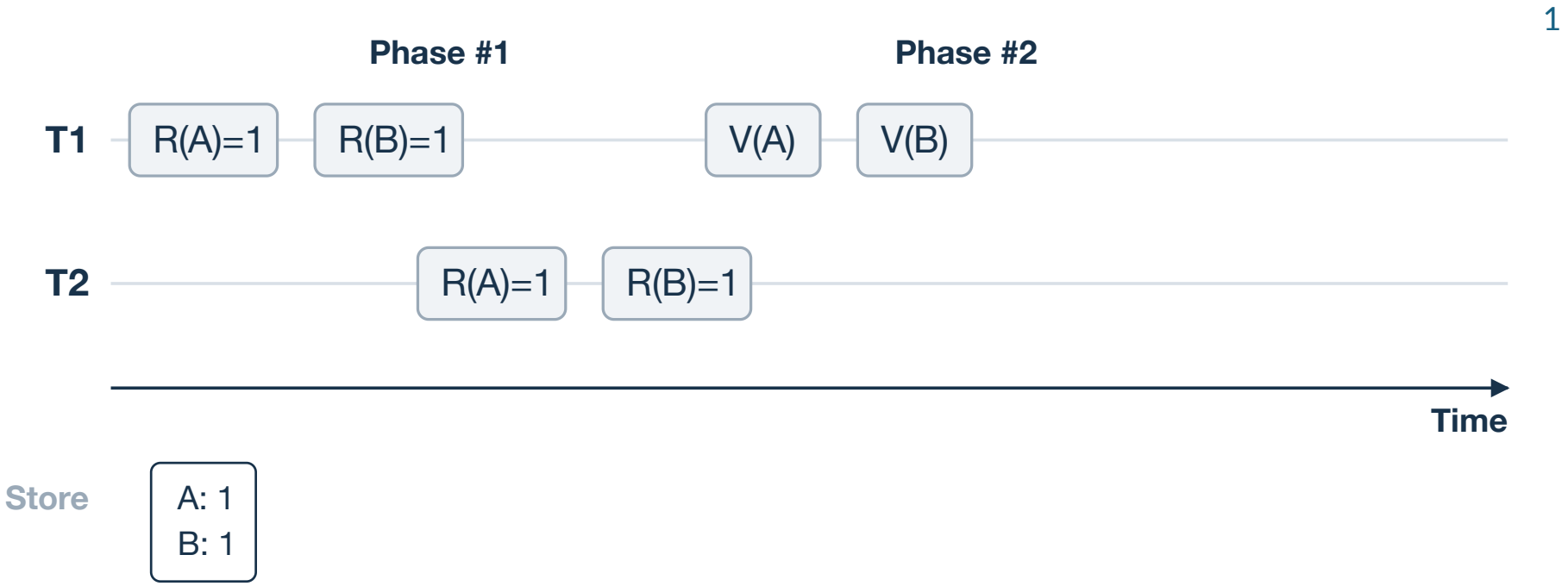
Both TXs run `A += 1; B += 1`



[1] R = read, W = write, V = validate, C = commit

Why must phase 2 & phase 3 be in a critical section?

Both TXs run `A += 1; B += 1`

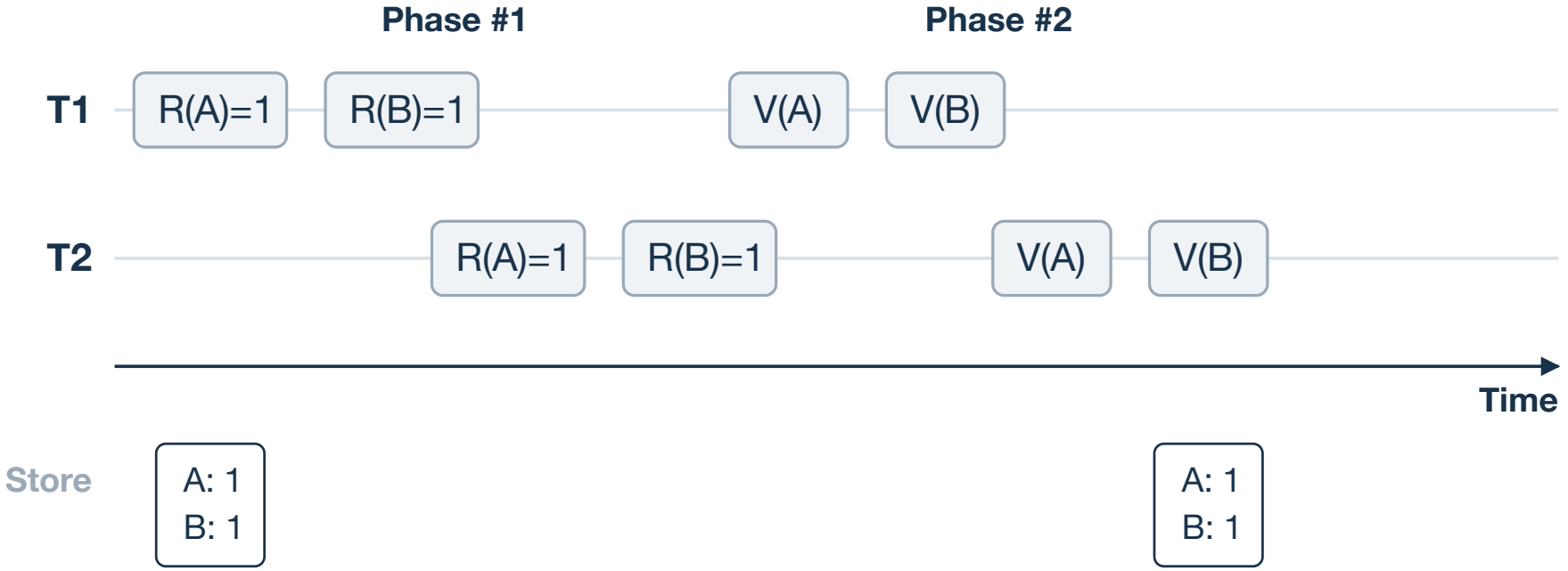


[1] R = read, W = write, V = validate, C = commit

Why must phase 2 & phase 3 be in a critical section?

Both TXs run `A += 1; B += 1`

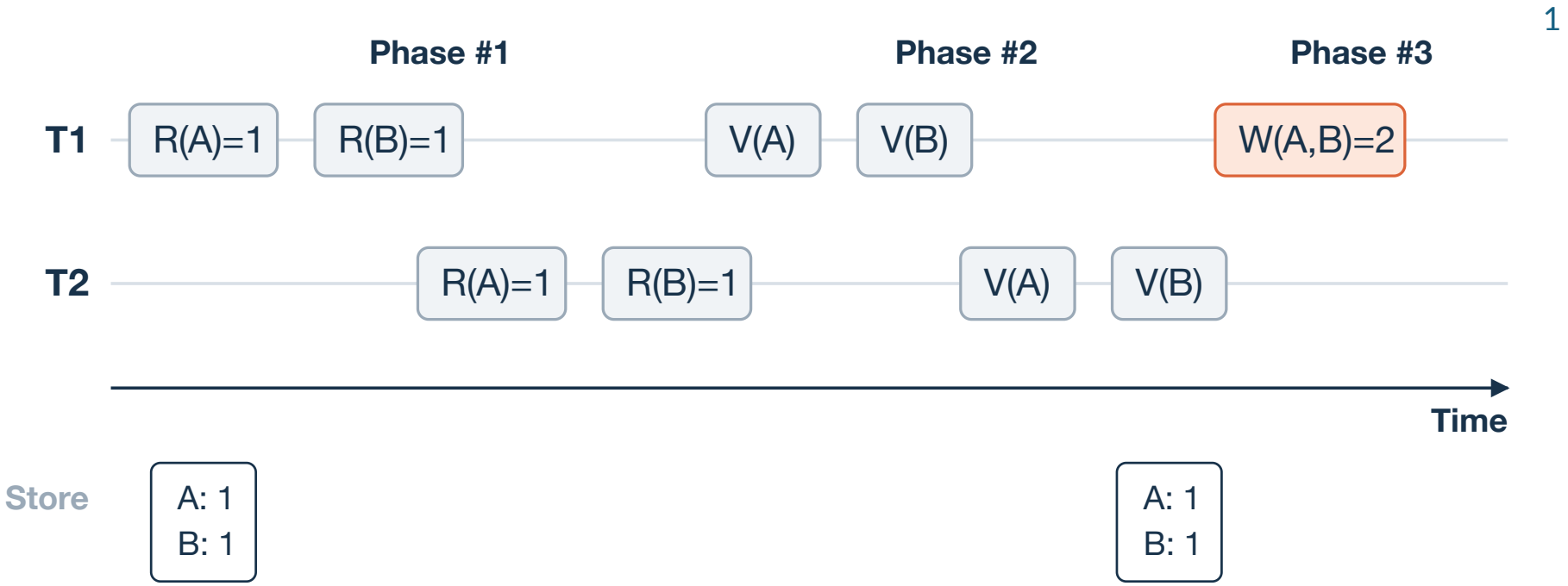
1



[1] R = read, W = write, V = validate, C = commit

Why must phase 2 & phase 3 be in a critical section?

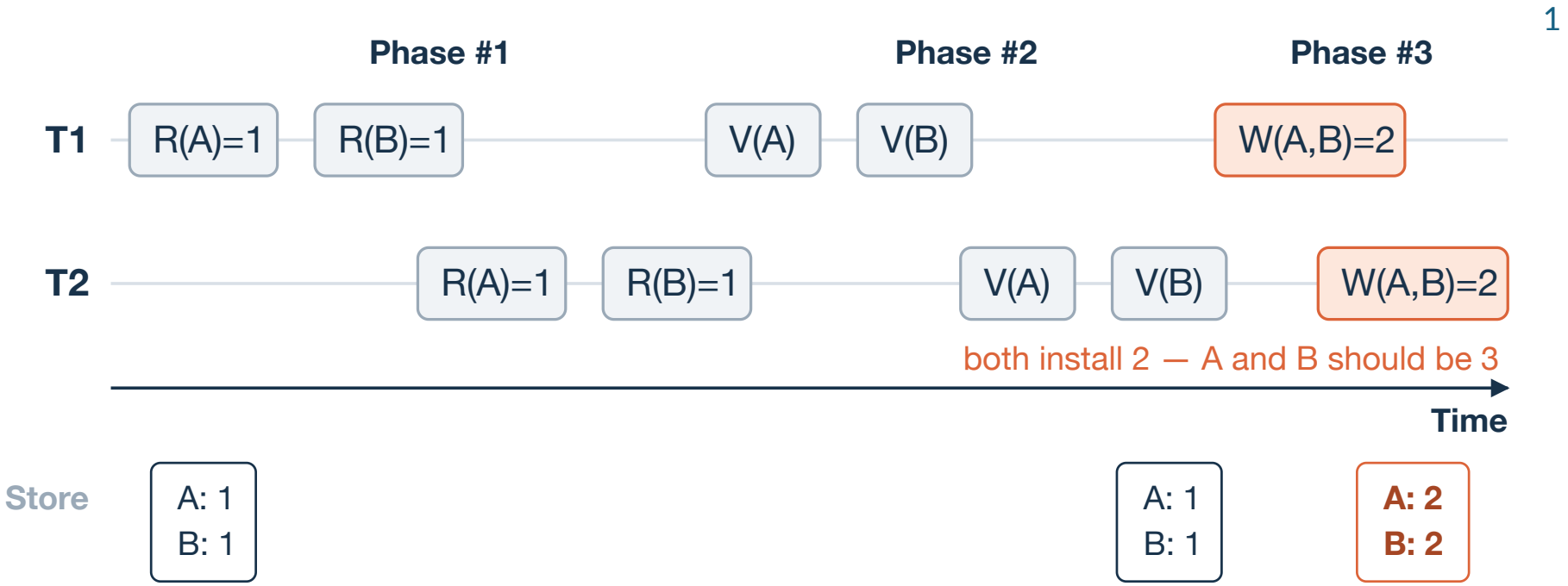
Both TXs run `A += 1; B += 1`



[1] R = read, W = write, V = validate, C = commit

Why must phase 2 & phase 3 be in a critical section?

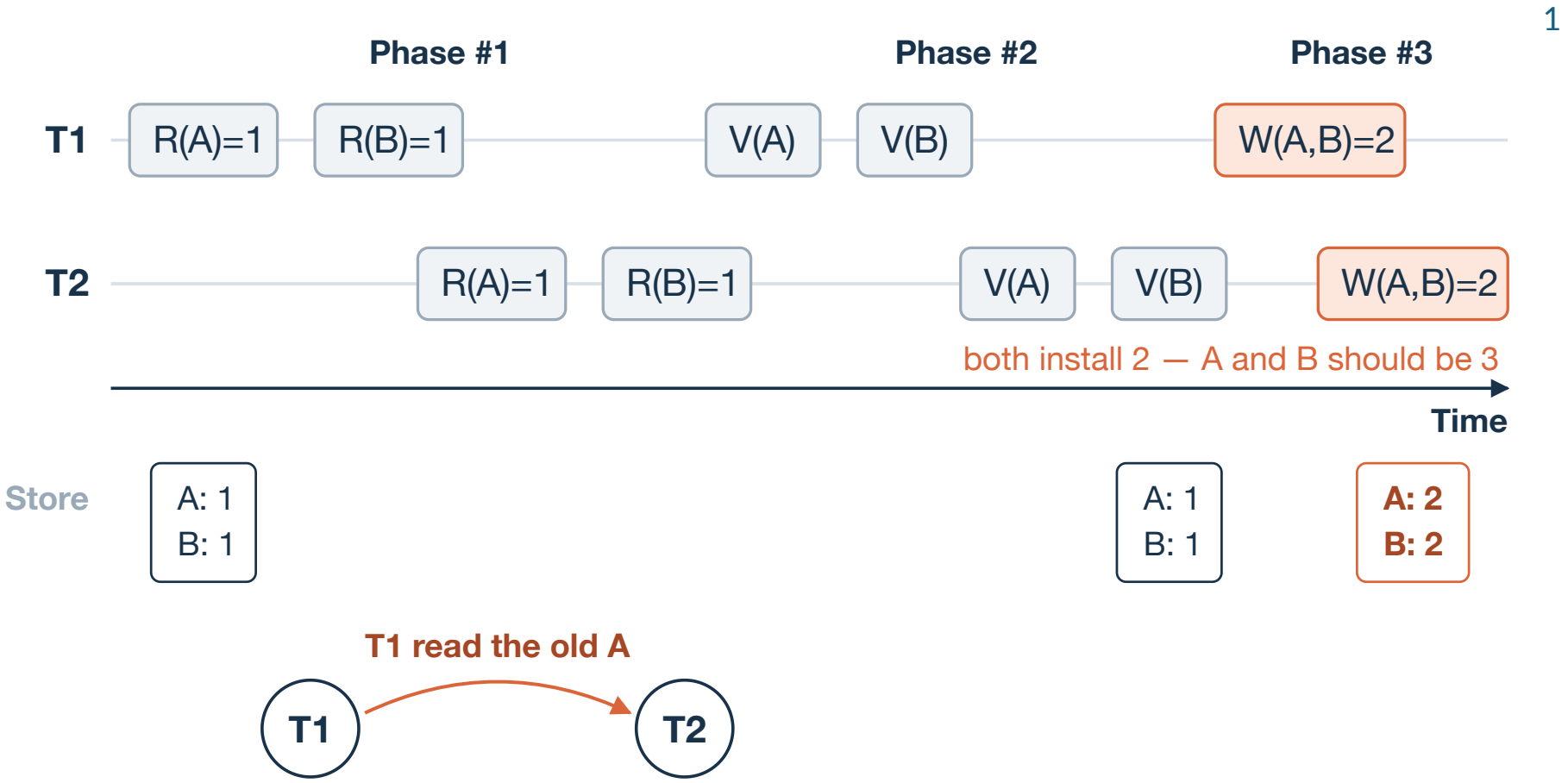
Both TXs run `A += 1; B += 1`



[1] R = read, W = write, V = validate, C = commit

Why must phase 2 & phase 3 be in a critical section?

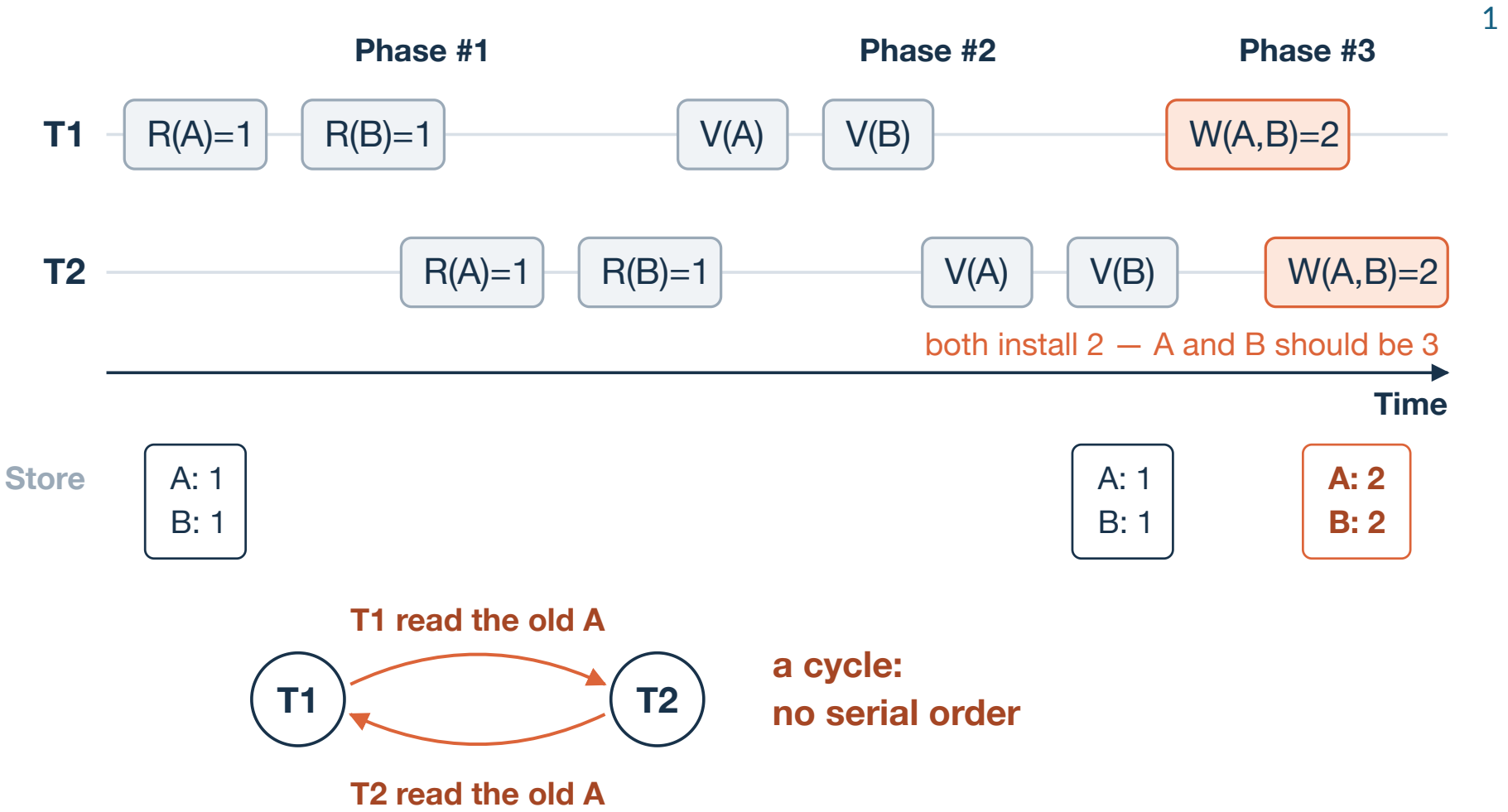
Both TXs run `A += 1; B += 1`



[1] R = read, W = write, V = validate, C = commit

Why must phase 2 & phase 3 be in a critical section?

Both TXs run `A += 1; B += 1`



[1] R = read, W = write, V = validate, C = commit

How to check whether a value has changed?

Two ways to implement `if d has changed`

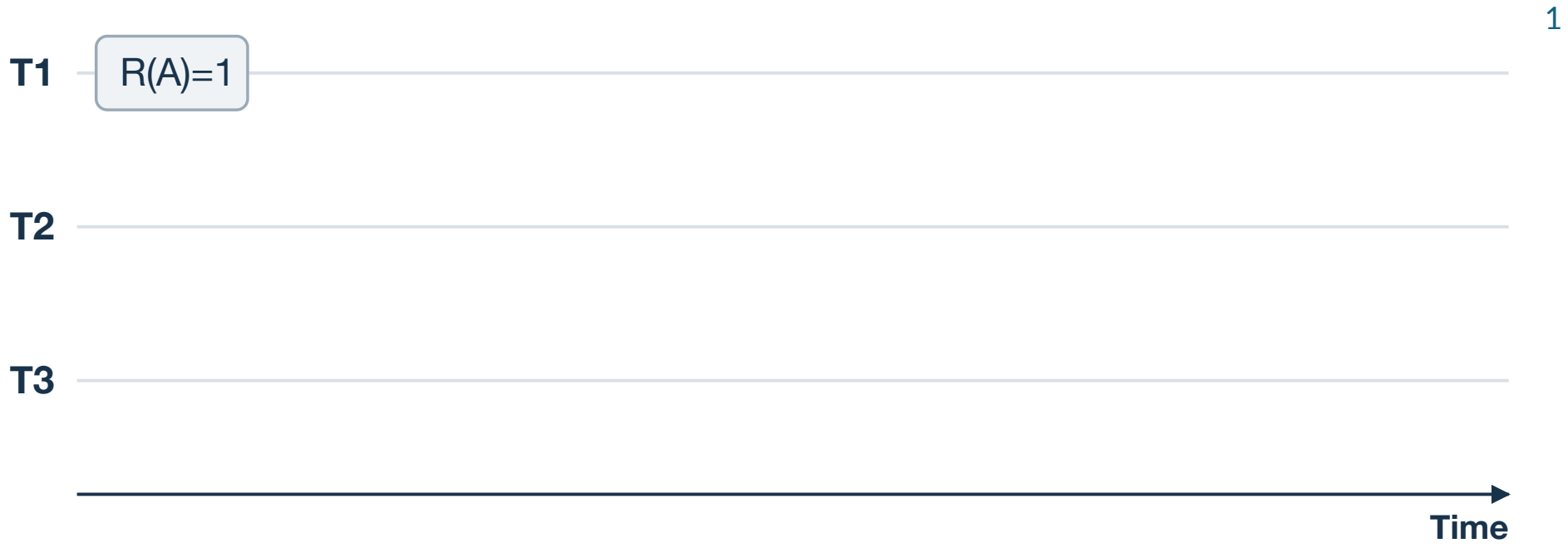
```
// method 1: keep the value we read
read-set[d] = read(d)          // phase 1
...
if read-set[d] != read(d)      // phase 2
    abort()
```

```
// method 2: keep the version we read
read-set[d] = d.version        // phase 1
...
if read-set[d] != d.version    // phase 2
    abort()
```

Question: do the two reject the same schedules?

The ABA problem

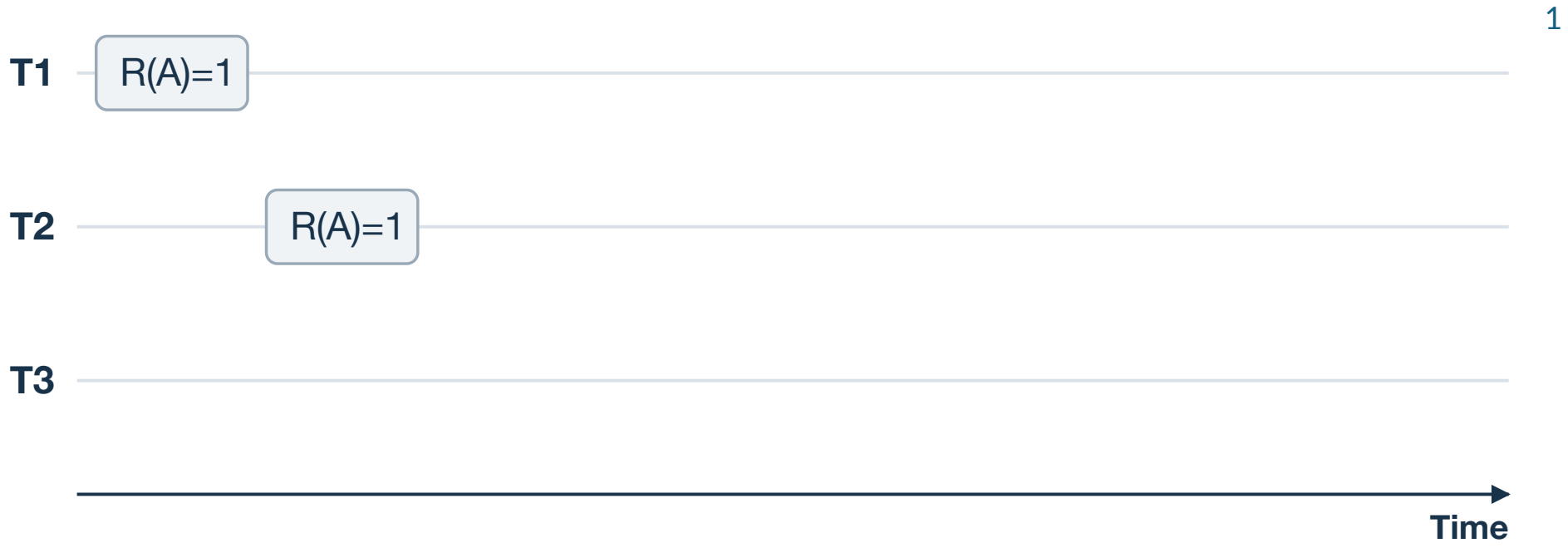
T1 and T2 both run `A += 1`, T3 runs `A = 1`; A initially is 1



[1] A = 1 to start with; R = read, V = validate, W = write, C = commit

The ABA problem

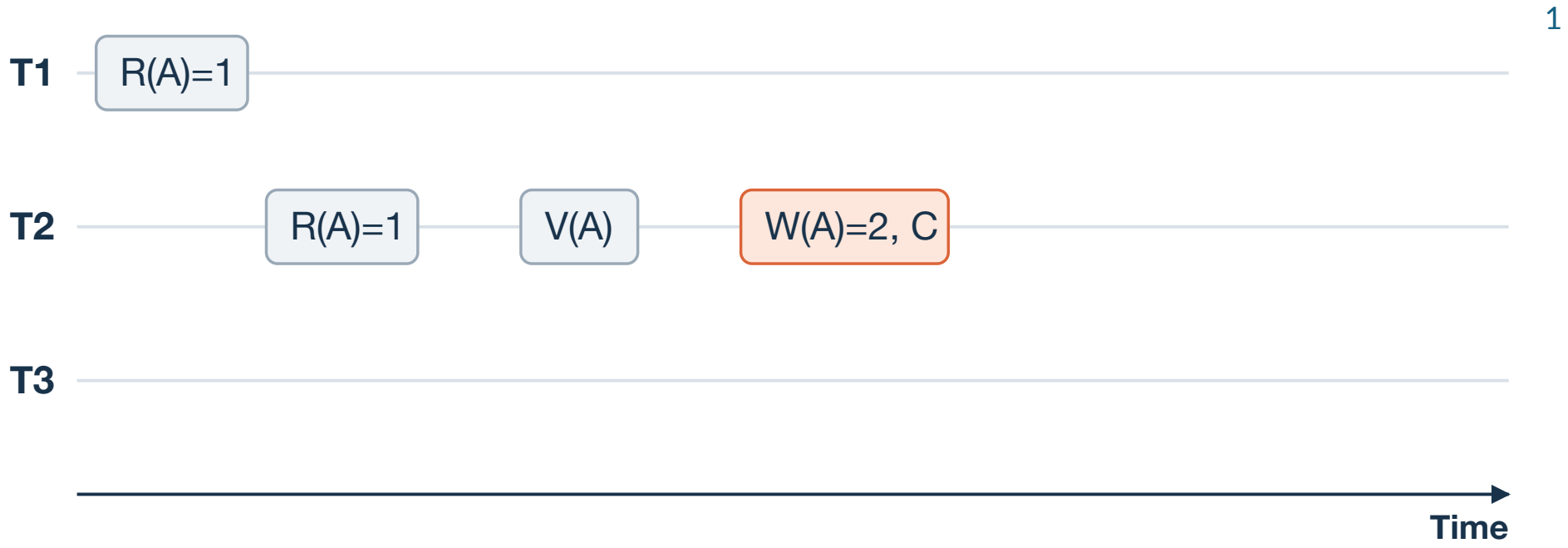
T1 and T2 both run `A += 1`, T3 runs `A = 1`; A initially is 1



[1] A = 1 to start with; R = read, V = validate, W = write, C = commit

The ABA problem

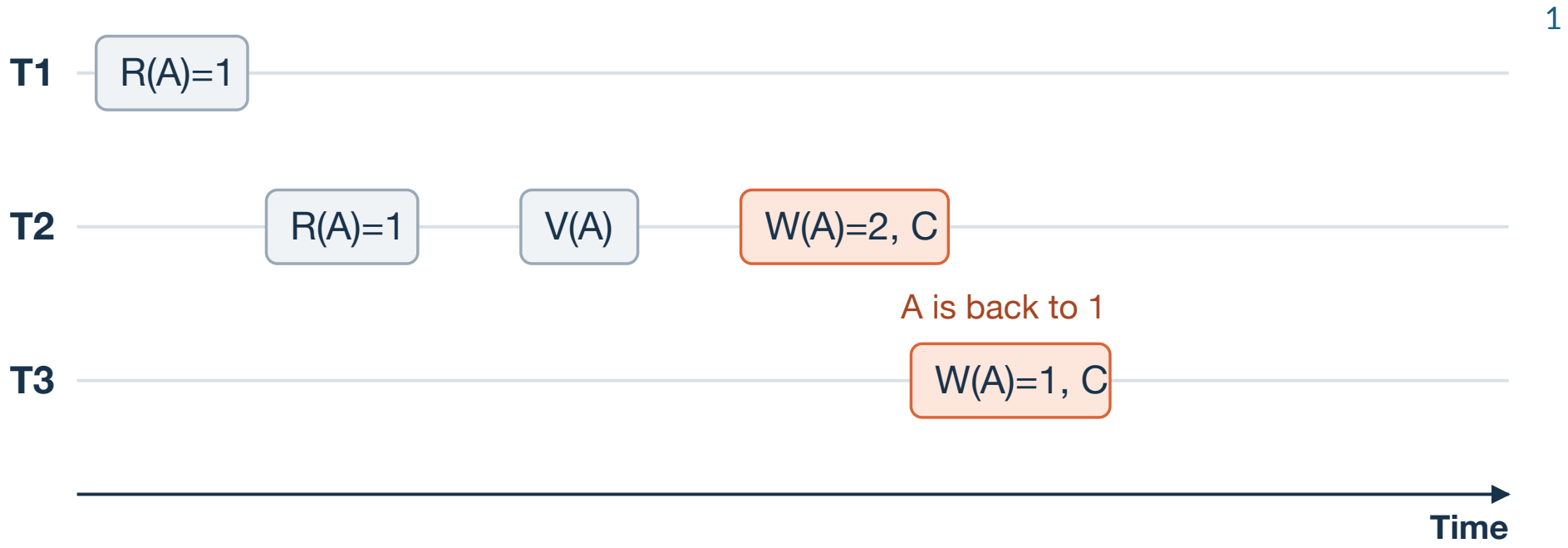
T1 and T2 both run `A += 1`, T3 runs `A = 1`; A initially is 1



[1] A = 1 to start with; R = read, V = validate, W = write, C = commit

The ABA problem

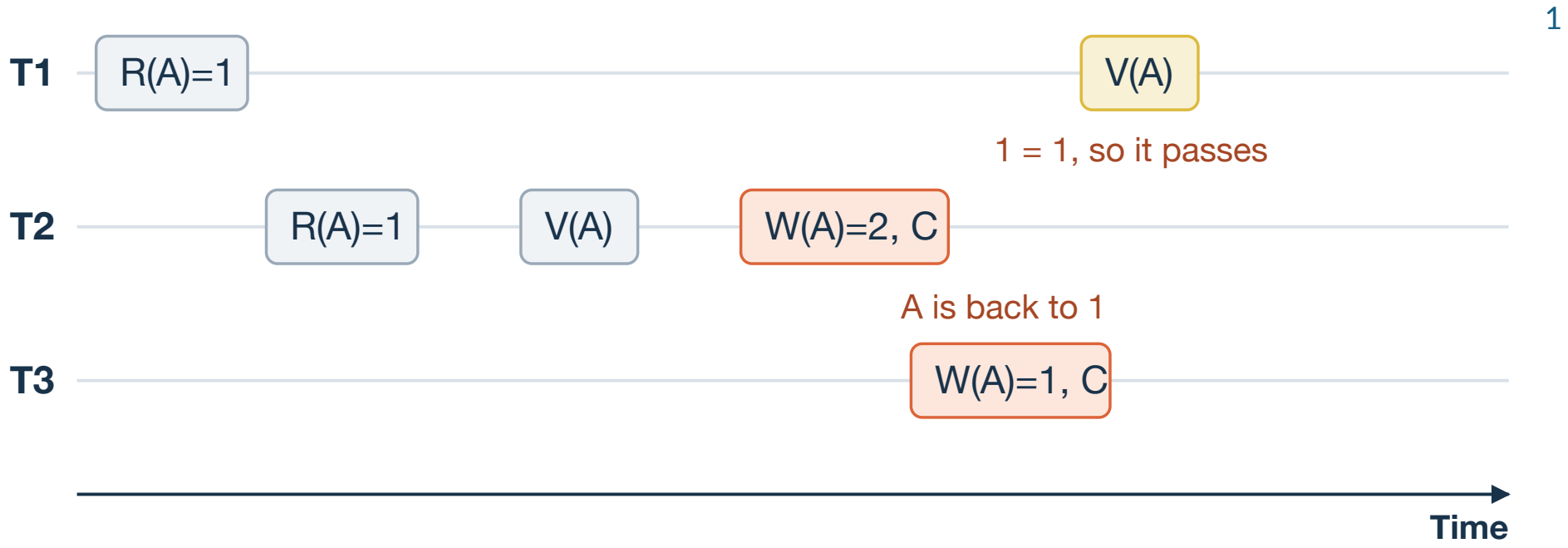
T1 and T2 both run `A += 1`, T3 runs `A = 1`; A initially is 1



[1] A = 1 to start with; R = read, V = validate, W = write, C = commit

The ABA problem

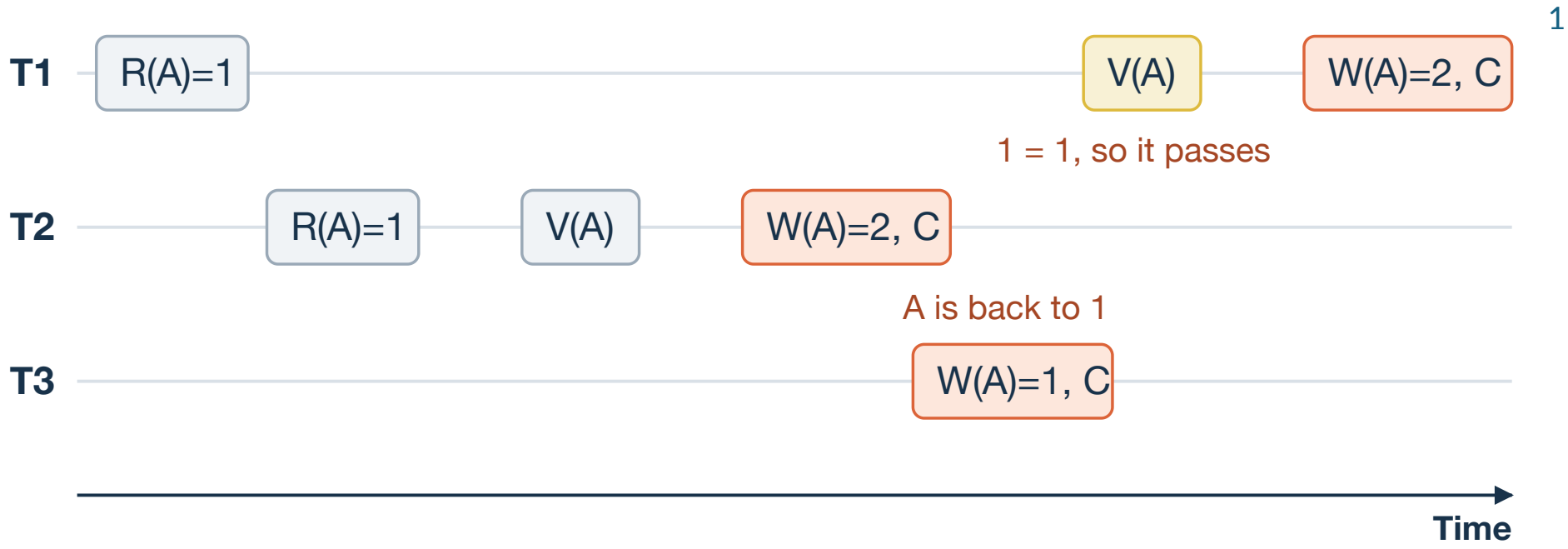
T1 and T2 both run `A += 1`, T3 runs `A = 1`; A initially is 1



[1] A = 1 to start with; R = read, V = validate, W = write, C = commit

The ABA problem

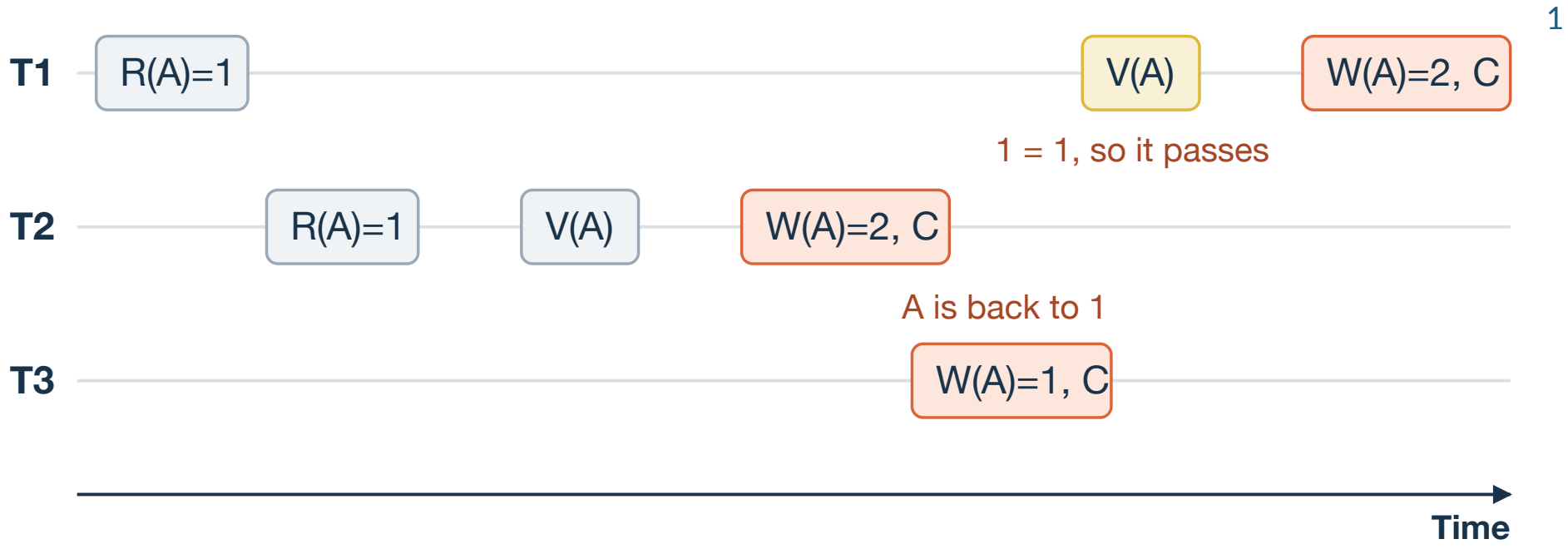
T1 and T2 both run `A += 1`, T3 runs `A = 1`; A initially is 1

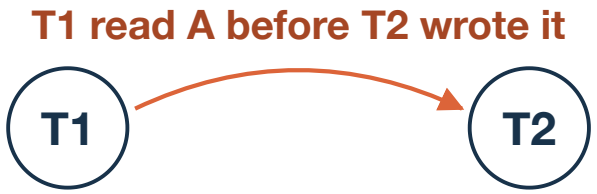


[1] A = 1 to start with; R = read, V = validate, W = write, C = commit

The ABA problem

T1 and T2 both run `A += 1`, T3 runs `A = 1`; A initially is 1

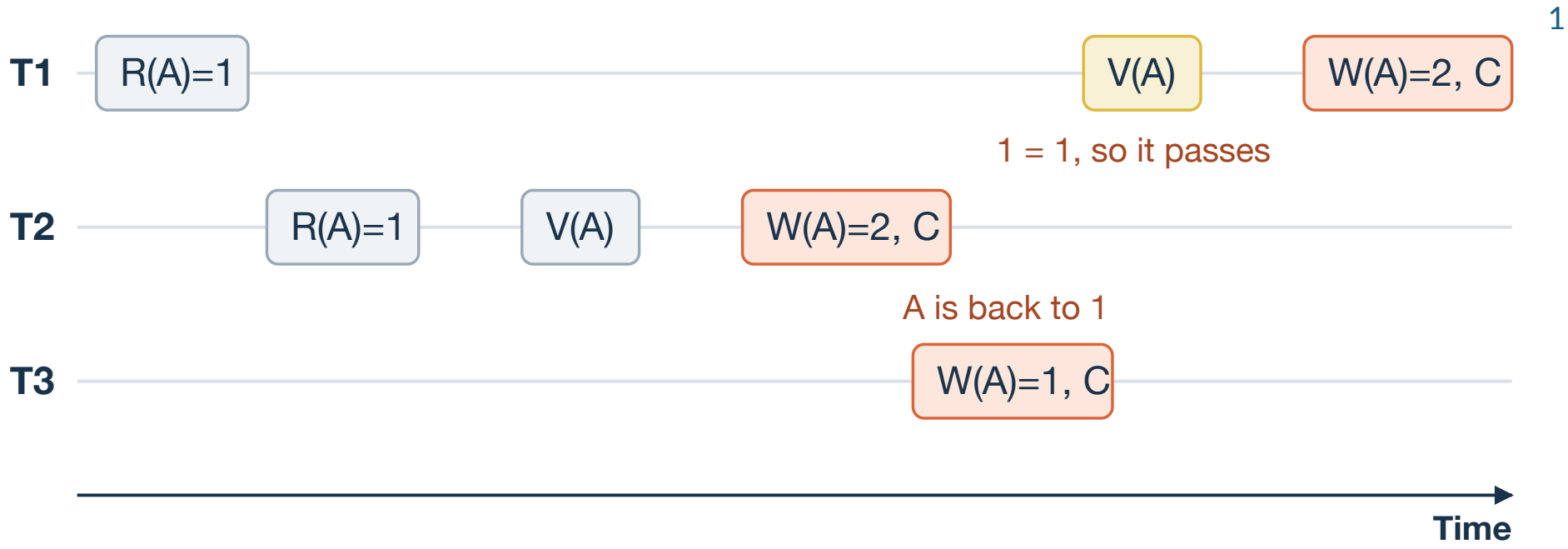


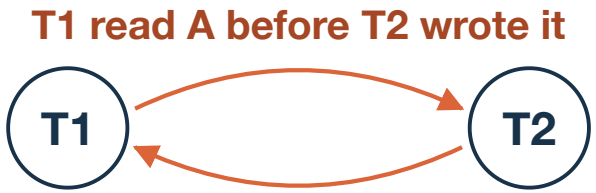


[1] A = 1 to start with; R = read, V = validate, W = write, C = commit

The ABA problem

T1 and T2 both run `A += 1`, T3 runs `A = 1`; A initially is 1





a cycle:
no serial order

[1] A = 1 to start with; R = read, V = validate, W = write, C = commit

How to implement the critical section for phase 2 & 3?

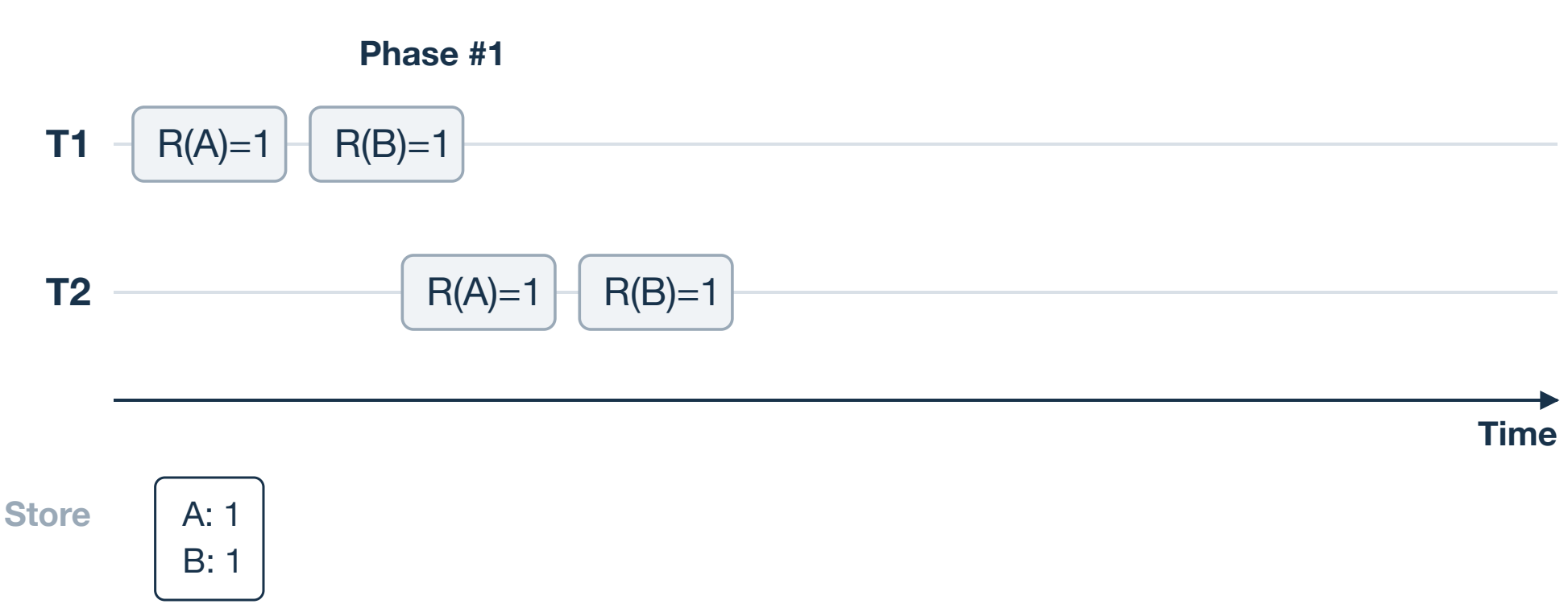
A global lock may work

- Why is there still concurrency? Phase 2 & 3 are typically short
  - No TX logic is executed, only the validation

```
def validate_and_commit() // phase 2 & 3
    global_lock.lock()
    for d in read-set:
        if d has changed:
            abort()
    for d in write-set:
        write(d)
    global_lock.unlock()
```

Re-run the two TXs with the global lock

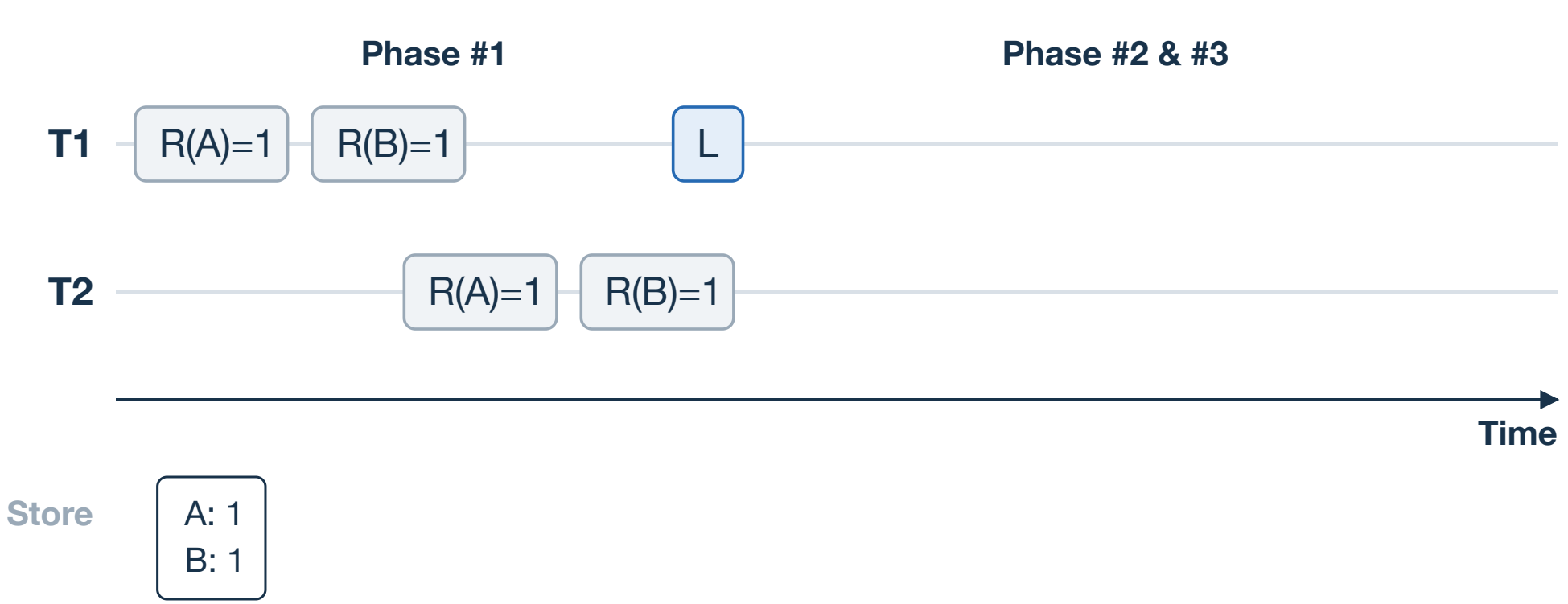
Both TXs run `A += 1; B += 1`, same schedule as before



[1] L = lock, U = unlock; an abort releases the lock as well

Re-run the two TXs with the global lock

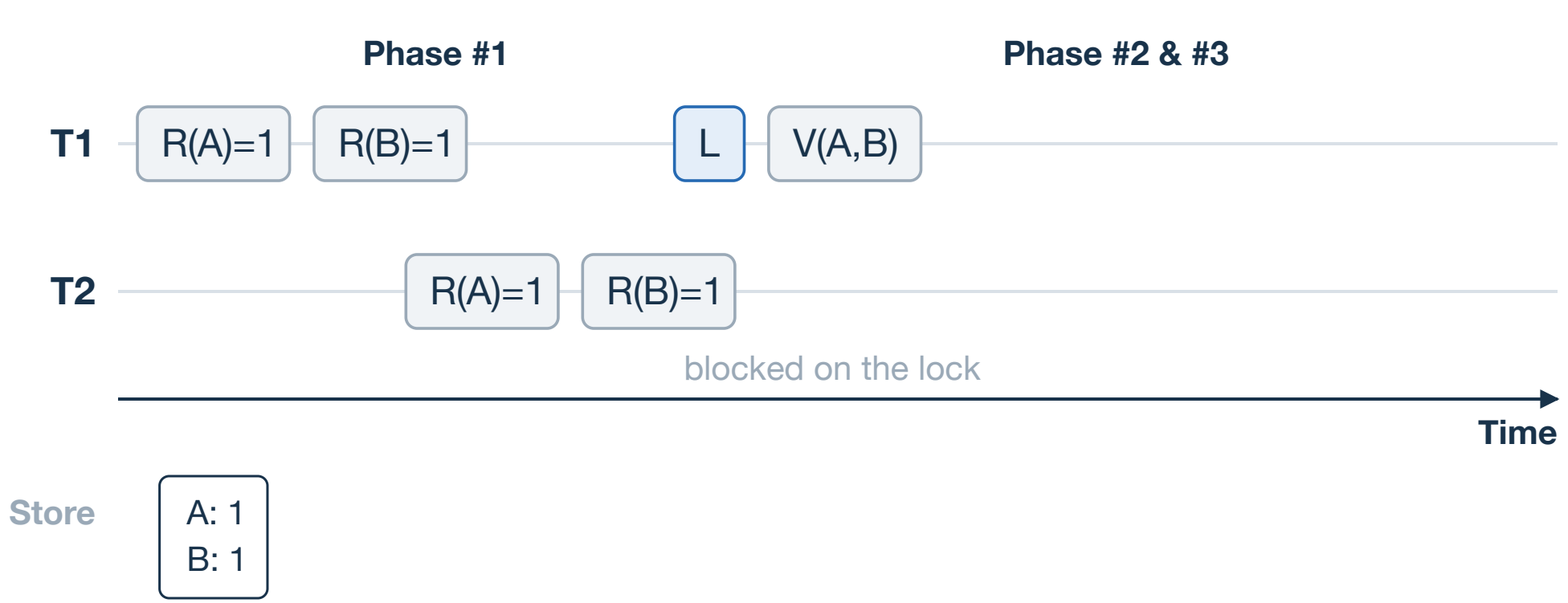
Both TXs run `A += 1; B += 1`, same schedule as before



[1] L = lock, U = unlock; an abort releases the lock as well

Re-run the two TXs with the global lock

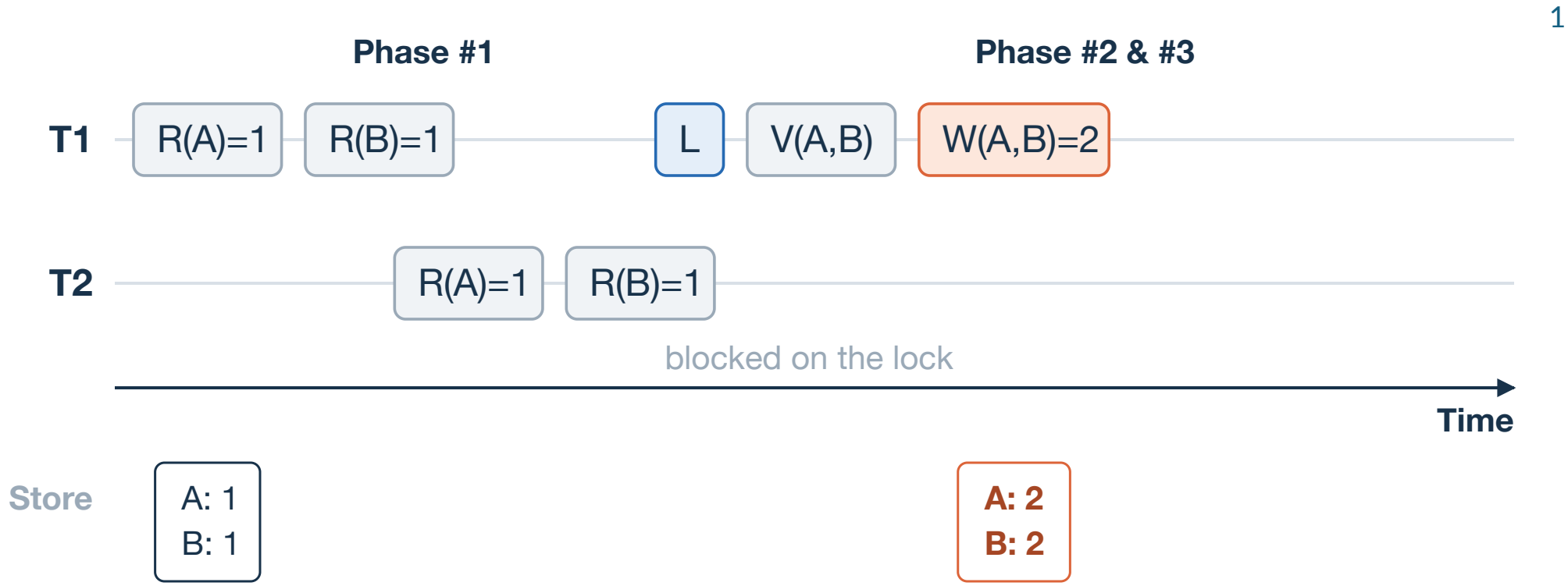
Both TXs run `A += 1; B += 1`, same schedule as before



[1] L = lock, U = unlock; an abort releases the lock as well

Re-run the two TXs with the global lock

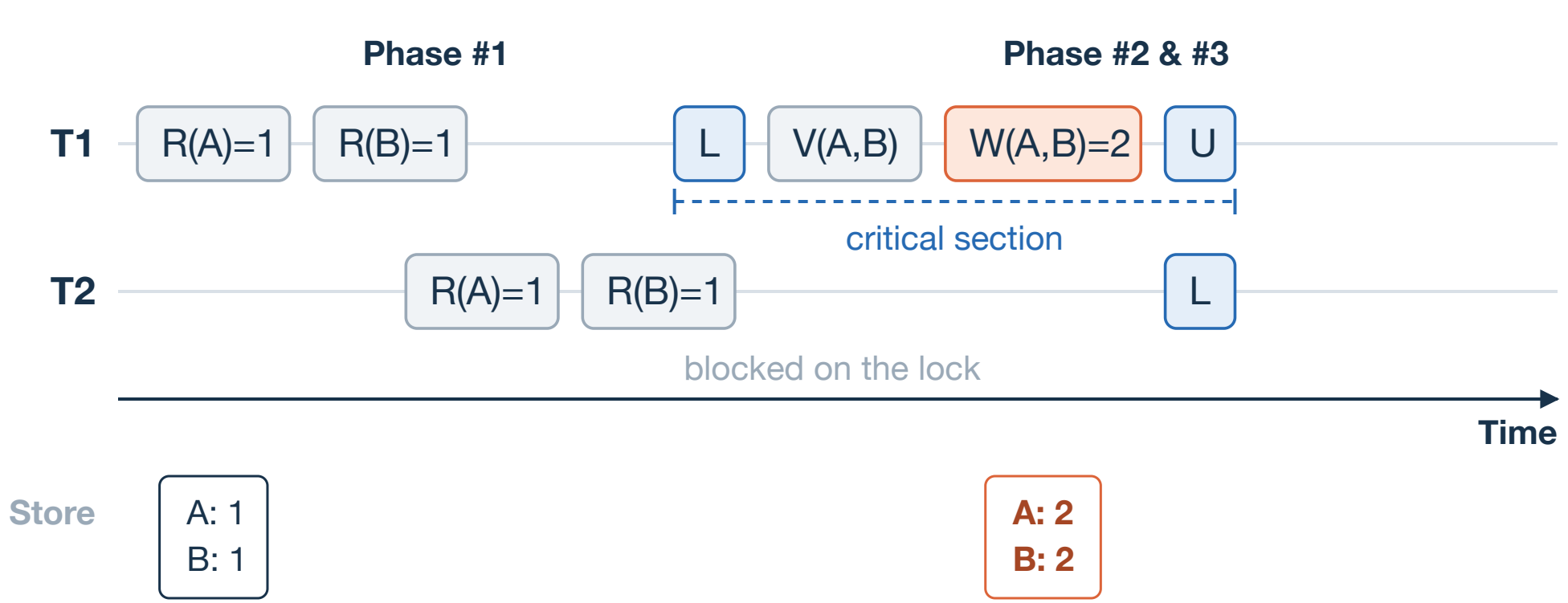
Both TXs run `A += 1; B += 1`, same schedule as before



[1] L = lock, U = unlock; an abort releases the lock as well

Re-run the two TXs with the global lock

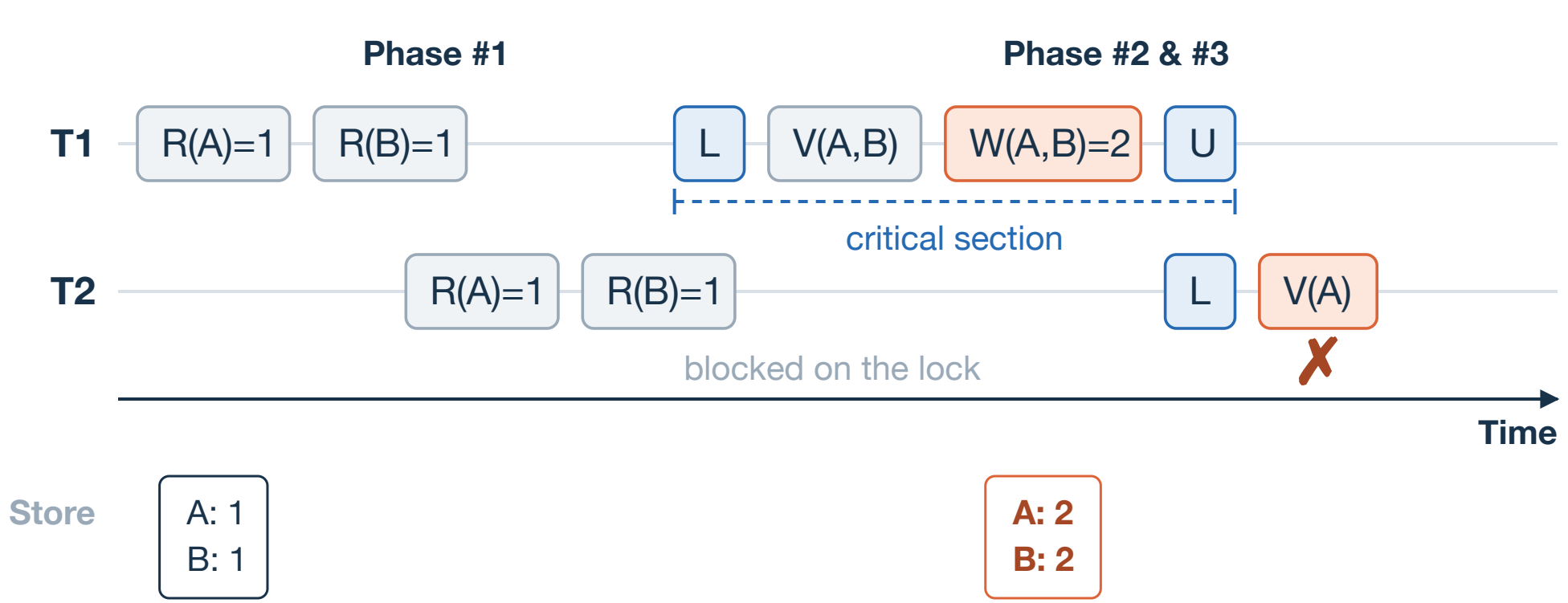
Both TXs run `A += 1; B += 1`, same schedule as before



[1] L = lock, U = unlock; an abort releases the lock as well

Re-run the two TXs with the global lock

Both TXs run `A += 1; B += 1`, same schedule as before

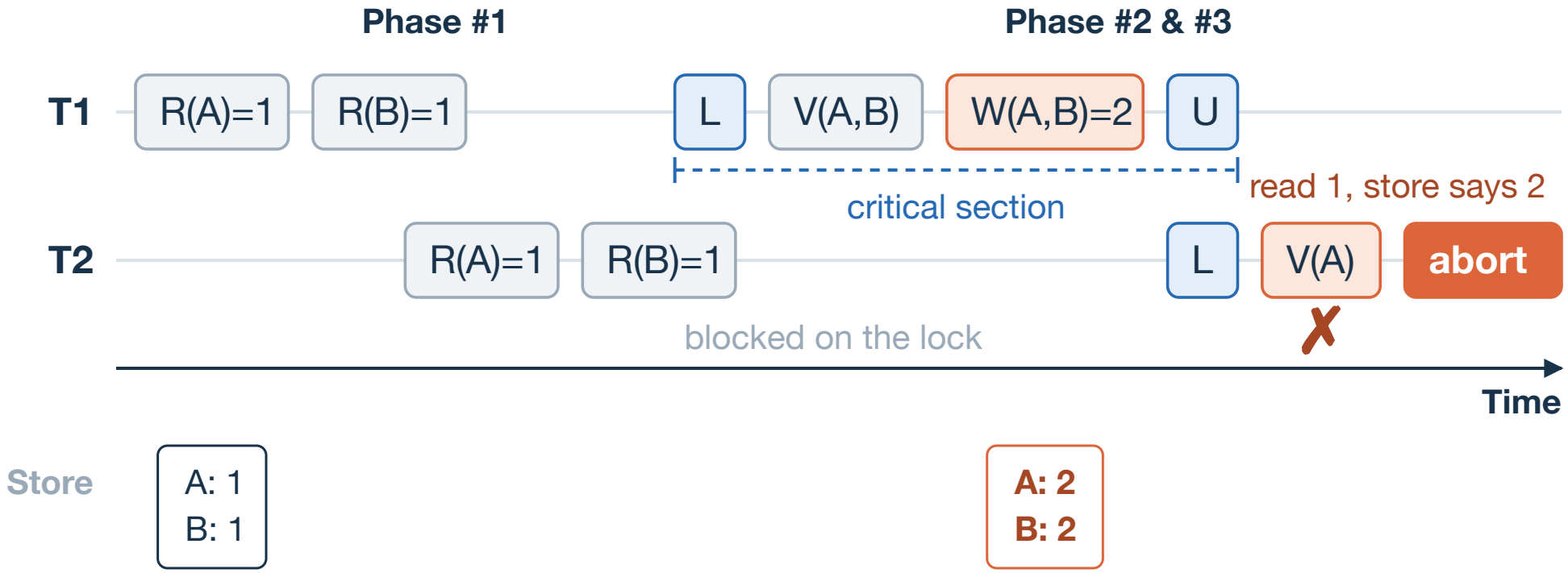


[1] L = lock, U = unlock; an abort releases the lock as well

Re-run the two TXs with the global lock

Both TXs run `A += 1; B += 1`, same schedule as before

[1]



T2 cannot overwrite T1 — it aborts and retries

[1] L = lock, U = unlock; an abort releases the lock as well

Improvement: enabling more concurrency for OCC

Using two-phase locking

- Allow more concurrency to improve performance
- What about the problem of deadlock?

```
def validate_and_commit() // phase 2 & 3
    for d in read-set:
        d.lock()
        if d has changed:
            abort() // abort will release all the held lock
    for d in write-set:
        d.lock()
        write(d)
    // release the locks
    ...
```

Improvement: enabling more concurrency for OCC

Using two-phase locking

- Allow more concurrency
- Use sort to avoid deadlock; but can we do better?

```
def validate_and_commit() // phase 2 & 3
    for d in sorted(read-set + write-set):
        d.lock()
    for d in read-set:
        if d has changed:
            abort()
    for d in write-set:
        write(d)
    // release the locks
    ...
```

Question: do we need to lock the read set?

How to implement the critical section for phase 2 & 3?

Observation (for the read set):

- If the validation passes, then it “appears” that a lock is held during

```
def validate_and_commit() // phase 2 & 3
    for d in sorted(read-set + write-set):
        d.lock()
    for d in read-set:
        if d has changed:
            abort()
    for d in write-set:
        write(d)
    // release the locks
    ...
```

So the read set can be dropped from the list of things we lock

How to implement the critical section for phase 2 & 3?

Observation (for the read set):

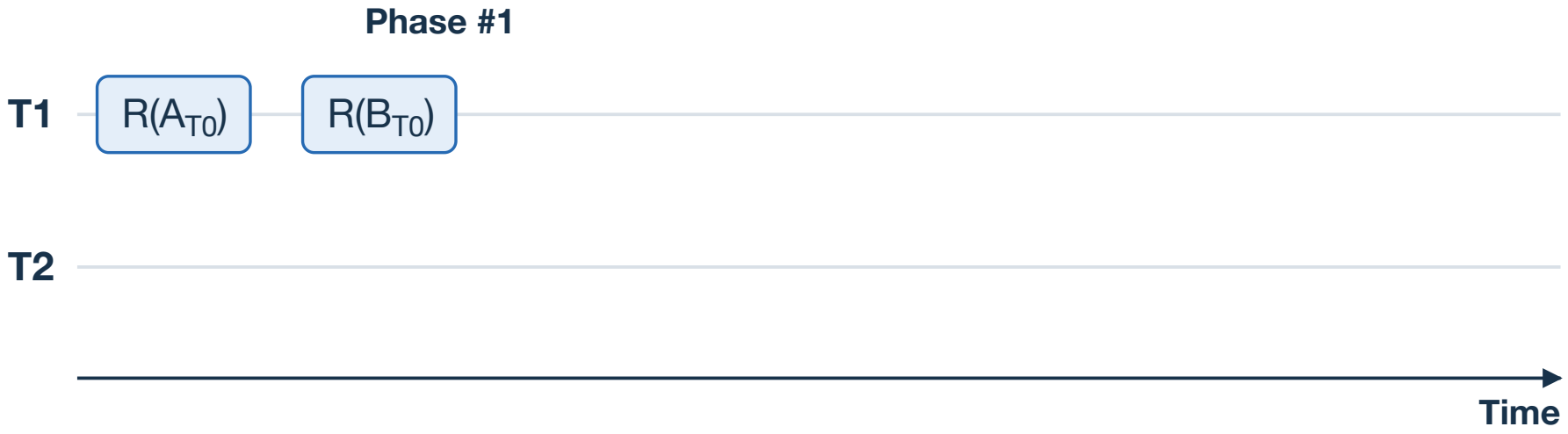
- If the validation passes, then it “appears” that a lock is held during

```
def validate_and_commit() // phase 2 & 3
    for d in sorted(write-set):
        d.lock()
    for d in read-set:
        if d has changed:
            abort()
    for d in write-set:
        write(d)
    // release the locks
    ...
```

Question: is this implementation correct?

Problem: read-write conflict

T1 runs `A += B`, T2 runs `B += A`



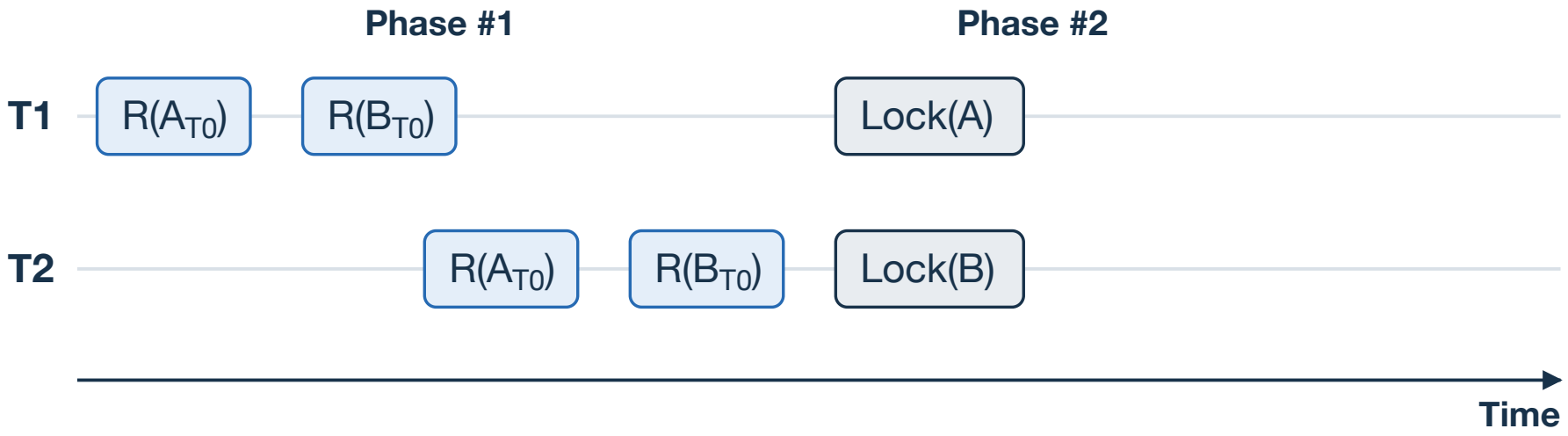
Problem: read-write conflict

T1 runs `A += B`, T2 runs `B += A`



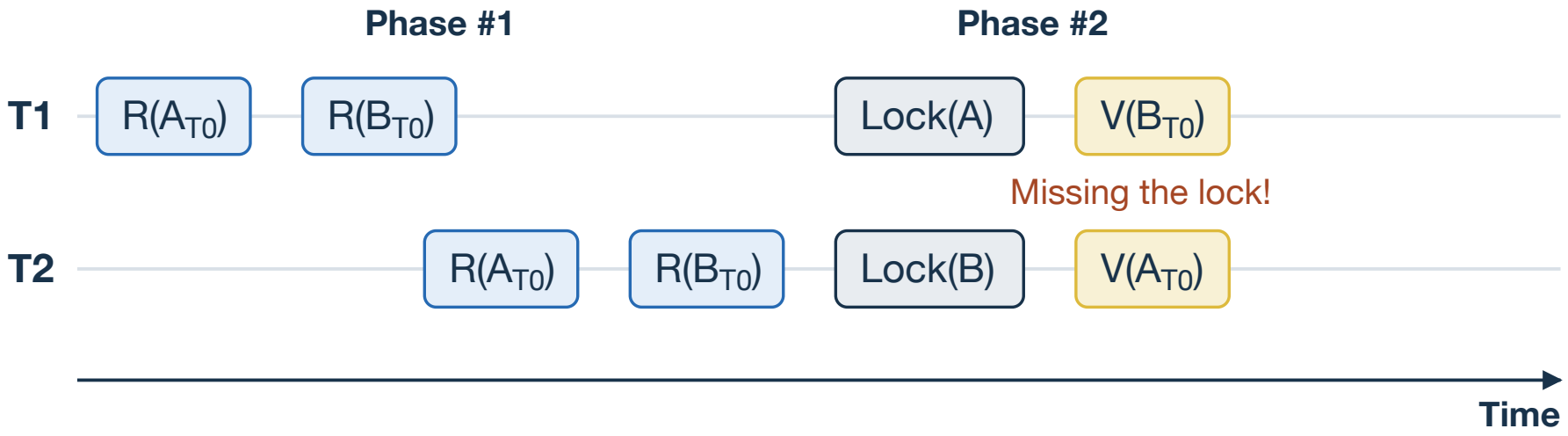
Problem: read-write conflict

T1 runs `A += B`, T2 runs `B += A`



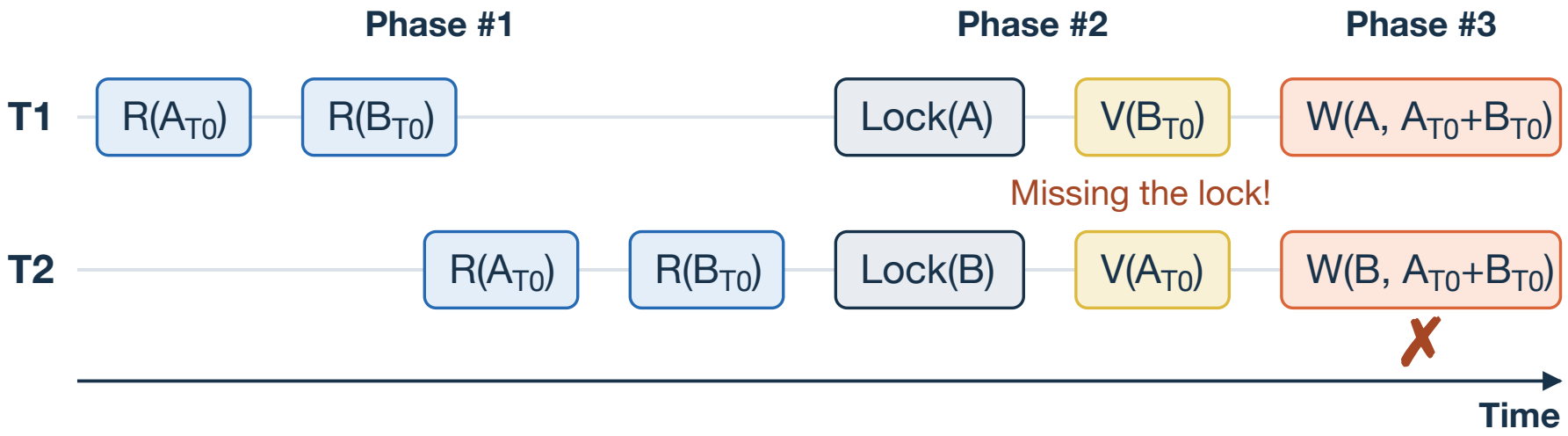
Problem: read-write conflict

T1 runs `A += B`, T2 runs `B += A`



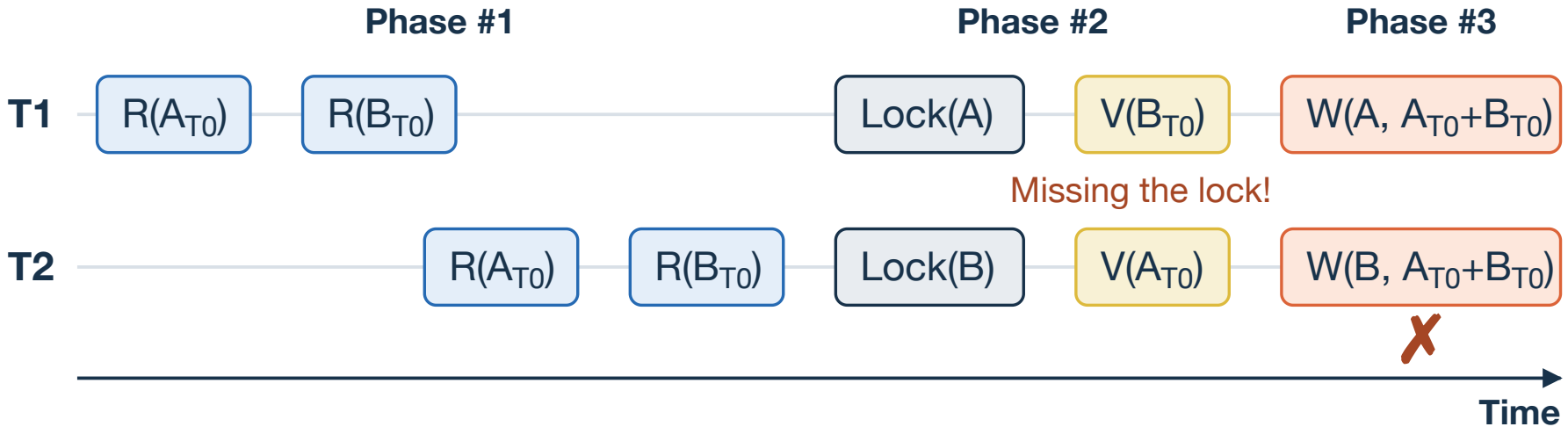
Problem: read-write conflict

T1 runs `A += B`, T2 runs `B += A`



Problem: read-write conflict

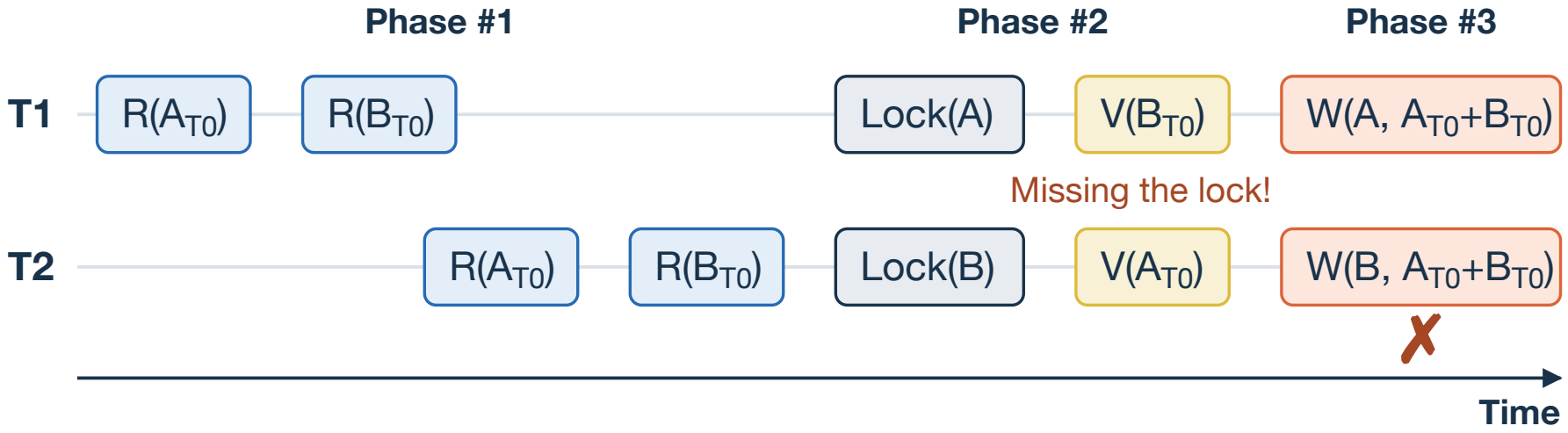
T1 runs `A += B`, T2 runs `B += A`

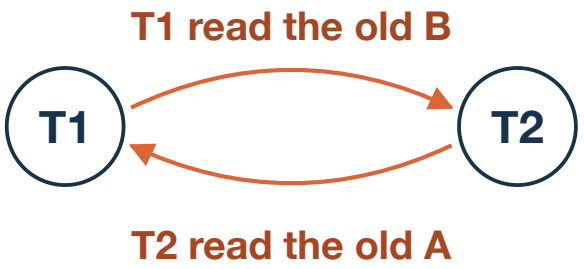




Problem: read-write conflict

T1 runs `A += B`, T2 runs `B += A`





a cycle:
no serial order

How to implement the critical section for phase 2 & 3?

Use two-phase locking + read validation

- Only lock the write set, after that,
- Check the read set has not changed & is not locked

```
def validate_and_commit() // phase 2 & 3 with before-or-after
    for d in sorted(write-set):
        d.lock()
    for d in read-set:
        if d has changed or d has been locked:
            abort()
    for d in write-set:
        write(d)
    // release the locks
    ...
```

OCC Advantages

Phase 1: Concurrent local processing of the transaction

- Reads data into a read set
- Buffers writes into a write set

Phase 2: Validation in a (typically small) critical section

- Validates whether serializability is guaranteed:
- Has any data in the read set been modified?

Phase 3: Commit the results in critical section or abort

- Aborts: aborts the transaction if validation fails
- Commits: installs the write set and commits the transaction

Needs synchronization (e.g., with lock), but usually very short at low contention

OCC Advantages (More)

Use reads more than writes (for each read-only data access)

- OCC (in the optimal case, i.e., no abort)
  - 1 read to read the data value
  - 1 read to validate whether the value has been changed or not (as well as locked)
- 2PL
  - 1 operation to acquire the lock (typically an atomic CAS)
  - 1 read to read the data value
  - 1 write to release the lock
    - A single CPU write is atomic, no need to do the atomic CAS

Locking is costly especially compared to reads

Locking preliminary

Many implementations exist to achieve the best performance & scalability

- Still an active topic in system research

We will focus on the basic design

- The details (& advanced methods) will be left to the OS class

Basic impl: using a flag to indicate whether the lock is held

- Acquire: read the current flag, if not locked, then set it as locked

Interestingly, acquiring the lock itself also requires before-or-after

- Both threads will read the current lock status
- If the status is not locked, update it to locked

What if two threads concurrently read an unlocked state?

Locking preliminary: atomic compare and swap

Hardware primitive to guarantee before-or-after atomicity

- `Compare-and-swap` (on SPARC)
- `Compare-and-exchange` (on x86)
- Lock prefix to ensure an instruction is atomically executed on a memory address

```
int CompareAndSwap(int *ptr, int expected, int new) { // with the lock prefix
    int actual = *ptr;
    if (actual == expected)
        *ptr = new;
    return actual;
} // hardware ensures atomicity with the lock prefix
```

```
; on x86 the function above is one instruction
mov      eax, expected
lock cmpxchg  DWORD PTR [ptr], new   ; without the prefix: read, compare, store
                                     ; -- 3 steps, another core can cut in
```

Locking (Spin Lock) using Compare-and-swap

```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) {
    // 0 indicates that lock is available, 1 that it is held
    lock->flag = 0;
}

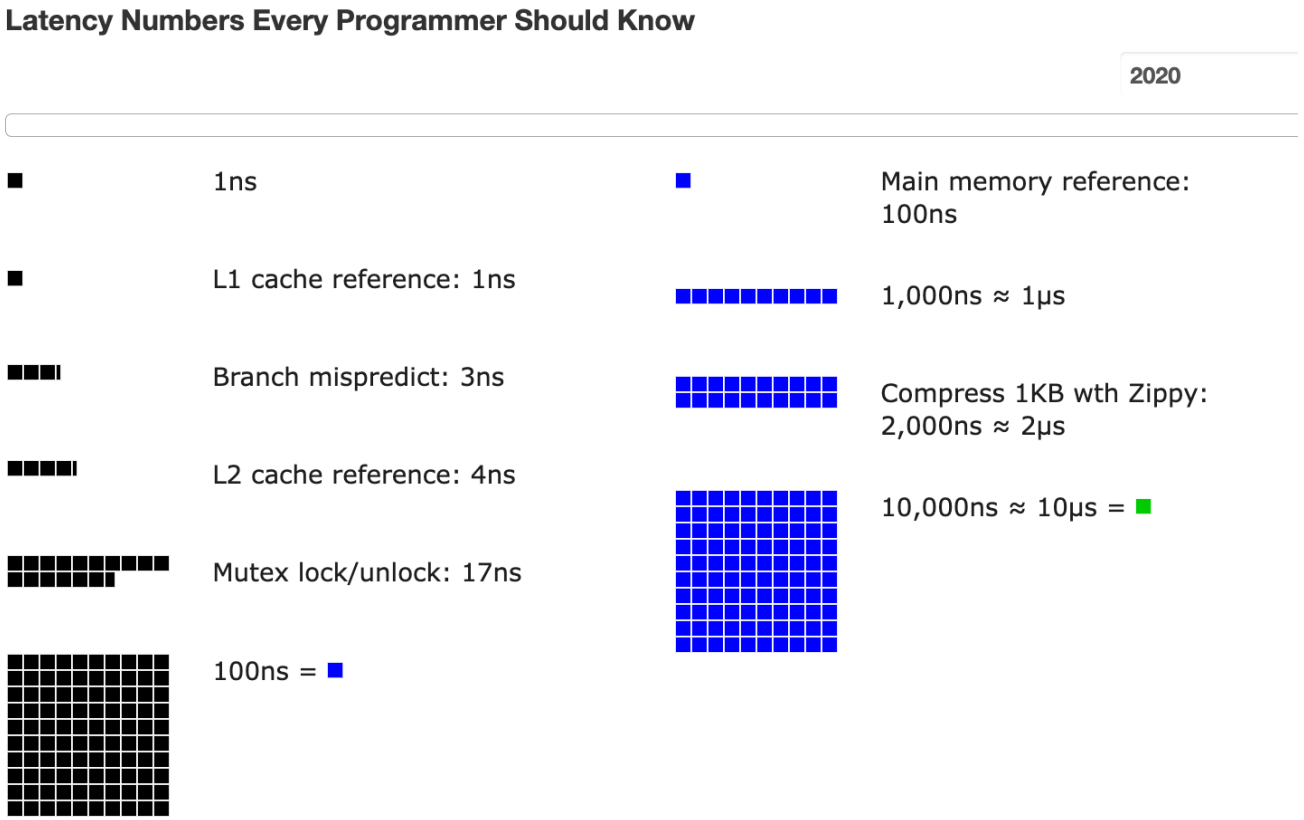
void lock(lock_t *lock) {
    while (CompareAndSwap(&lock->flag, 0, 1) == 1)
        ; // spin-wait (do nothing)
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

Atomic operations are costly for spin lock

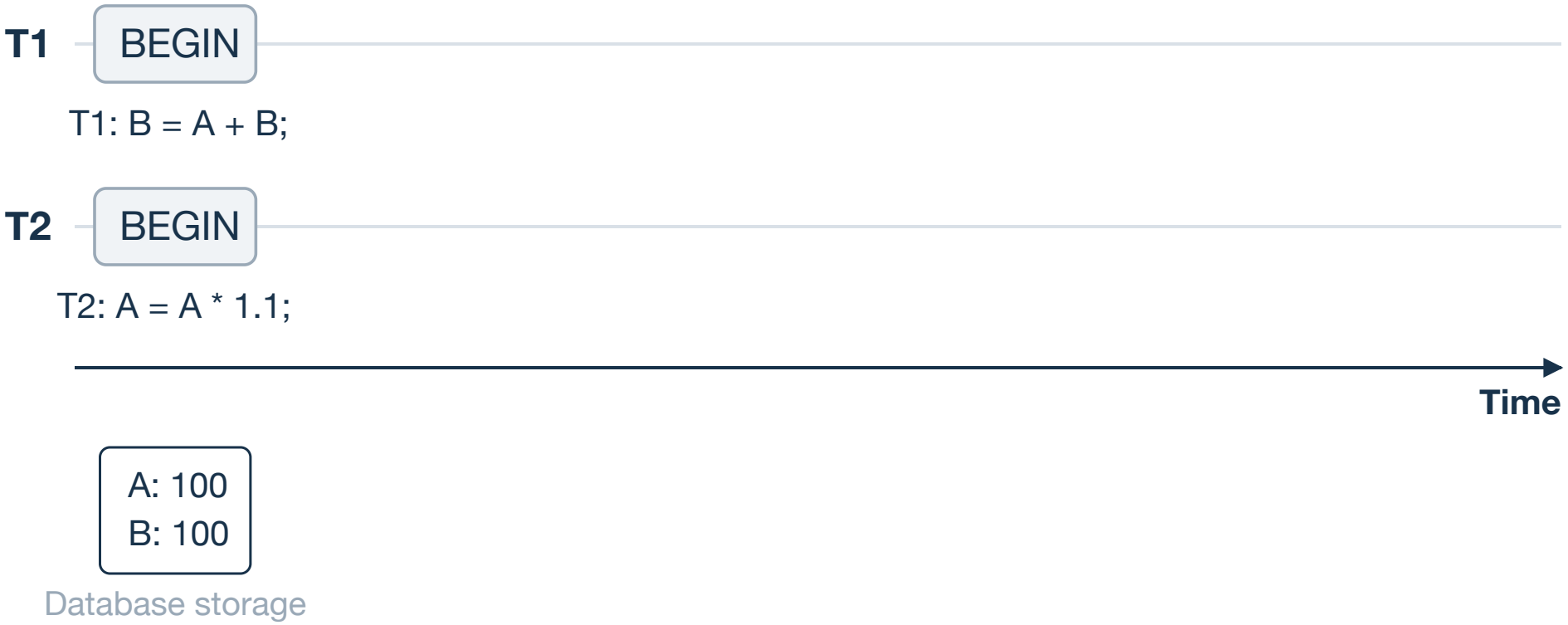
An L1 cache reference is 1 ns; a mutex lock/unlock pair is 17 ns

[1]

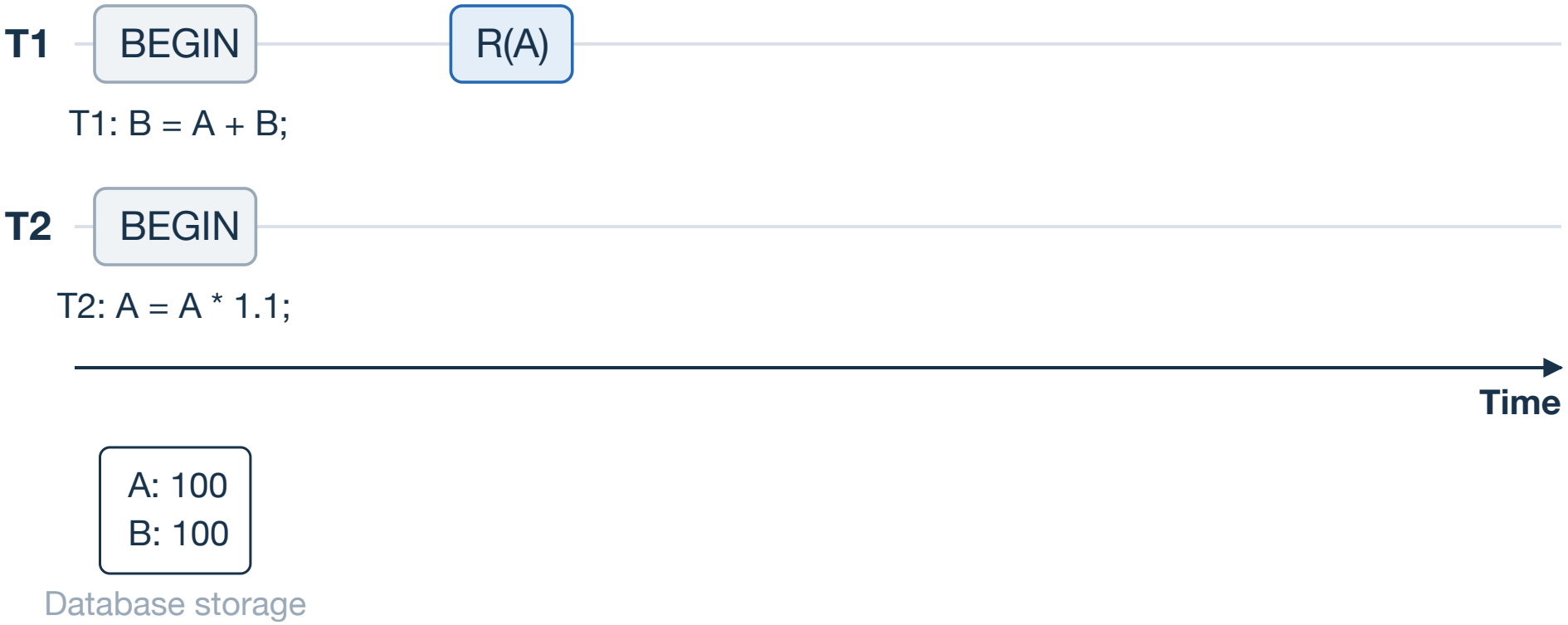


[1] Source: Numbers Every Programmer Should Know By Year

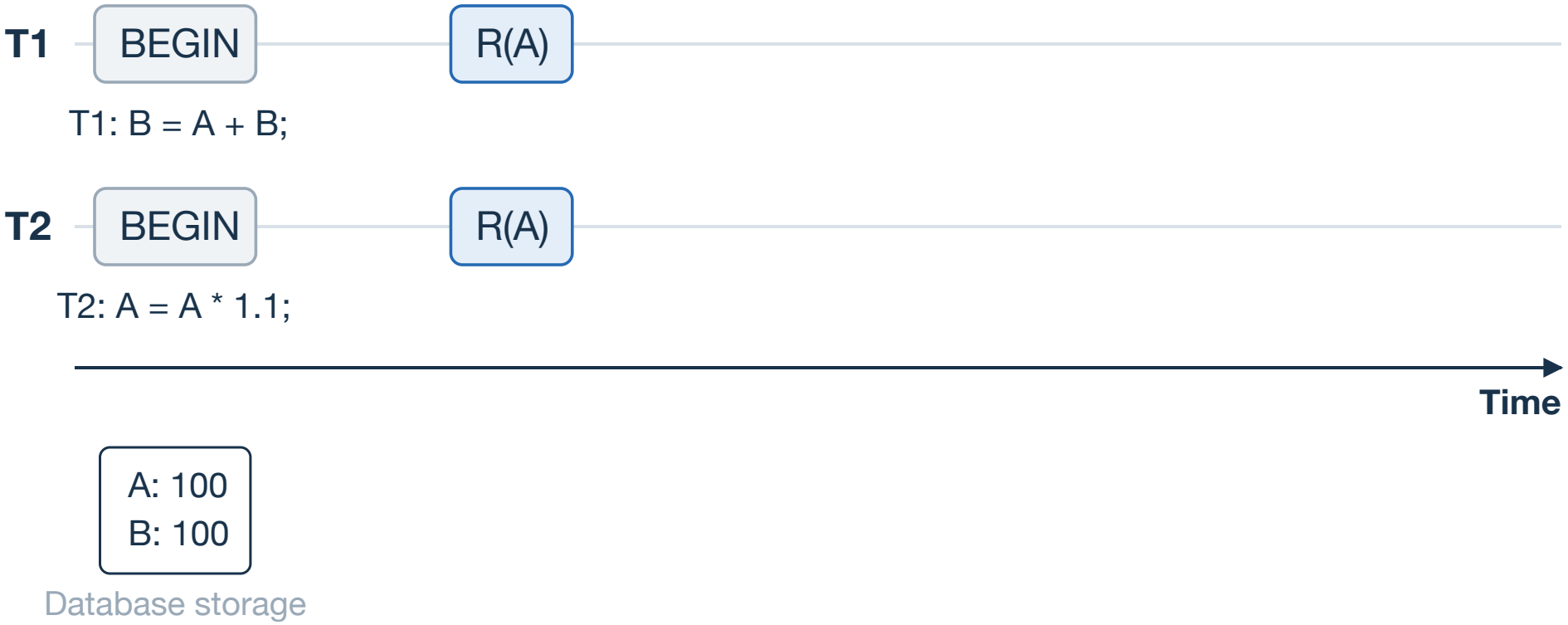
Put it all together: OCC in an example



Put it all together: OCC in an example



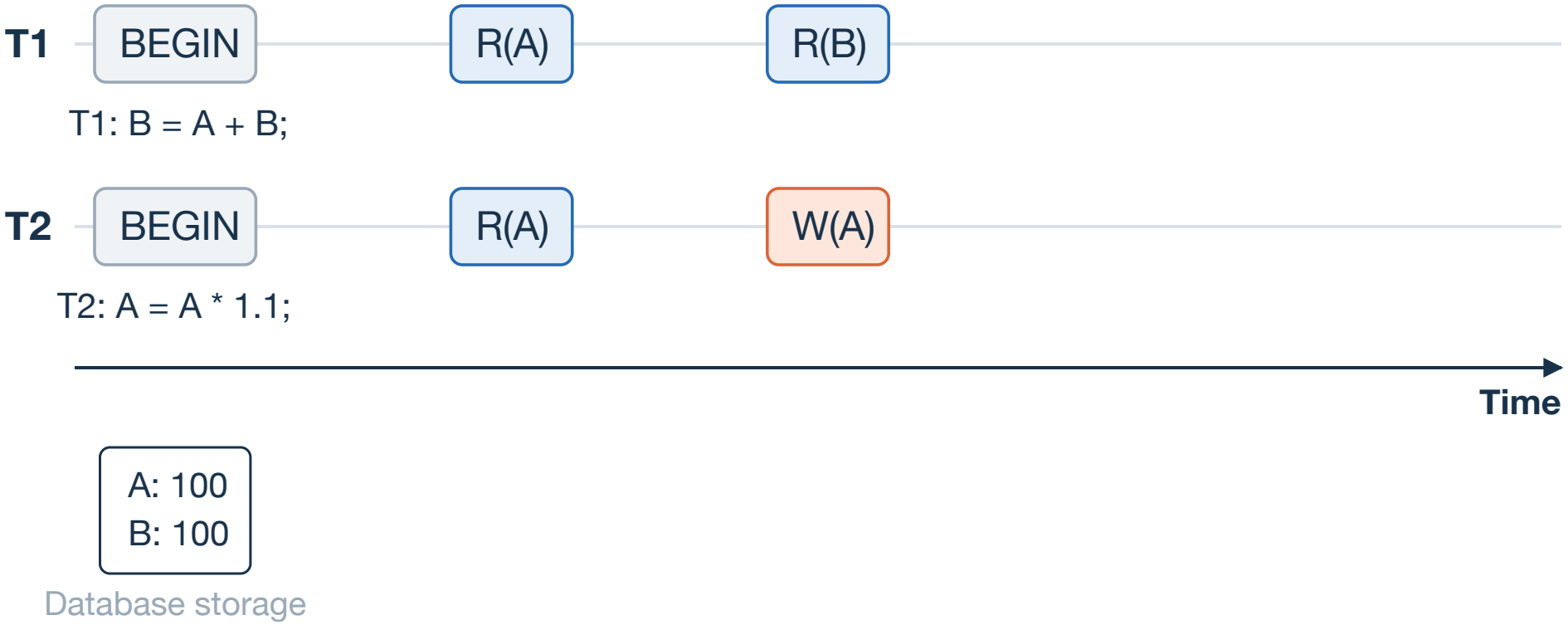
Put it all together: OCC in an example



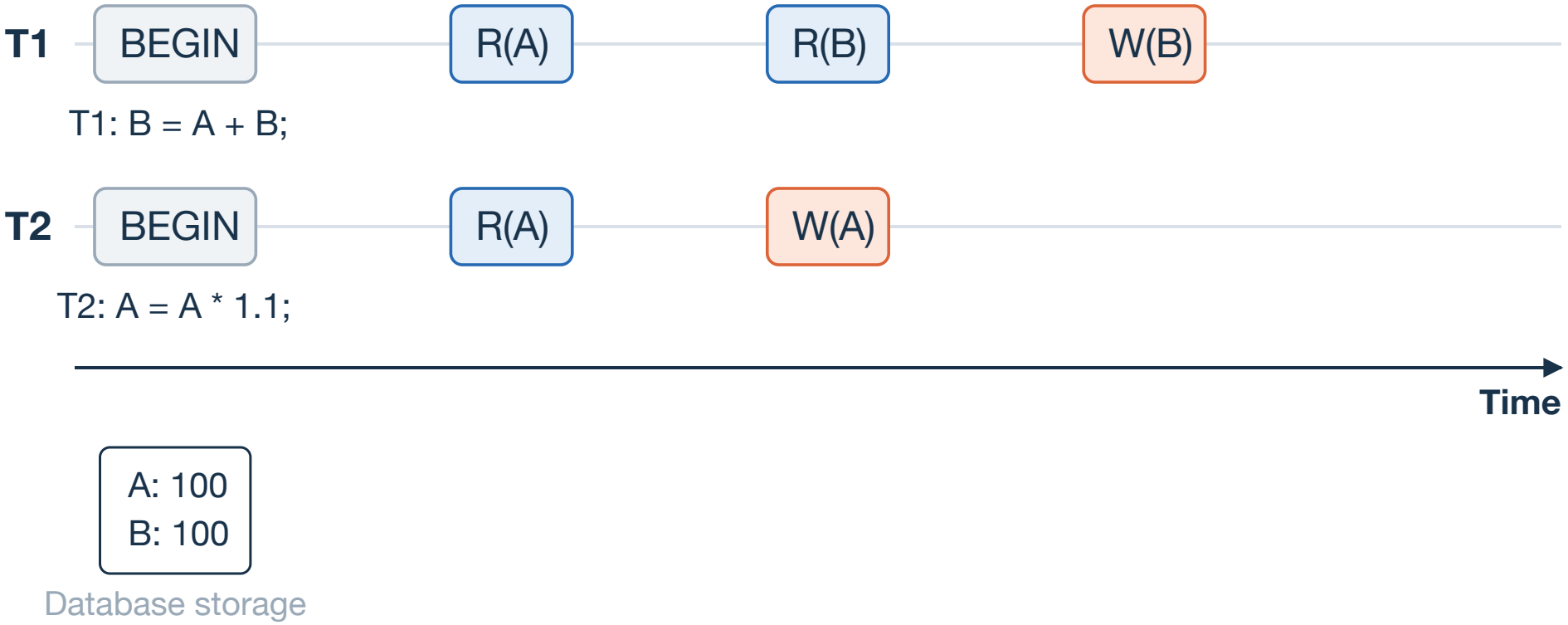
Put it all together: OCC in an example



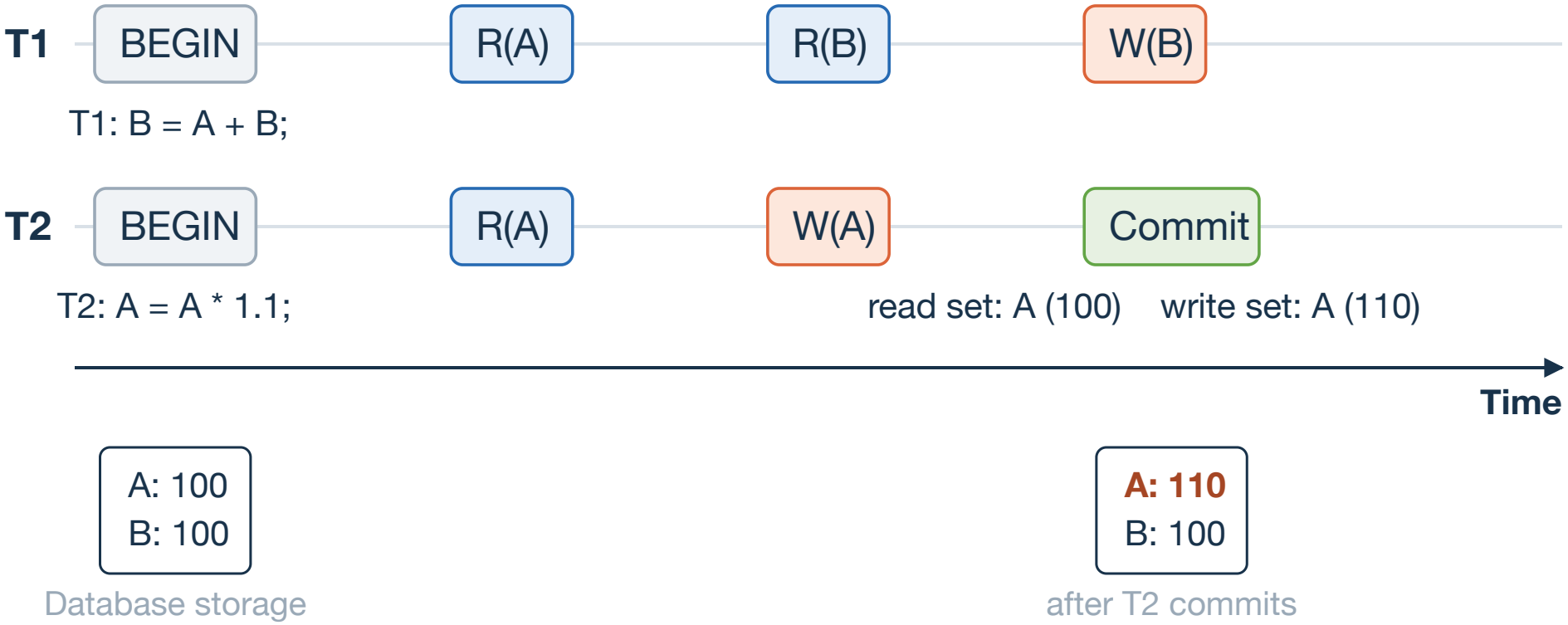
Put it all together: OCC in an example



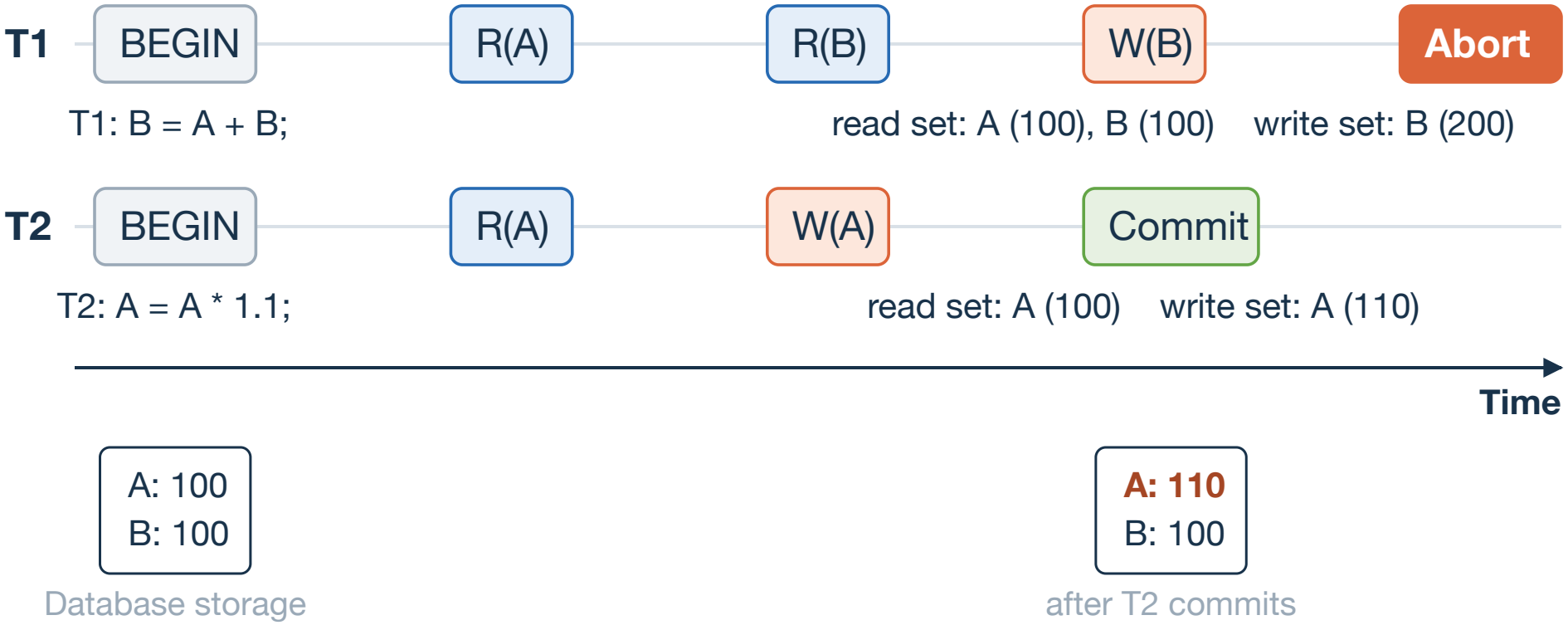
Put it all together: OCC in an example



Put it all together: OCC in an example



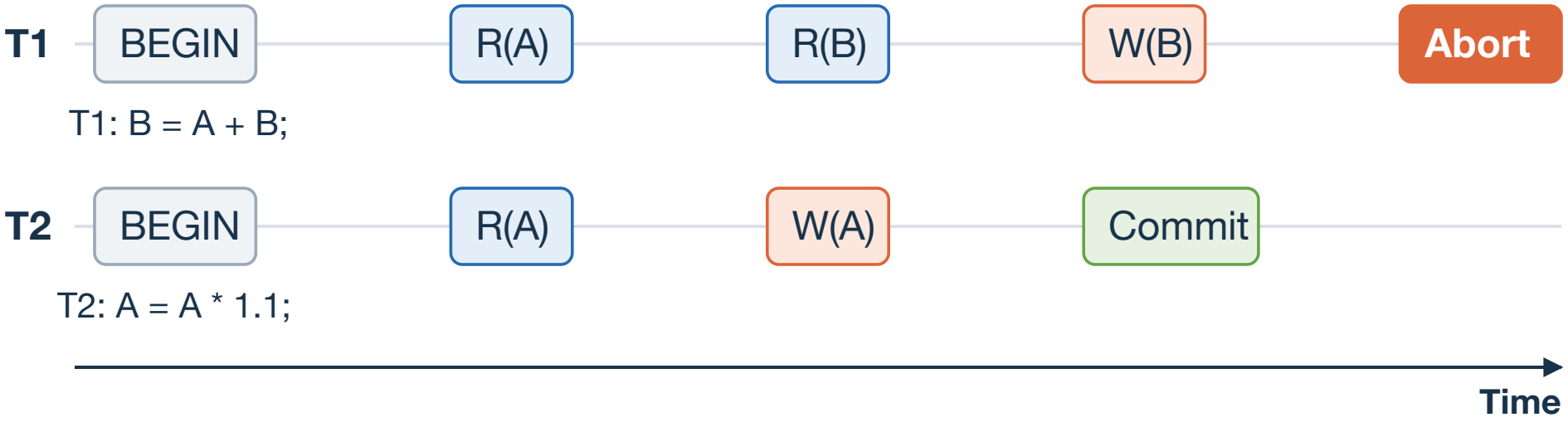
Put it all together: OCC in an example



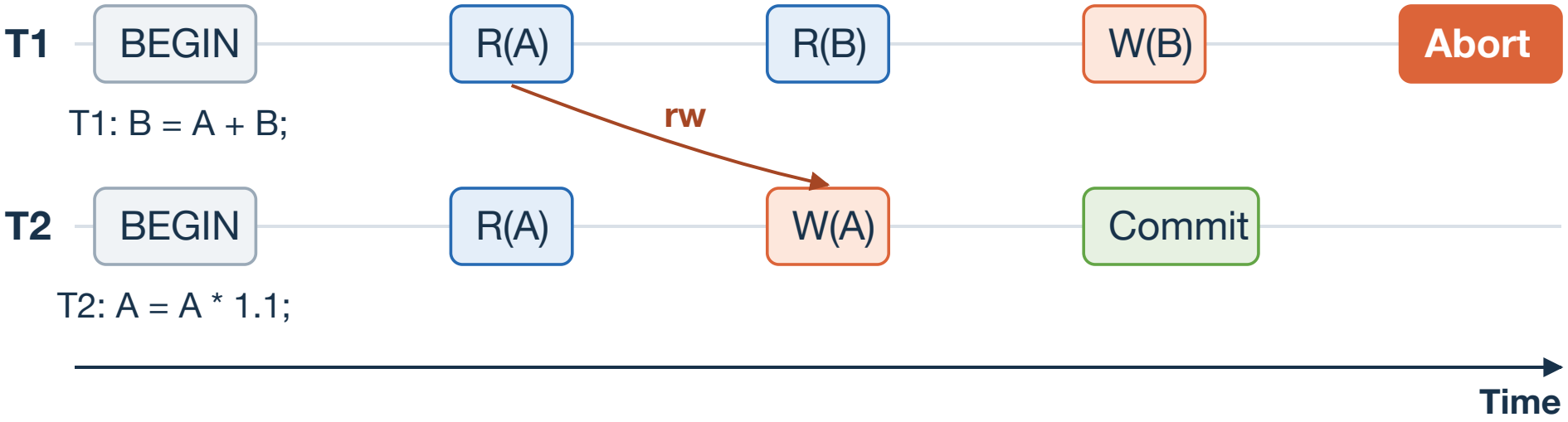
Tuple A in T1's read set has been changed by T2, thus T1 must be aborted.

T2's read set remains unchanged, thus T2 is allowed to commit

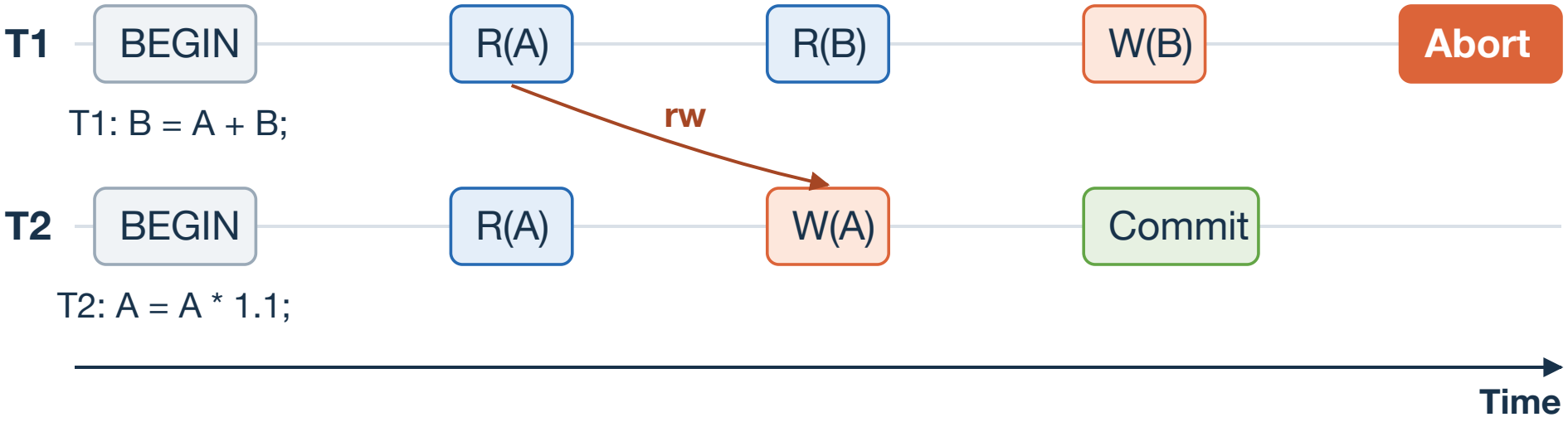
OCC's Problem: False Aborts

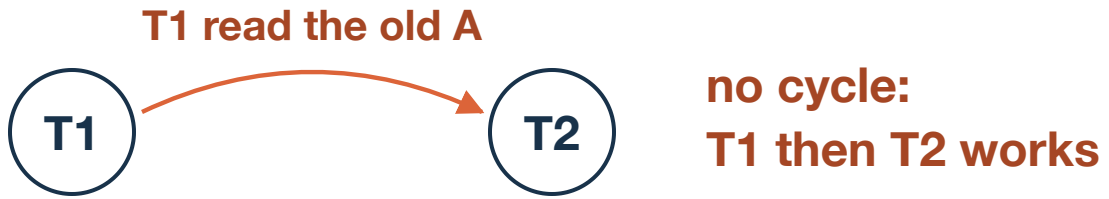


OCC's Problem: False Aborts



OCC's Problem: False Aborts





no cycle:
T1 then T2 works

T1 is falsely aborted: OCC can only serialize T2, T1

OCC's Problem: False Aborts & Livelock

Some transactions aborted by OCC could have been allowed to commit, i.e., without causing an unserializable schedule

False aborts are problematic especially for TXs with a lot of reads

- E.g., read a lot of items → large read set → easier to abort
- May abort *even under low contention*

Under high contention, the same TX may be aborted again and again — livelock

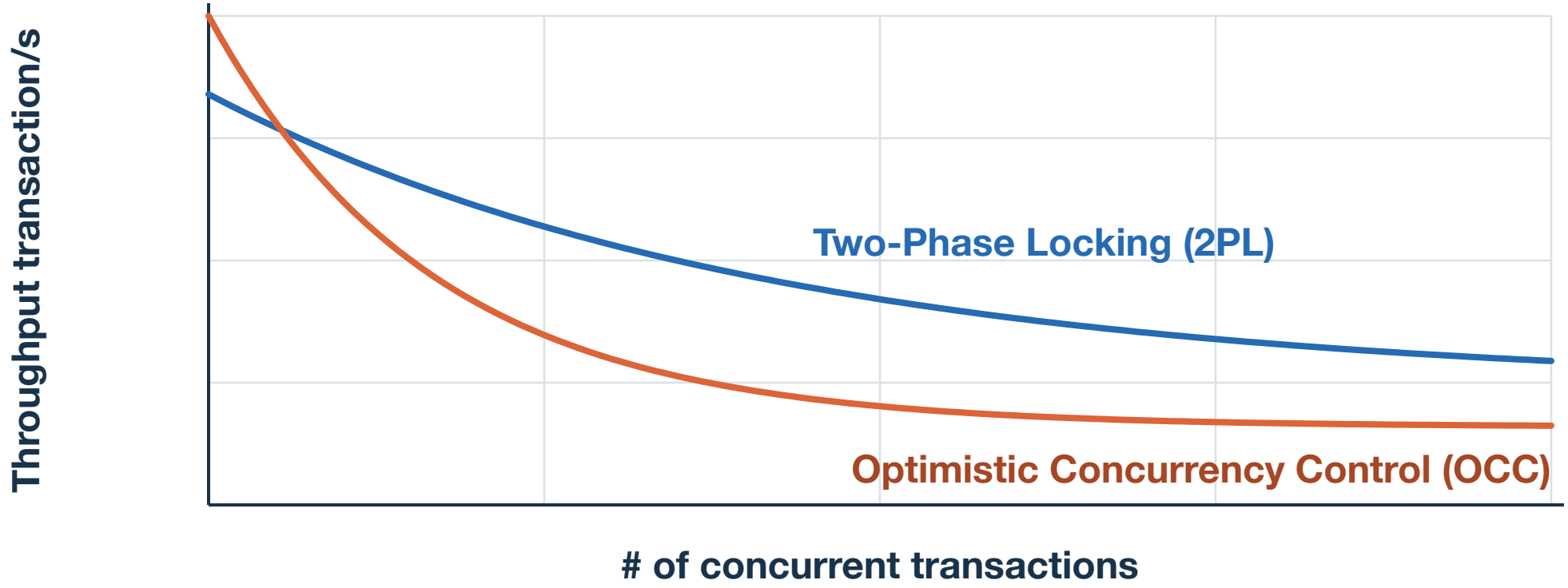
- Especially with large read/write sets, or long execution time

Transaction is executing, but no progress!

2PL vs. OCC: in a nutshell

OCC starts higher, but drops below 2PL once the transactions start to contend

[1]



[1] Source: Extracting More Concurrency from Distributed Transactions [OSDI'14]

Implementing 2PL & OCC is challenging.

Can we avoid it?

Hardware Transactional Memory

A new CPU hardware feature for writing multi-core concurrent programs

CPU guarantees the before-or-after atomicity of memory reads/writes

- i.e., no race conditions & no need for 2PL & OCC!
- Benefits: near zero costs if it executes normally + easy to program

Intel supports HTM named as RTM



- Restricted Transactional Memory
- First released in its Haswell processor

Other implementations also exist

- E.g., ARM Transactional Memory Extension (TME)

A little introduction to RTM's ISA

ISA support for transactional memory

Recall: Writing with TX is straightforward

- Use `TX.begin` to mark when a TX starts
- Use `TX.commit` to commit the TX

In RTM, the concept is similar (except with new instructions)

- Use `xbegin` to mark an RTM execution start
- Use `xend` to mark an RTM end

RTM code region ~= critical section

Programming with RTM

If transaction starts successfully

- Do work protected by RTM without locking/OCC

Fallback routine to handle abort event (RTM cannot guarantee always success)

- If it aborts, the system rolls back to xbegin and returns an abort code

Manually abort inside a transaction

```
if _xbegin() == _XBEGIN_STARTED:
    if conditions:
        _xabort()
    critical code              # user code, protected by RTM
    _xend()
else:
    fallback routine
```

Programming with RTM

If an HTM transaction starts successfully

- Do work protected by RTM, and then try to commit

Fallback routine to handle abort event

- If it aborts, the CPU rolls back to xbegin, returns an abort code, and goes to the fallback

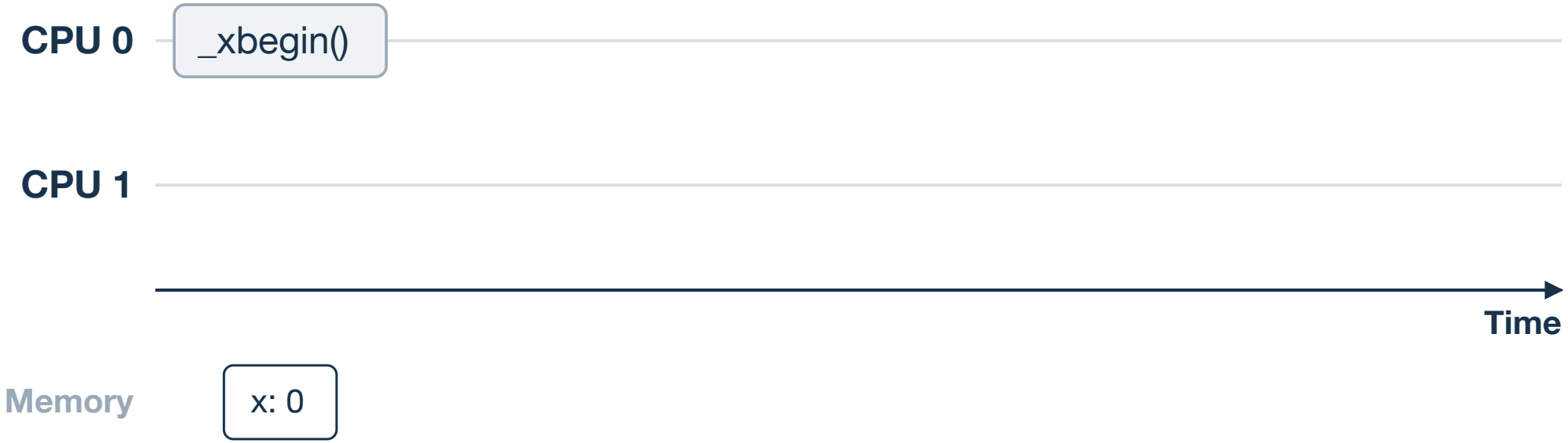
Manually abort inside a transaction

```
if _xbegin() == _XBEGIN_STARTED:
    if conditions:
        _xabort()
    critical code (access x)   # user code, protected by RTM
    _xend()
else:
    fallback routine
```

Question: what if another process, running at the same time: `x = 1`

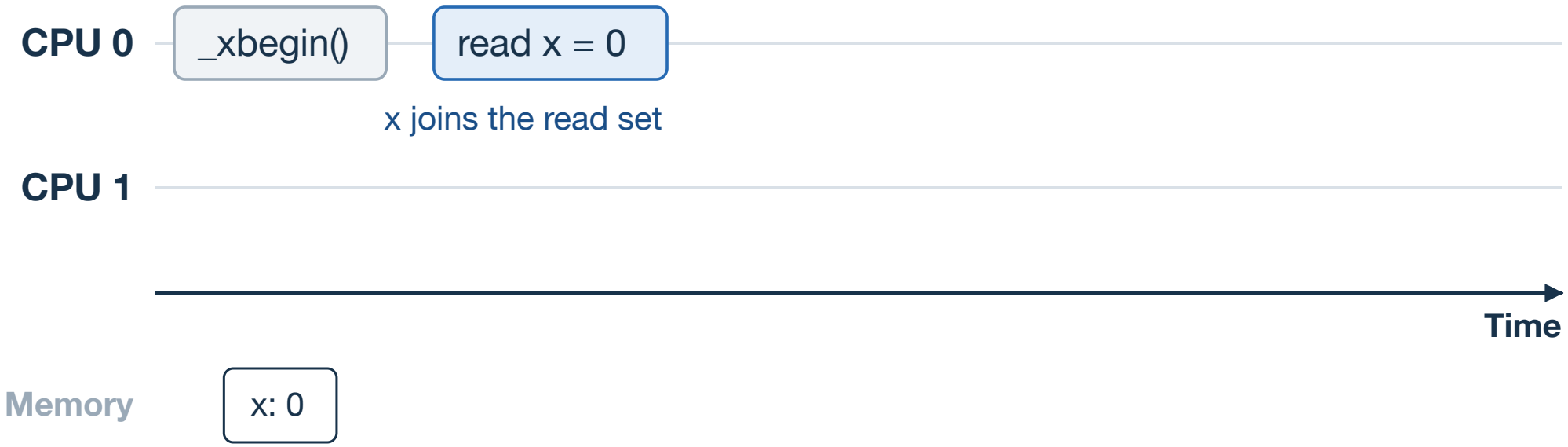
The concurrent write aborts the RTM region

CPU 0 is inside the region and has read x; CPU 1 writes x outside any region



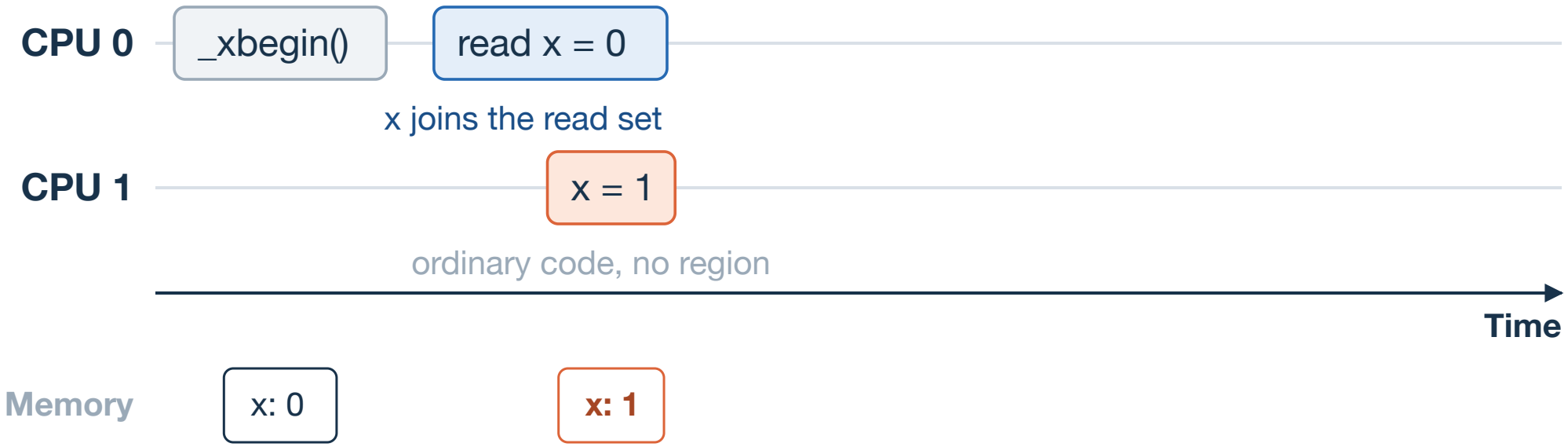
The concurrent write aborts the RTM region

CPU 0 is inside the region and has read x; CPU 1 writes x outside any region



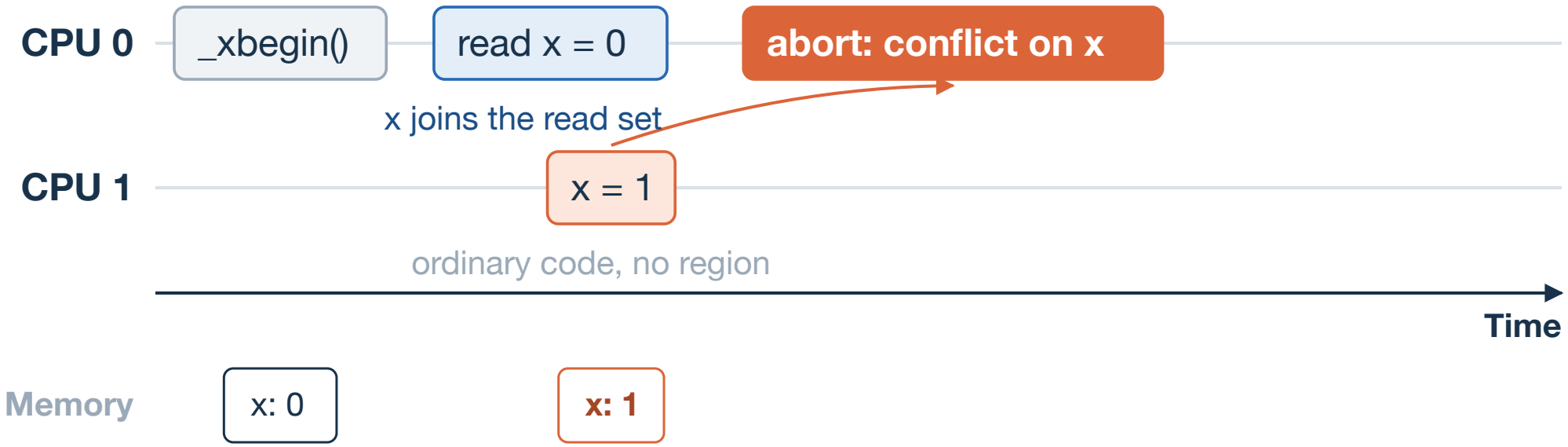
The concurrent write aborts the RTM region

CPU 0 is inside the region and has read x; CPU 1 writes x outside any region



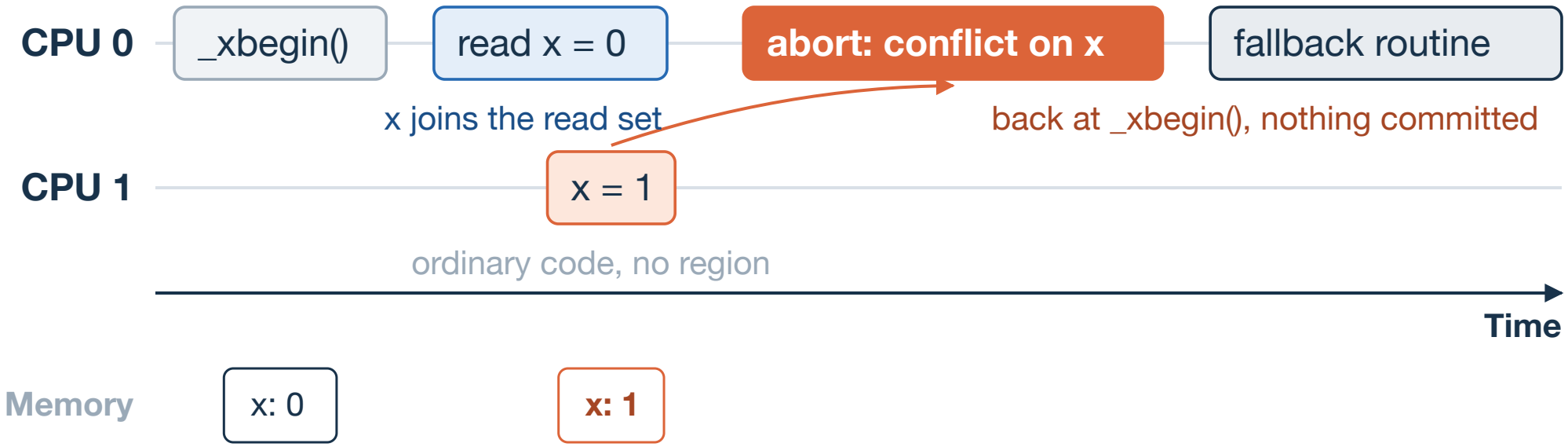
The concurrent write aborts the RTM region

CPU 0 is inside the region and has read x; CPU 1 writes x outside any region



The concurrent write aborts the RTM region

CPU 0 is inside the region and has read x; CPU 1 writes x outside any region



A conflict aborts the region, never blocks the writer

Challenge: how to handle aborts?

RTM provides new instruction set

- `xbegin()`, `xend()` & `xabort()`

```
Beginning:
    if _xbegin() == _XBEGIN_STARTED
        /* do some critical work */
        _xend()
    else
        goto beginning
```

Question: does a simple retry work?

Can we use a simple retry? No

Problem: RTM cannot guarantee success

- Similar to OCC, code in RTM can be frequently aborted due to conflicts

Must switch to a fallback path after some retries (e.g., using a counter for the retry times)

- E.g., Fallback to use lock in the fallback path; typically use global locking to simplify coding

```
if _xbegin() == _XBEGIN_STARTED
    if lock.held()
        xabort()
    /* do some critical work */
    _xend()
else
    lock.acquire()
```

`lock.held()` checks the lock status to avoid conflict with fallback handlers on other CPUs

Why can't RTM guarantee success? Fun facts about the hardware implementation of RTM

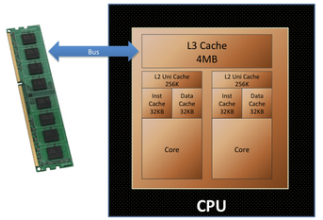
The benefits of RTM

- Memory operations between `xbegin()` & `xend()` satisfy before-or-after & all-or-nothing properties
- No need to implement OCC / 2PL manually
- High performance if no abort happens (performance similar to bare metal execution)

Question: how does HTM (RTM) implements this?

Implementation of RTM: OCC!

Background: Intel implements RTM using OCC

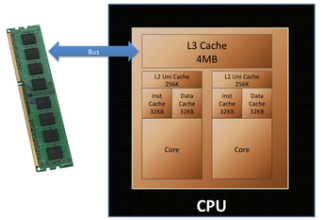


- Use CPU cache to track the read/write sets
- How to detect conflicts? Based on cache coherence protocols

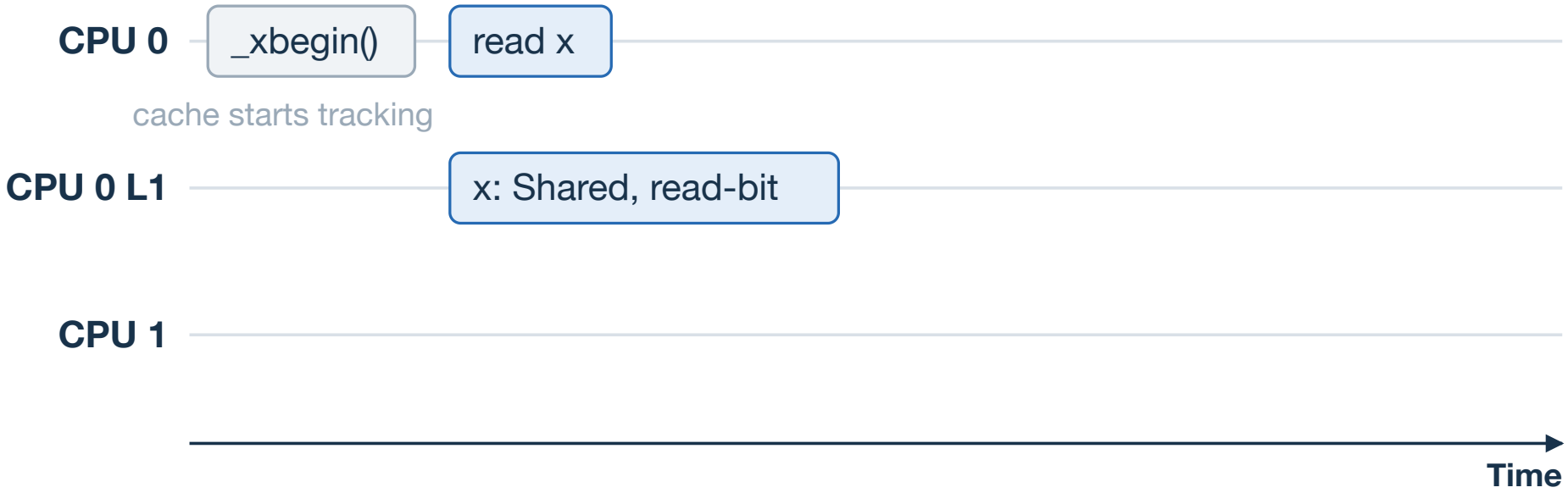


Implementation of RTM: OCC!

Background: Intel implements RTM using OCC

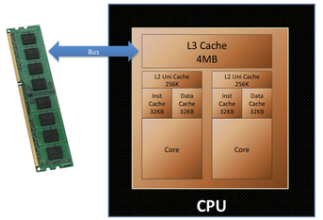


- Use CPU cache to track the read/write sets
- How to detect conflicts? Based on cache coherence protocols

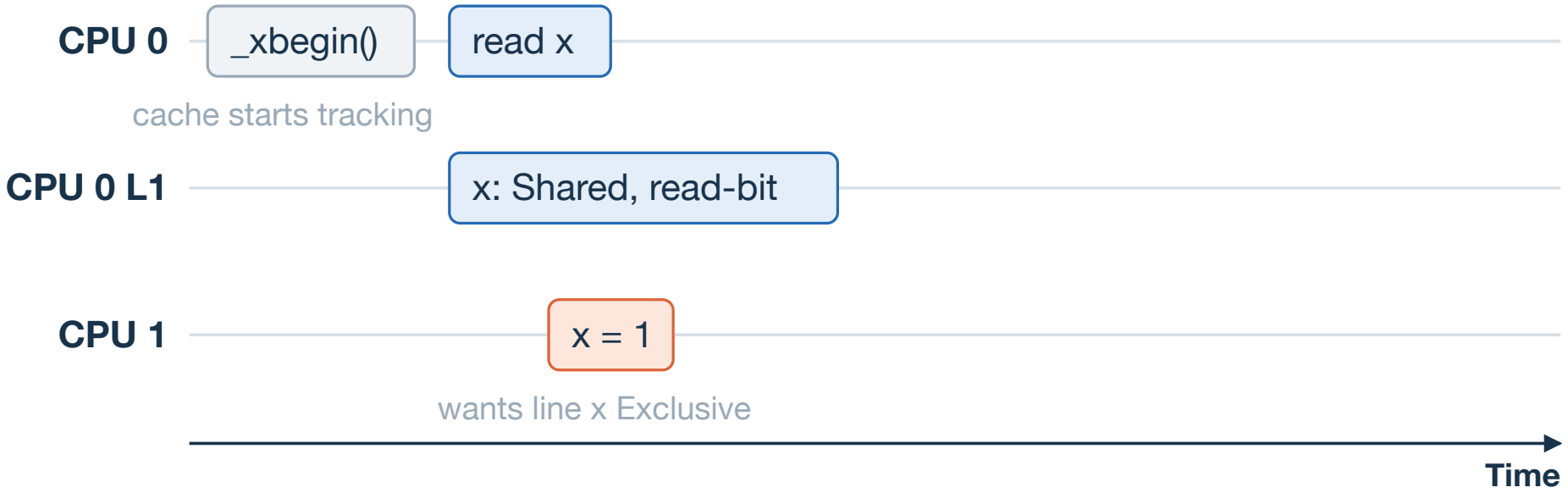


Implementation of RTM: OCC!

Background: Intel implements RTM using OCC



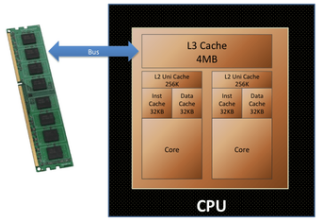
- Use CPU cache to track the read/write sets
- How to detect conflicts? Based on cache coherence protocols

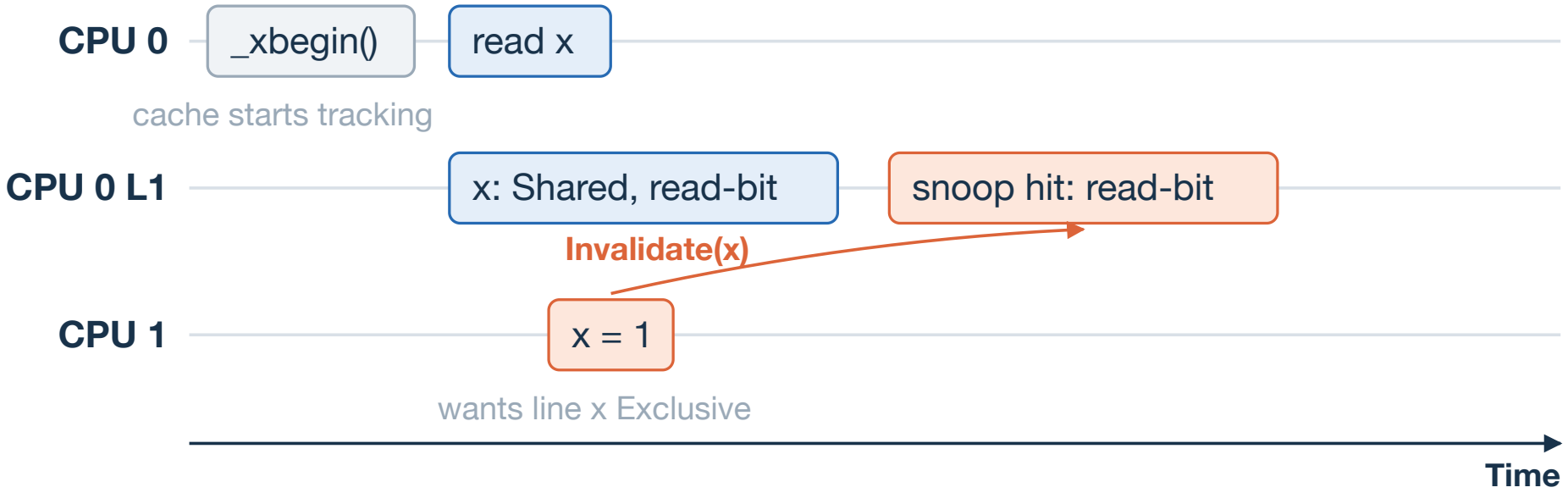


Implementation of RTM: OCC!

Background: Intel implements RTM using OCC

- Use CPU cache to track the read/write sets
- How to detect conflicts? Based on cache coherence protocols

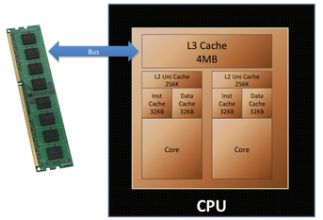


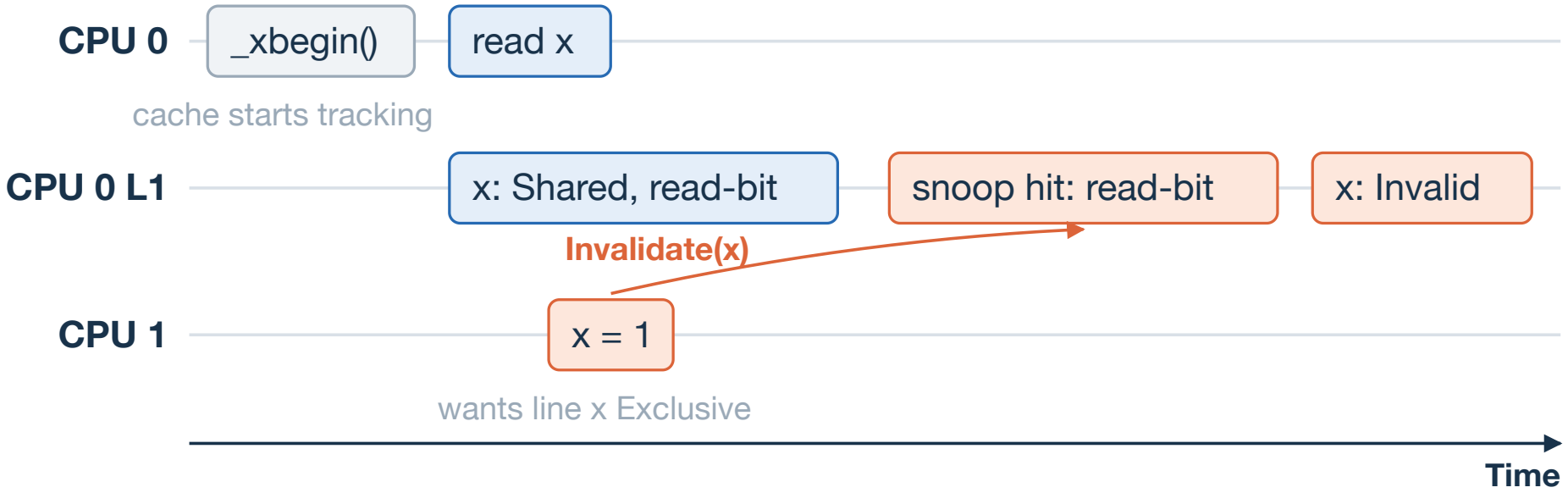


Implementation of RTM: OCC!

Background: Intel implements RTM using OCC

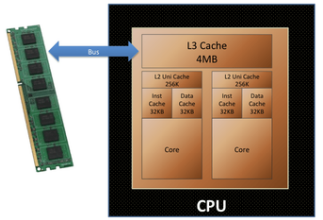
- Use CPU cache to track the read/write sets
- How to detect conflicts? Based on cache coherence protocols



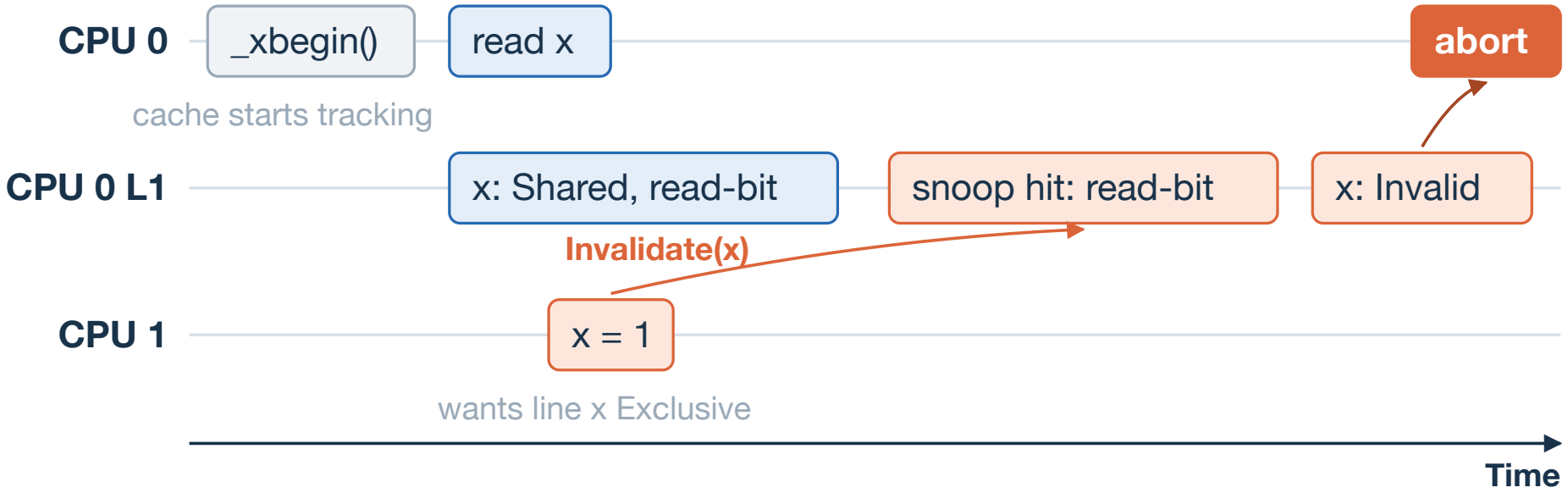


Implementation of RTM: OCC!

Background: Intel implements RTM using OCC



- Use CPU cache to track the read/write sets
- How to detect conflicts? Based on cache coherence protocols



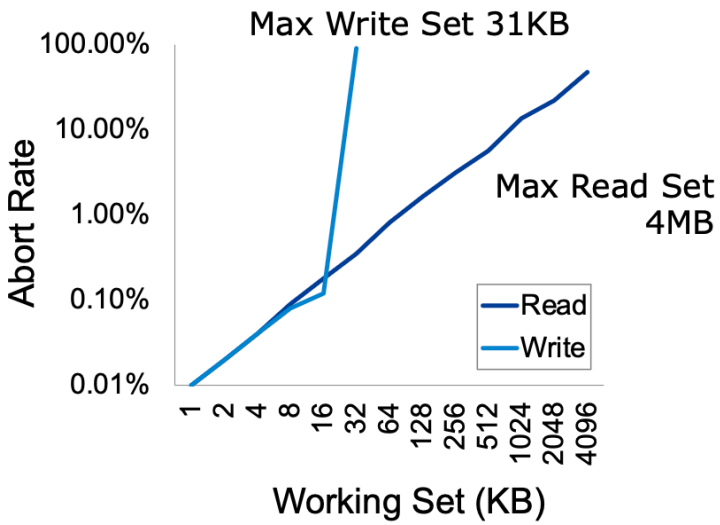
All in hardware, and mostly with existing hardware mechanisms, so it is efficient

The issue behind the efficient implementation: Limited working set

Reason: CPU has limited cache size

How big is the RTM read/write sets?

- Depends on various factors
  - Hardware setup (e.g., CPU cache size)
  - Access pattern: read or write
  - L1 cache tracks all the writes
  - L2 or L3 tracks all the reads [1]



[1] Source: Wang et al., Using Restricted Transactional Memory to Build a Scalable In-Memory Database, EuroSys'14, Figure 1(a)

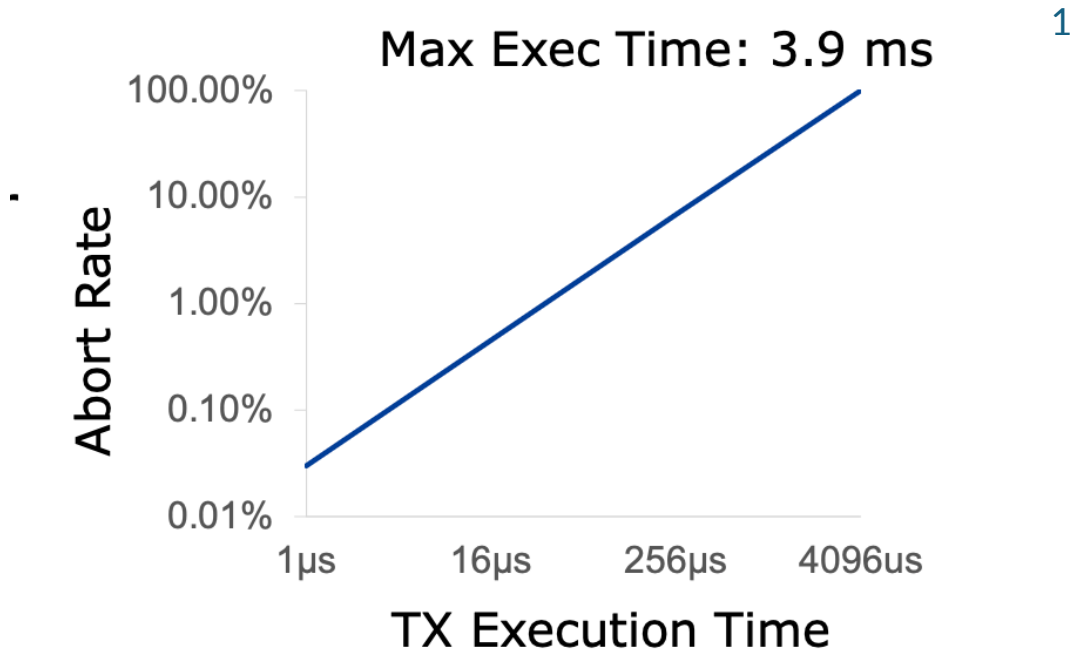
Issue: External events break the RTM execution

HTM TX cannot run a very long time

- The longer the execution, the higher the probability a TX aborts

Affected by external events, because cache is typically evicted under such events

- Network IO, (timer) interrupt, etc.



[1]

[1] Source: Wang et al., Using Restricted Transactional Memory to Build a Scalable In-Memory Database, EuroSys'14, Figure 1(d)

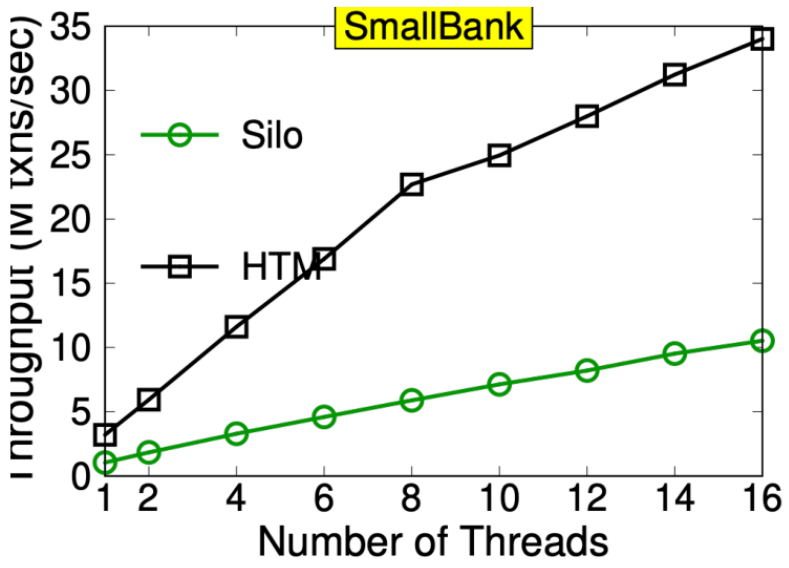
Hardware vs. Software-based OCC

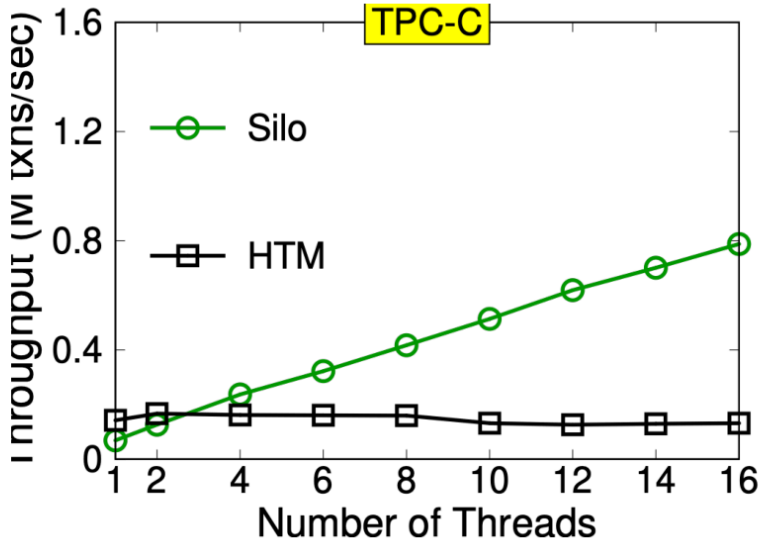
Naïve: using HTM to execute the in-memory transaction

- Smallbank: transfer & audit
- TPC-C: much more complex, insert 10+ orders and update 10+ stocks

Silo@SOSP'13: the fastest in-memory OCC implementation

[1]





[1] Silo: Speedy Transactions in Multicore In-Memory Databases, SOSP'13

Short summary of HTM

Hardware support for transactional memory

- Easy programming model for the programmer (no need to wrap transaction in read/write API of the transaction)
  - As long as the programmer does the in-memory computations, e.g., in-memory database
- Good performance if using properly
  - No need for locking & atomic operations

However, the programmer should handle its pitfalls

- So the programming model is not that easy :)

It is an application of OCC (in the hardware)

- OCC is a general technique that can be applied to many fields

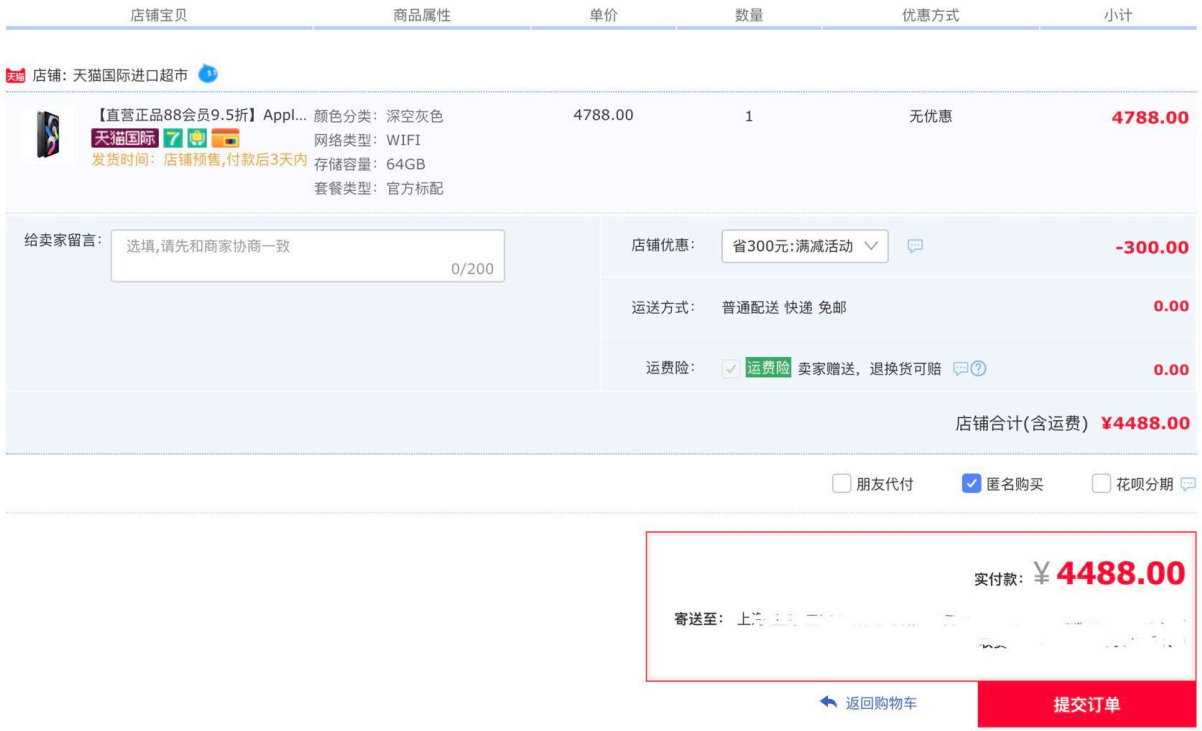
Put it all together: Transactions

What a TX is, what ACID buys you, and why isolation is the hard part

Widely used transaction (TX)s

```
if item.count > 0:
    item.count -= 1
    Cart.add(item)
```

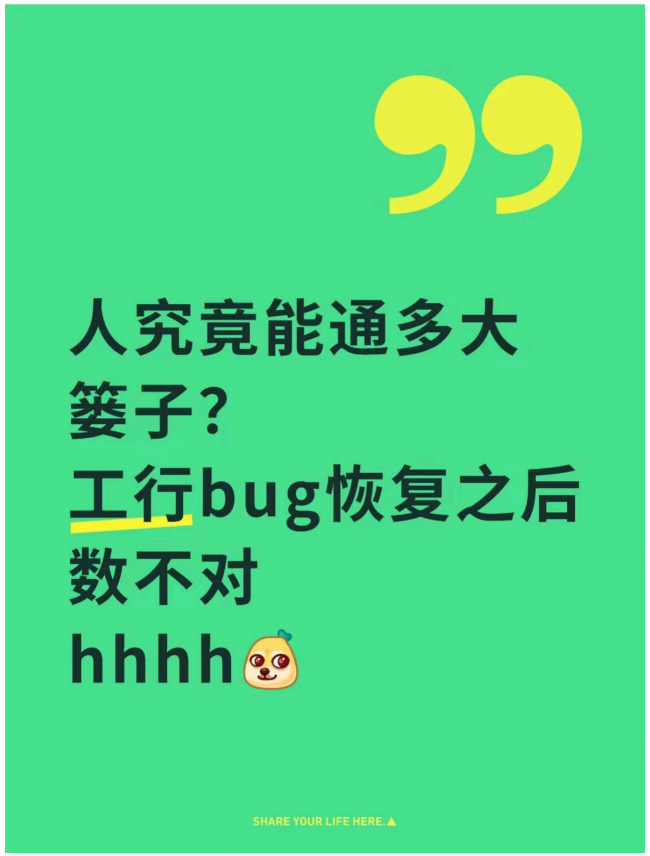
1



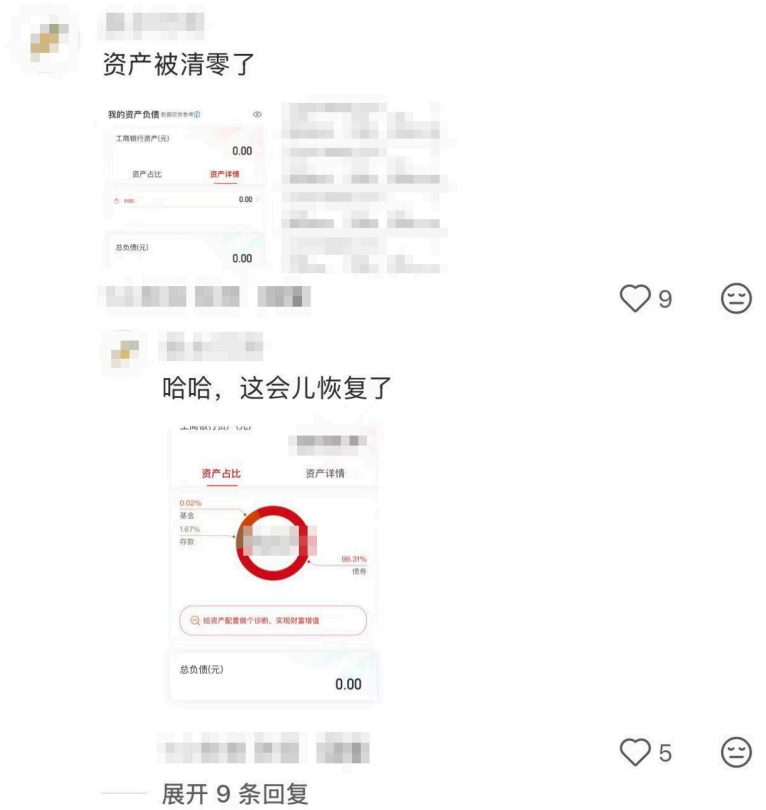
[1] Screenshot: Tmall checkout page

Widely used transaction (TX)s

Why are financial applications typically implemented with TXs?



[1]



[1] Screenshot: a public social-media post about an ICBC outage; avatars, nicknames and amounts are masked

What is a transaction (TX)?

An abstraction to manage the data

Data is also an abstract concept, and can be arbitrary computing data. Concrete examples include:

- Key-value store entries
- File system metadata (e.g., directory, inode, etc)
- ...

Looks like a similar program, with data managed by the TX system, and a mark to denote the start/end of a TX

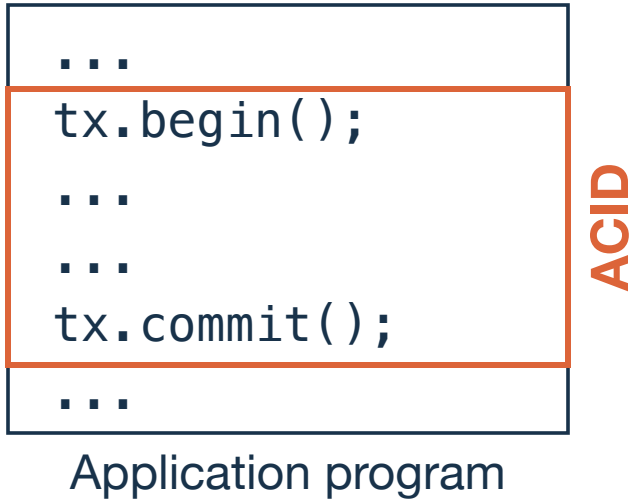
- Why the mark? Otherwise we don't know what to protect!
- Protecting everything is typically impractical

Why TX is good at managing the data? It provides ACID

The program between `{ tx.begin() & tx.commit() }` has the following properties:



- **A**: Atomicity
- **C**: Consistency
- **I**: Isolation
- **D**: Durability



Atomicity: All-or-nothing



[1] Redrawn after Saltzer & Kaashoek, *Principles of Computer System Design*

Isolation & Durability

Isolation: Two concurrent transactions are isolated (e.g., before or after)

- E.g., not viewing the intermediate results of another TX
- Avoid the race conditions

Durability

- Once a transaction is committed, its changes (e.g., writes) must be durably stored to a persistent storage

Consistency

Transaction must change the data from a consistent state to another

- What is consistent is **defined by the applications**
- E.g., transfer should leave the `sum(bank[a] + bank[b])` **unchanged**

Transaction alone is not sufficient for application consistency

- E.g., If the programmer writes the incorrect program:
  `bank[a] = bank[a] - amt * 2`

```
transfer(bank, a, b, amt):
    bank[a] = bank[a] - amt
    bank[b] = bank[b] + amt
```

The database community has studied how to check data consistency with invariants

- Out of the scope of today's lecture

Enforcing ACID properties (in the single-machine case)

Enforcing A & D:

- Logging and recovery (see the all-or-nothing atomicity lecture)

Enforcing C:

- Database constraint system (out of the scope of this class)

Enforcing I:

- Concurrency control methods (the before-or-after atomicity lecture, e.g., 2PL / OCC)

Though enforcing C is out of the scope, enforcing I simplifies enforcing C, because the programmer only needs to reason about the correctness of single-threaded consistency

Why Isolation (serializability) is ideal for consistency?

Assumption: programmers are pro at writing single-thread programs

- Specially, $\forall_{tx}\ C_i \rightarrow tx \rightarrow C_j$, in a single-thread context, tx can move data from a consistent state (C_i) to another consistent state (C_j)

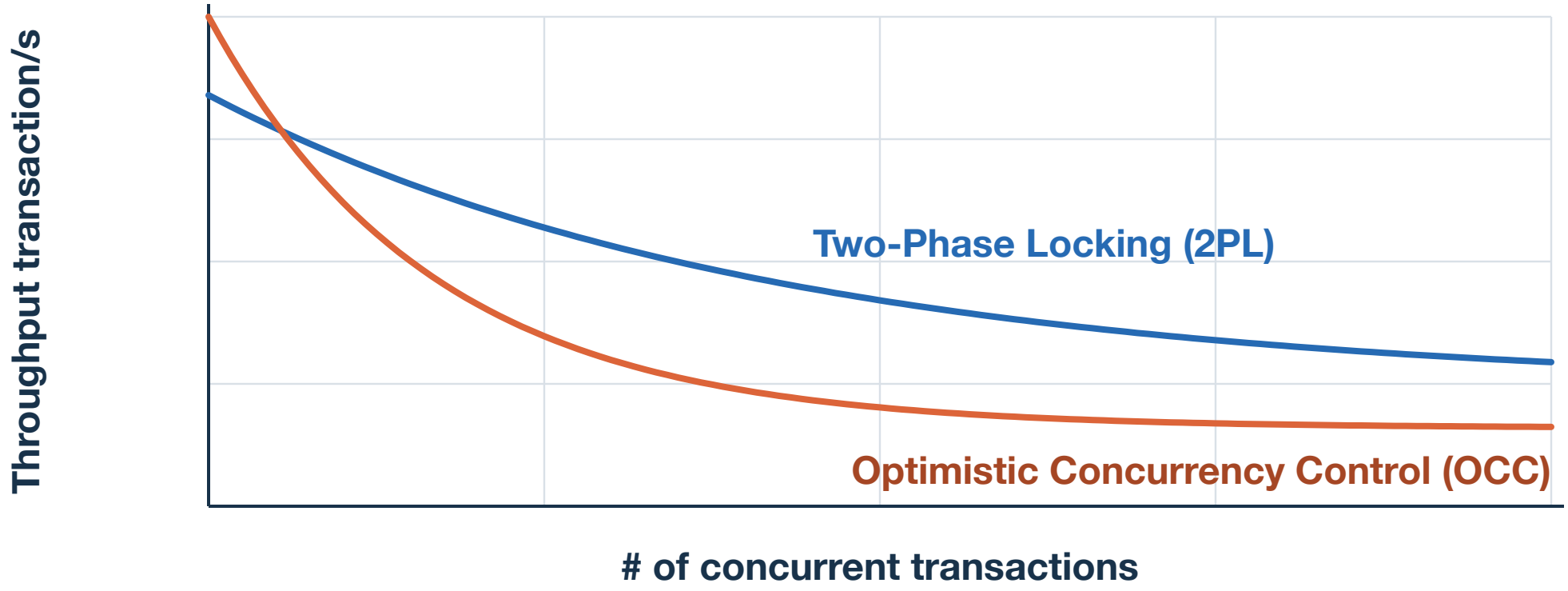
Then, if transactions guarantee serializability, then the final state of concurrent execution is consistent

- i.e., the concurrent execution can reduce to $C_0 \rightarrow tx_0 \rightarrow tx_1 \rightarrow \cdots \rightarrow C_n$
- If C_0 is consistent, then C_n must be consistent

Note that the database community may define other, weaker isolation levels that can break this property

Review: 2PL vs. OCC: in a nutshell

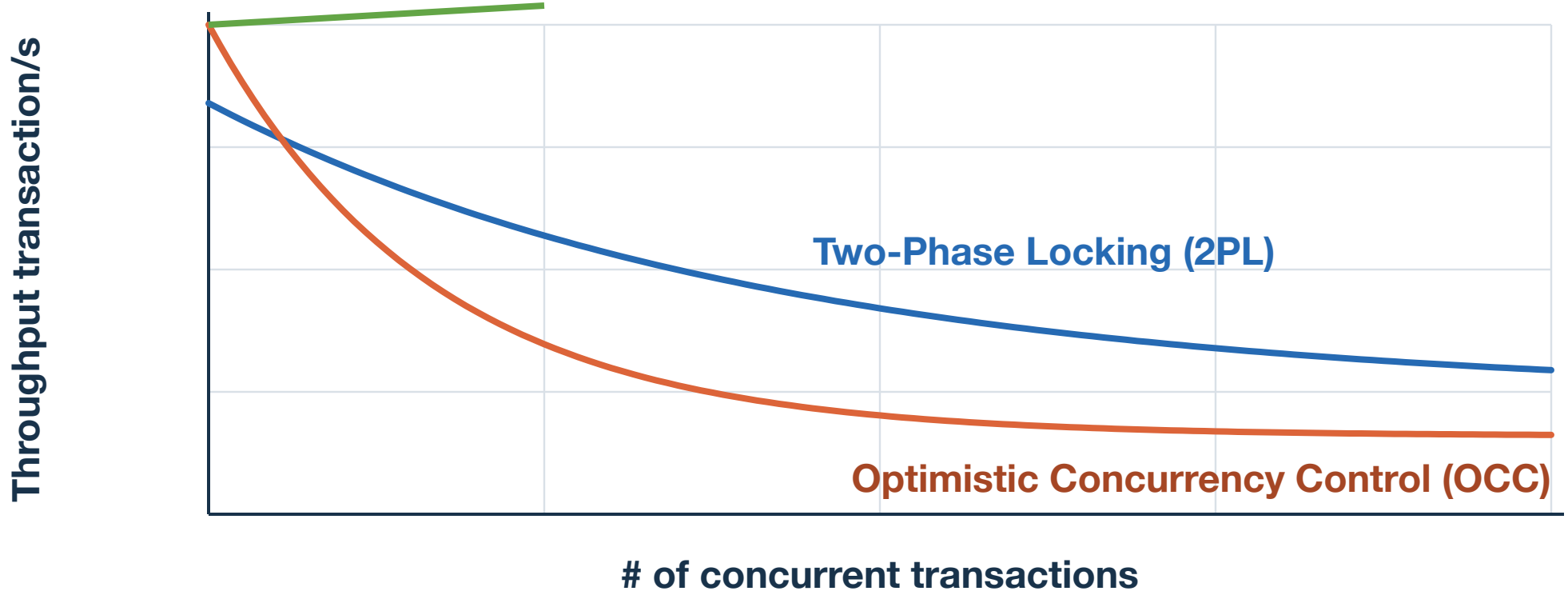
1



[1] Source: Extracting More Concurrency from Distributed Transactions [OSDI'14]

Review: 2PL vs. OCC: in a nutshell

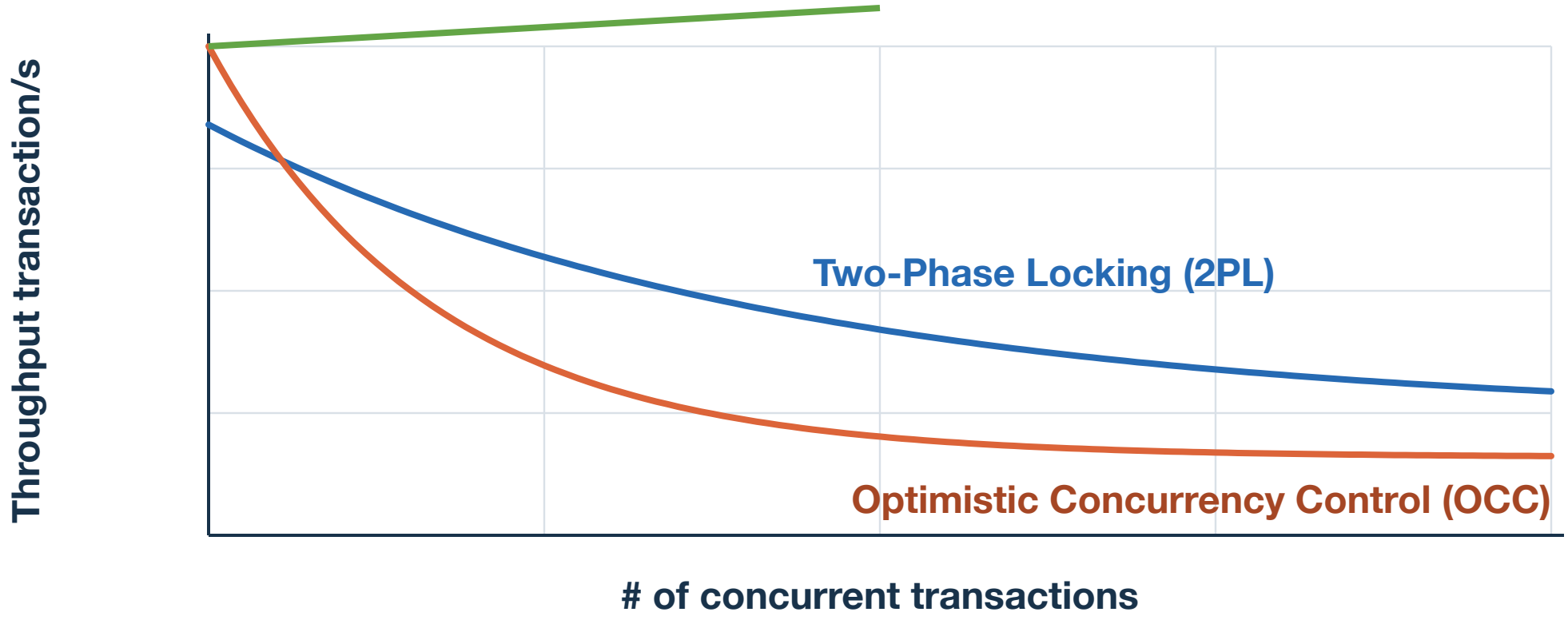
1



[1] Source: Extracting More Concurrency from Distributed Transactions [OSDI'14]

Review: 2PL vs. OCC: in a nutshell

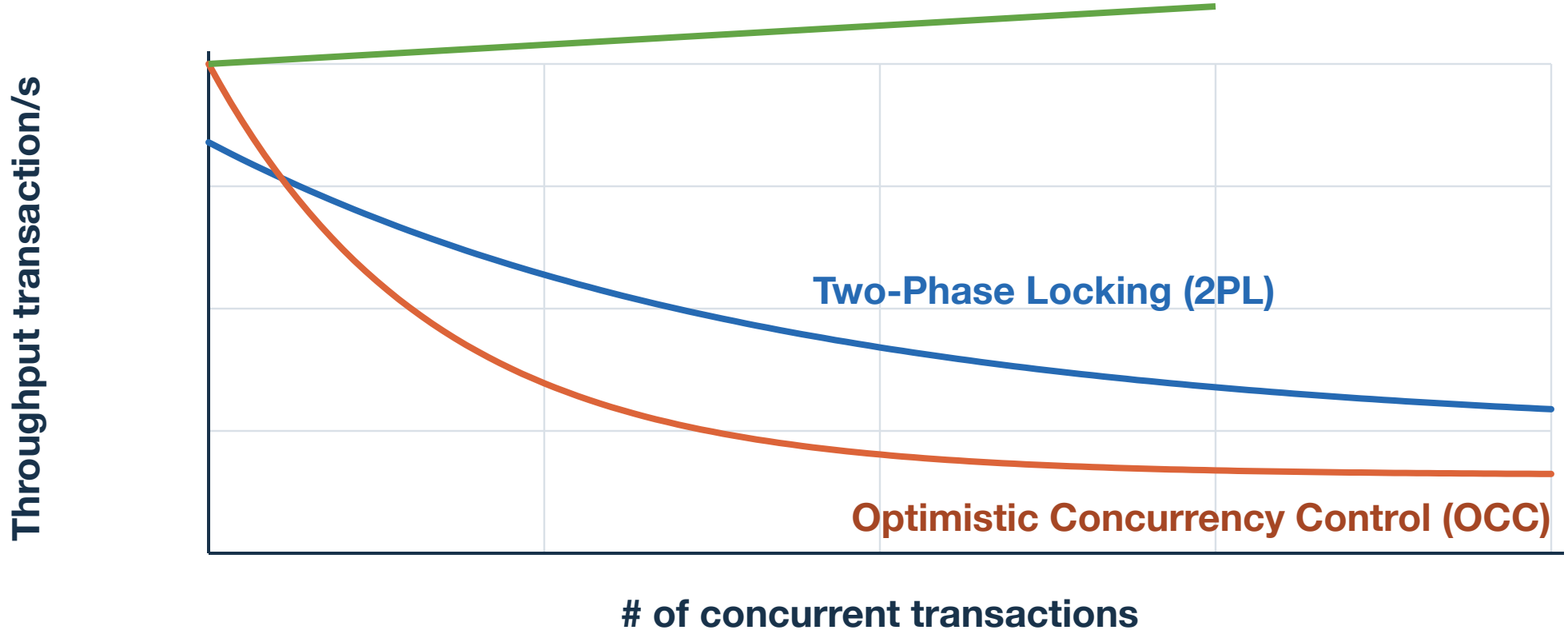
1



[1] Source: Extracting More Concurrency from Distributed Transactions [OSDI'14]

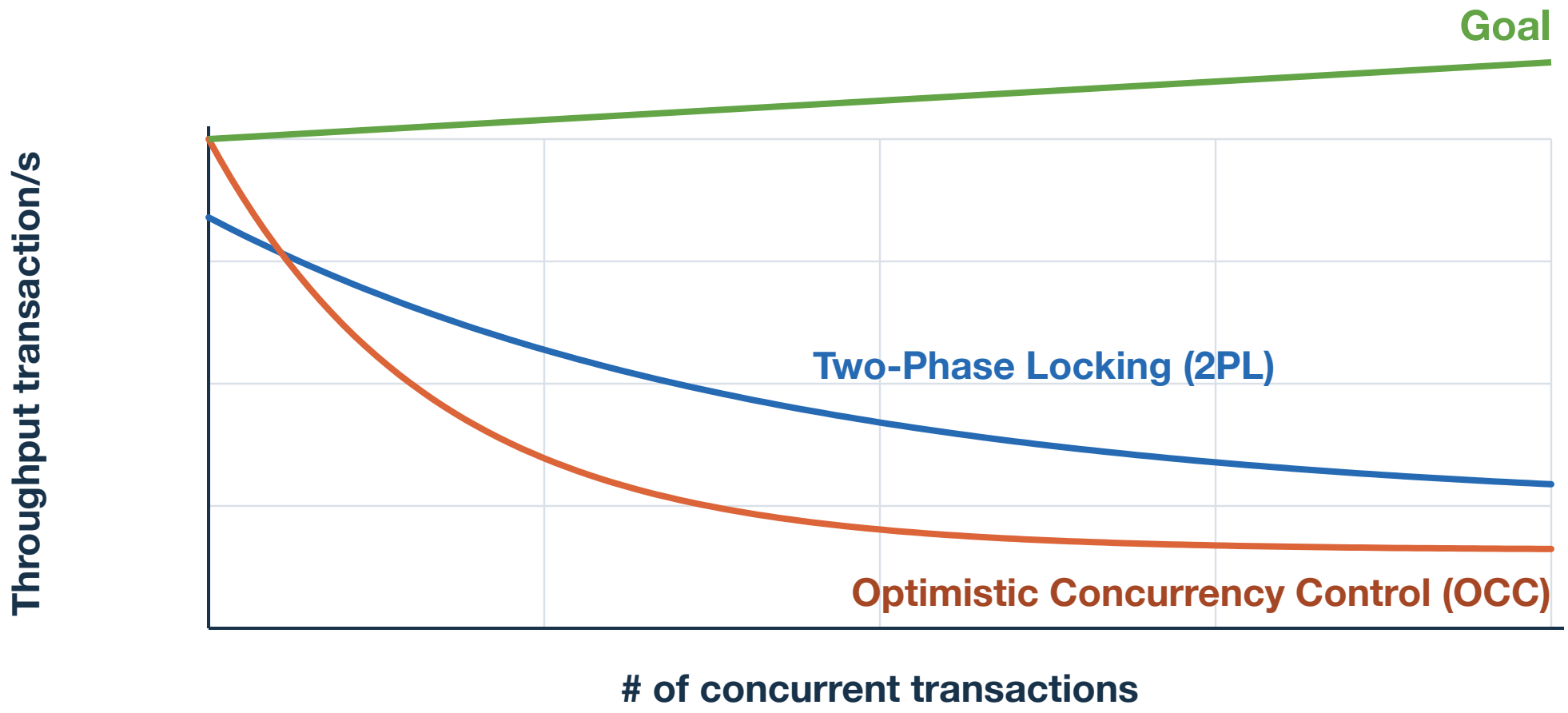
Review: 2PL vs. OCC: in a nutshell

1



[1] Source: Extracting More Concurrency from Distributed Transactions [OSDI'14]

Review: 2PL vs. OCC: in a nutshell



[1]

[1] Source: Extracting More Concurrency from Distributed Transactions [OSDI'14]

2PL and OCC treat every TX the same.

For specific cases, we can achieve high-performance isolation.

Motivation: TX usage in real-world application

Scenario: long-running read-only or read-mostly transactions are common

- e.g., for Taobao, we rarely buy items, but extensively read the items



Question: can 2PL or OCC efficiently support such transactions?

- E.g., a TX reading a lot of items

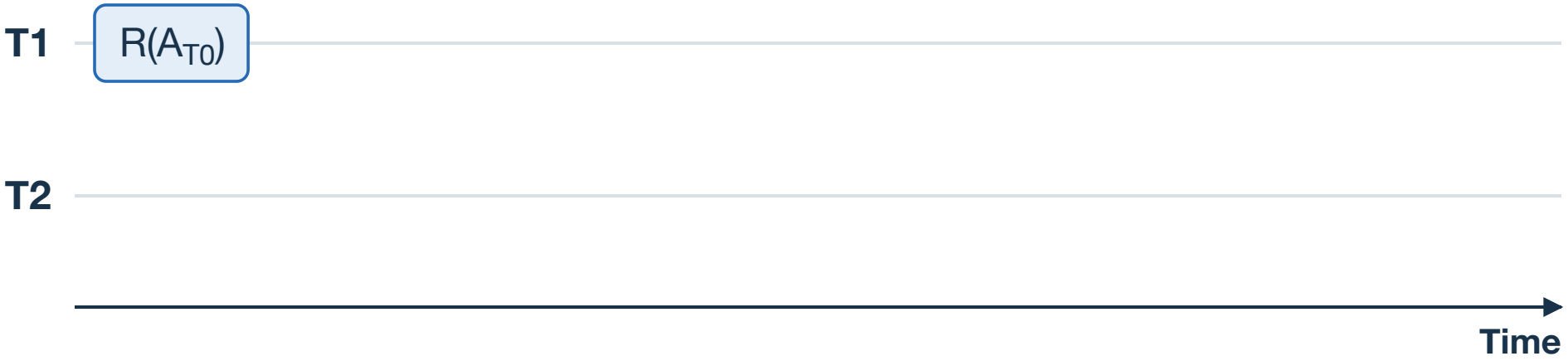
2PL: deadlock + locking overhead
OCC: aborts due to validation failures

Question: can we improve their performance under such scenarios?

A starting case: can we avoid aborts due to validation failures in OCC in read-only cases?

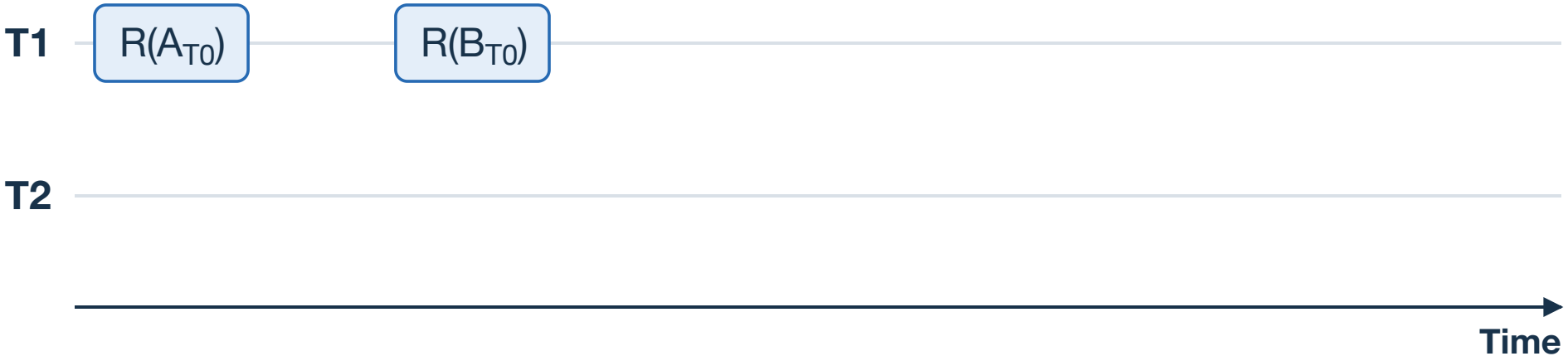
An analysis of read aborts in OCC

T1: `Print(A+B)` T2: `B = 2; A = 3`



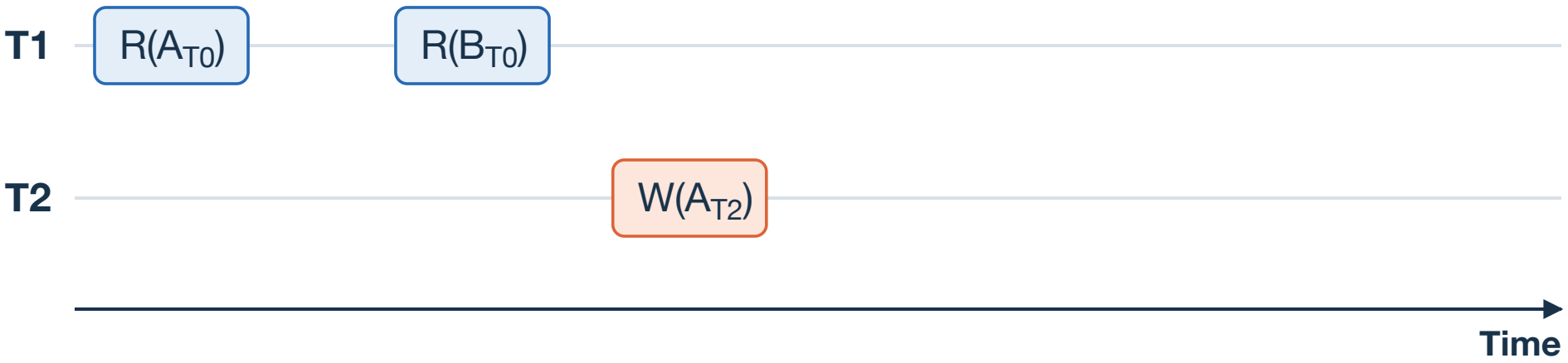
An analysis of read aborts in OCC

T1: `Print(A+B)` T2: `B = 2; A = 3`



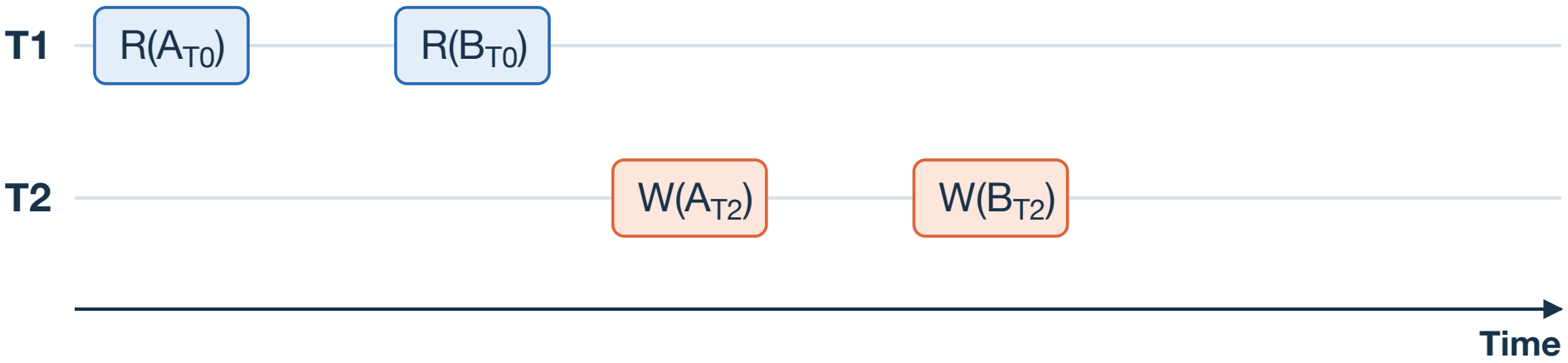
An analysis of read aborts in OCC

T1: `Print(A+B)` T2: `B = 2; A = 3`



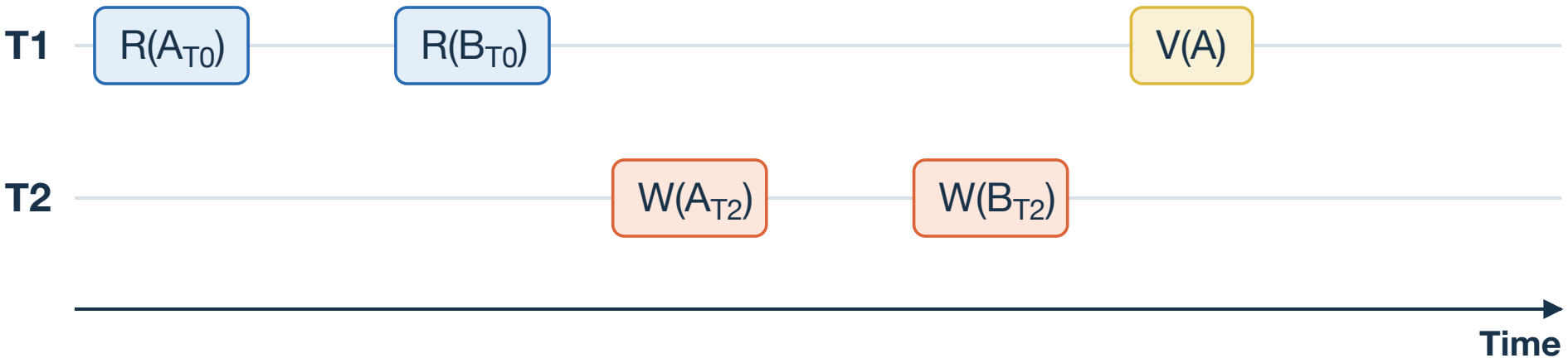
An analysis of read aborts in OCC

T1: `Print(A+B)` T2: `B = 2; A = 3`



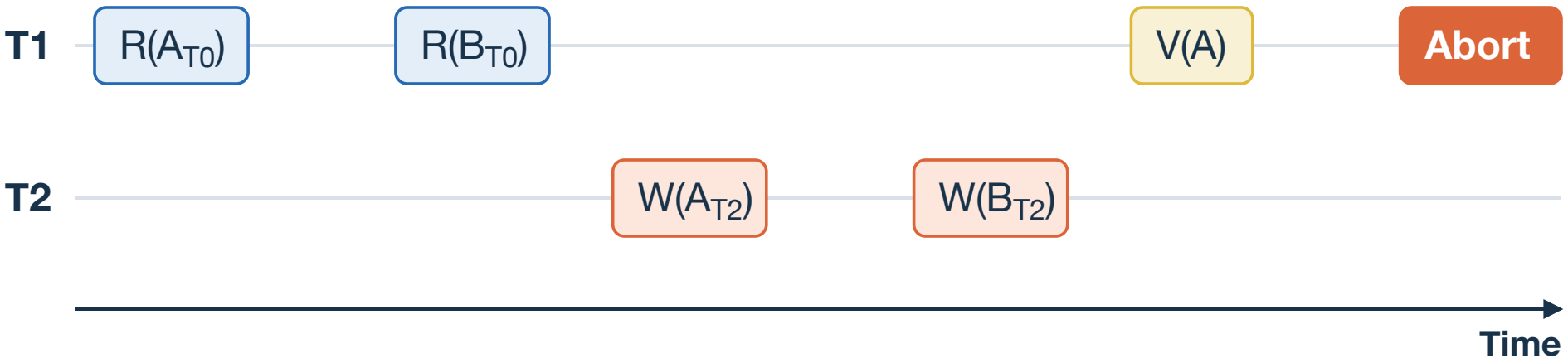
An analysis of read aborts in OCC

T1: `Print(A+B)` T2: `B = 2; A = 3`



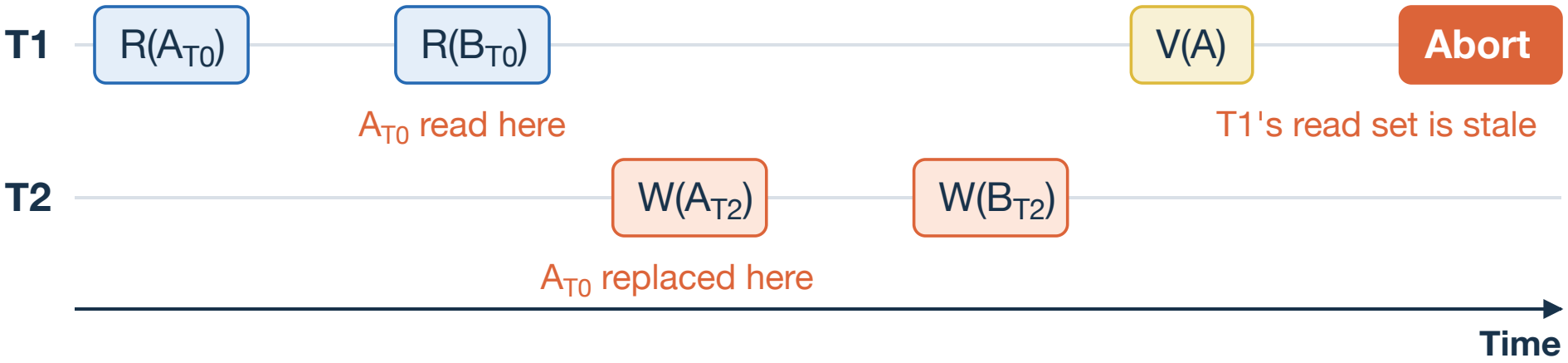
An analysis of read aborts in OCC

T1: `Print(A+B)` T2: `B = 2; A = 3`



An analysis of read aborts in OCC

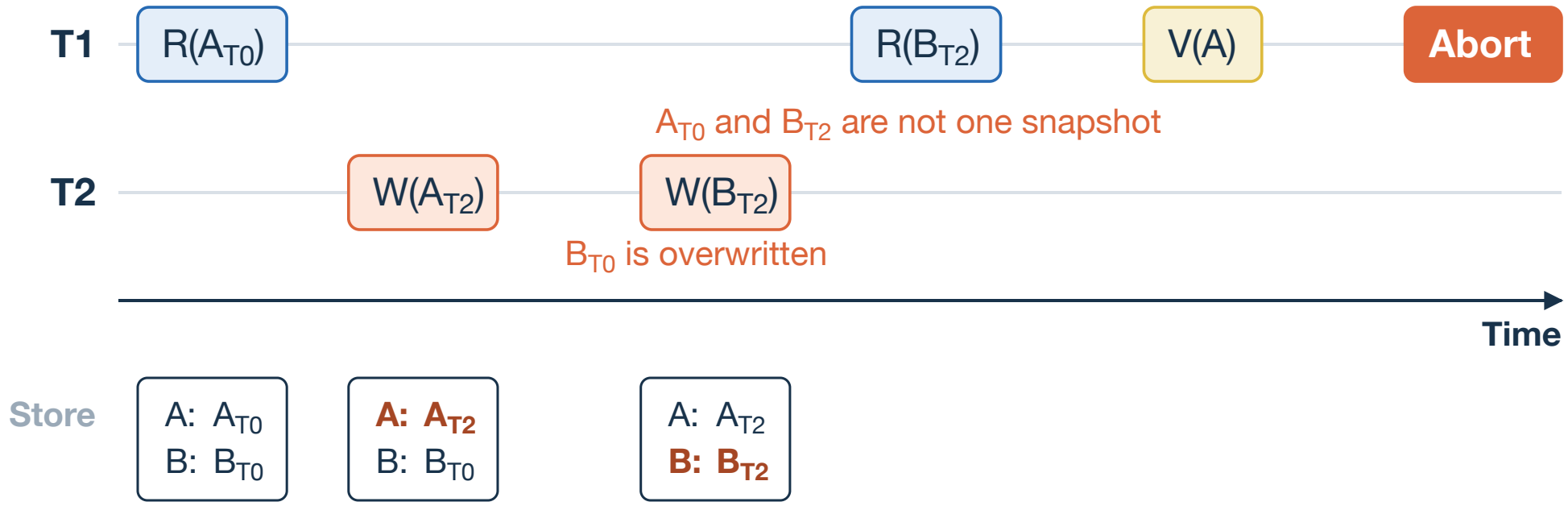
T1: `Print(A+B)` T2: `B = 2; A = 3`



Do we necessarily need to abort T1?

Abort in OCC case (2)

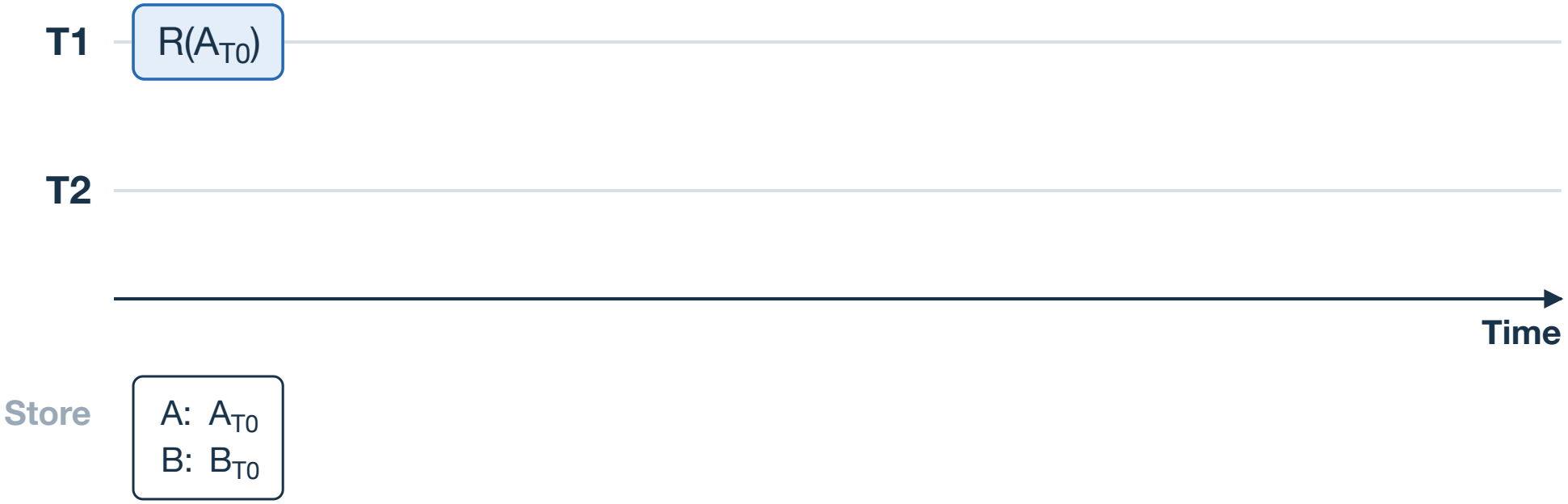
T1: `Print(A+B)` T2: `B = 2; A = 3`



Question: can we avoid aborting T1?

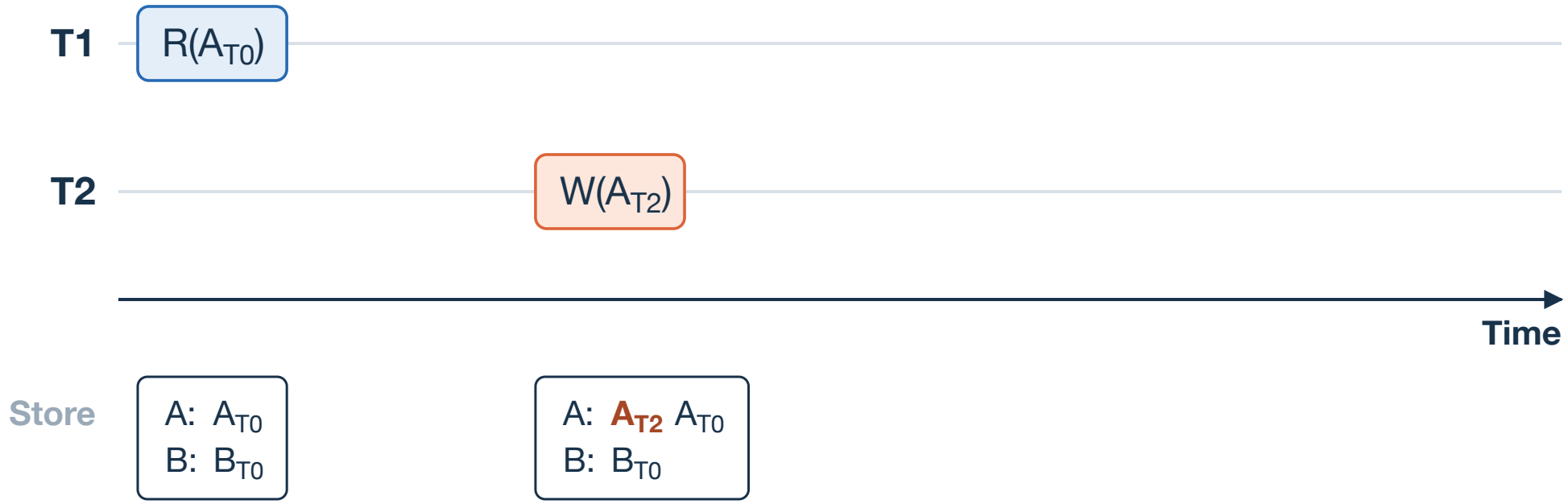
Abort in OCC case (2)

T1: `Print(A+B)` T2: `B = 2; A = 3`



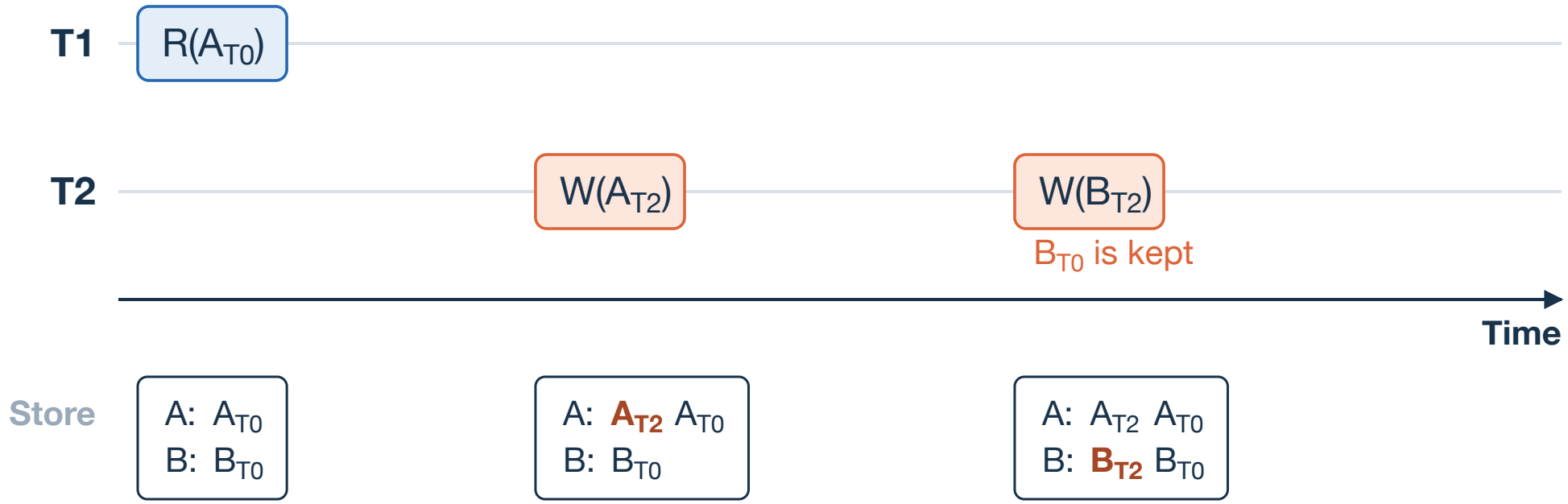
Abort in OCC case (2)

T1: `Print(A+B)` T2: `B = 2; A = 3`



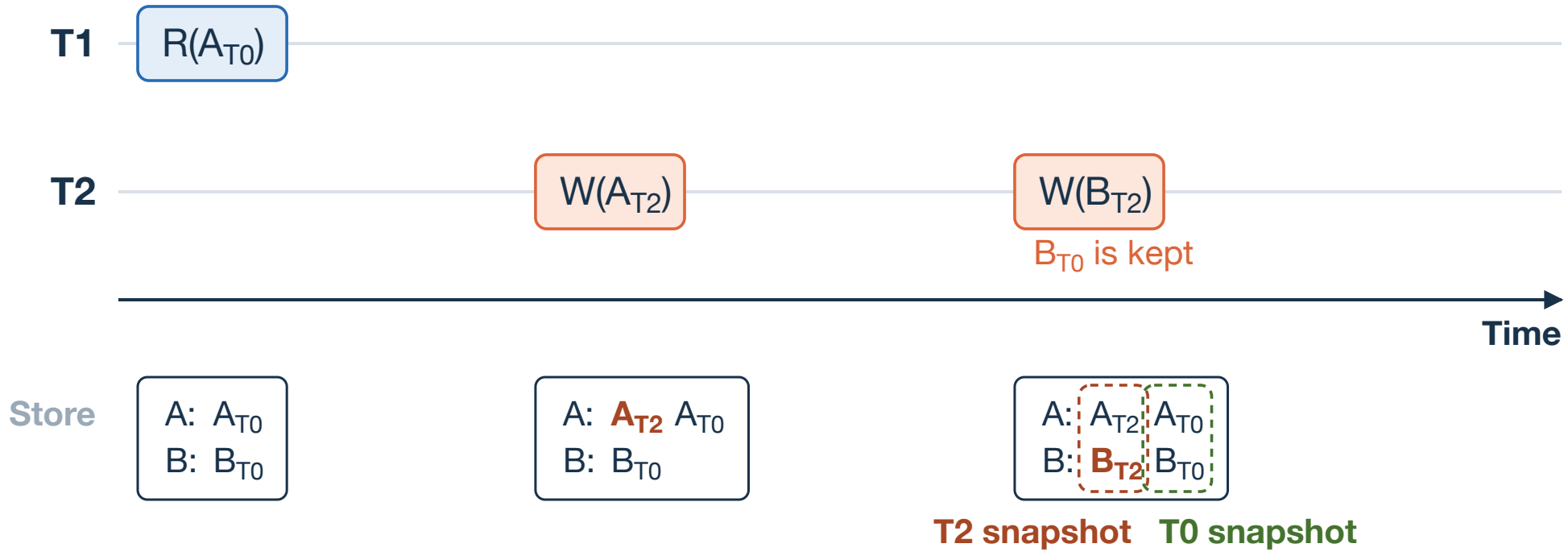
Abort in OCC case (2)

T1: `Print(A+B)` T2: `B = 2; A = 3`



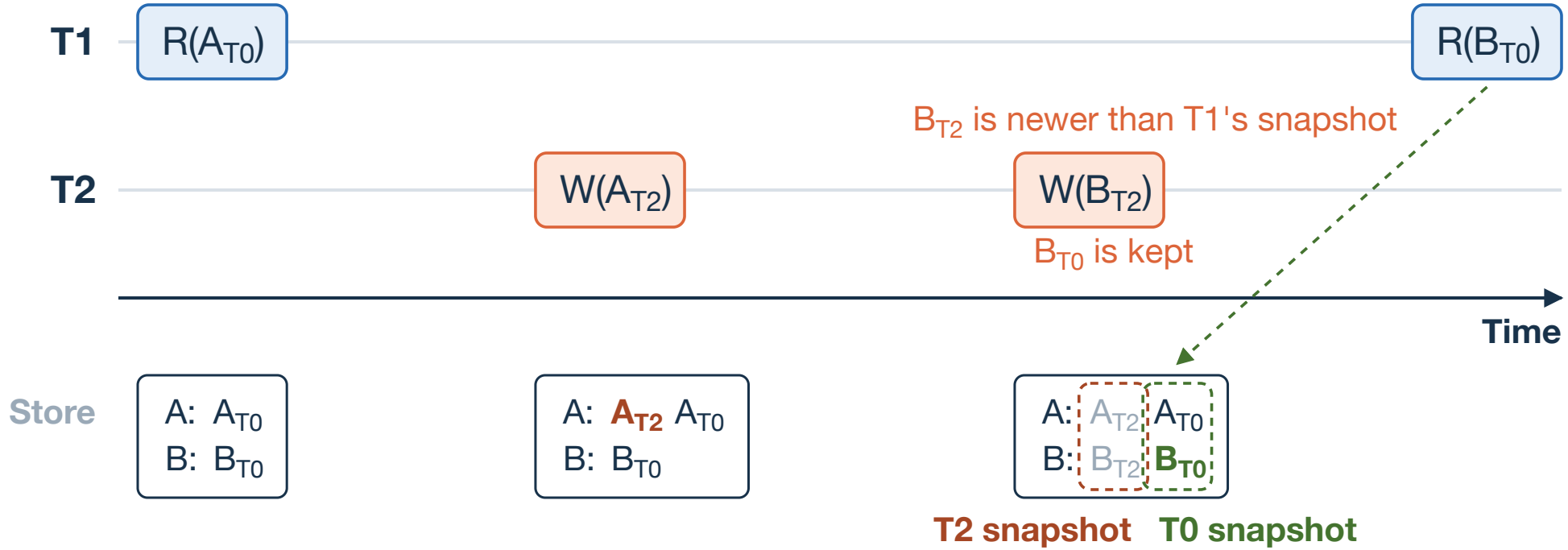
Abort in OCC case (2)

T1: `Print(A+B)` T2: `B = 2; A = 3`



Abort in OCC case (2)

T1: `Print(A+B)` T2: `B = 2; A = 3`

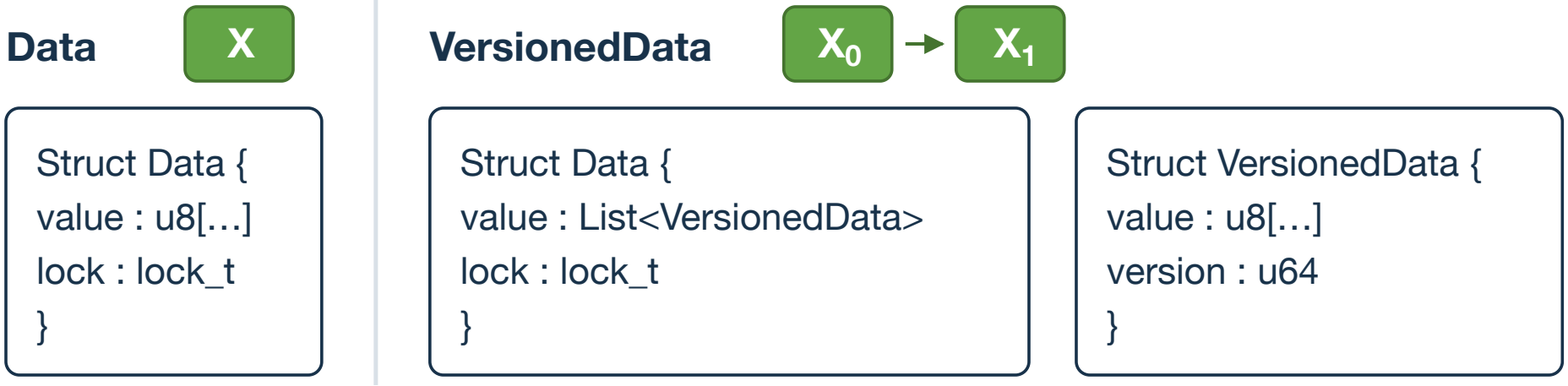


Suppose our data store now keeps the old versions of the data, and transactions don't overwrite old versions, and reads can read the old (consistent) versions

Idea: multi-versioning concurrency control

Each data has multiple-versions instead of a single version

- A set of versioned data represents a snapshot of the system state (data)
  - E.g., $[A_{T0}, B_{T0}]$
- The version ~= the time a TX makes the updates



Idea: multi-versioning concurrency control

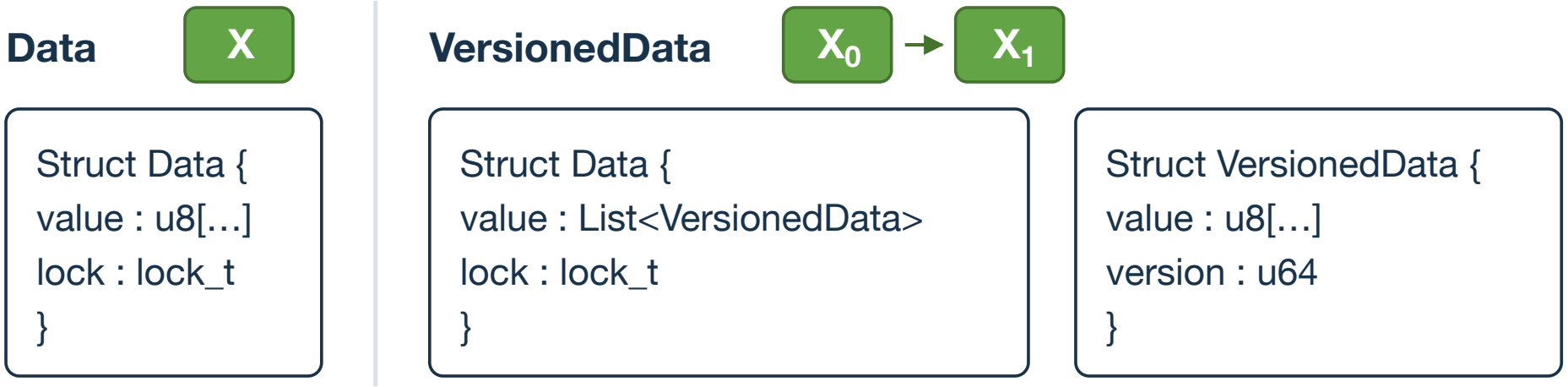
Read

- Only read from a consistent snapshot at a start time

Write

- Install a new version of the data instead of overwriting the existing one
- Version ~= the commit time of the TX

Benefit: reads and writes to different snapshots have a one-way dependency that cannot form a cycle, so a read does not need to abort



Time: a key method to determine the snapshot

The timestamp reflects TX's serial execution order

- E.g., if T1 finishes before T2, it will have a smaller start & commit timestamp
- As a result, T1's snapshot will be different from T2's

Simplest (& widely used) solution: global counter

- Using atomic fetch and add (FAA) to get the TX's begin & commit time
- TX Begin: use FAA to get the start time (for deciding the read timestamp)
- TX Commit: use FAA to get the commit time (for the snapshot to write)

Try #1: Optimize OCC w/ MV (incomplete)

Acquire the start time

Phase 1: Concurrent local processing

- Reads data belongs to the snapshot, i.e., records with versions closest to the start time
- Buffers writes into a write set

Acquire the commit time

Phase 2: Commit the results in critical section

- Commits: installs the write set with the commit time

Compared to OCC, no validation is needed, so no aborts :)

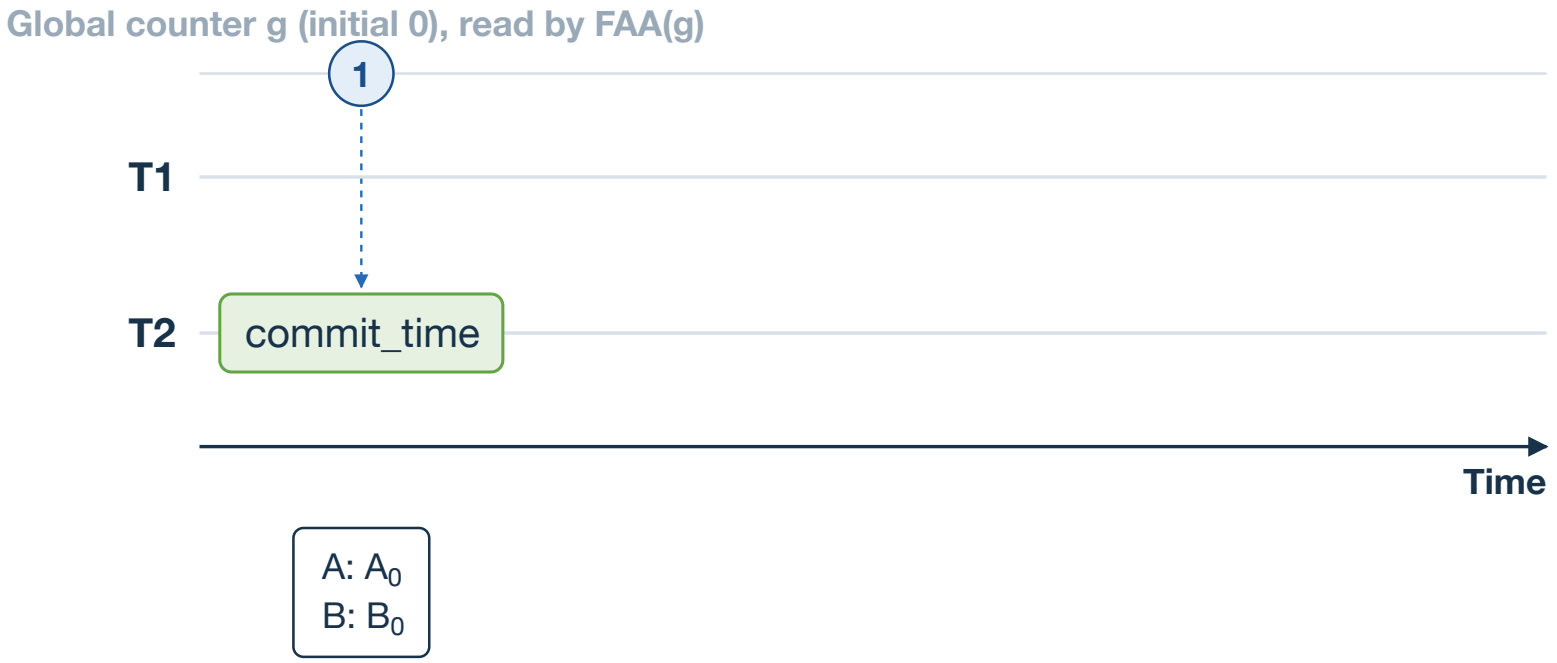
Try #1: Optimize OCC w/ MV (incomplete)

```
Get(tx, record):
    for version,value in record.sort_version_in_decreasing():
        if version <= tx.start_time:
            return value
```

```
Commit(tx):
    for record in tx.write_set:
        lock(record)
    let commit_ts = FAA(global_counter)
    for record in tx.write_set:
        record.insert_new_version(commit_ts, ...)
        unlock(record)
```

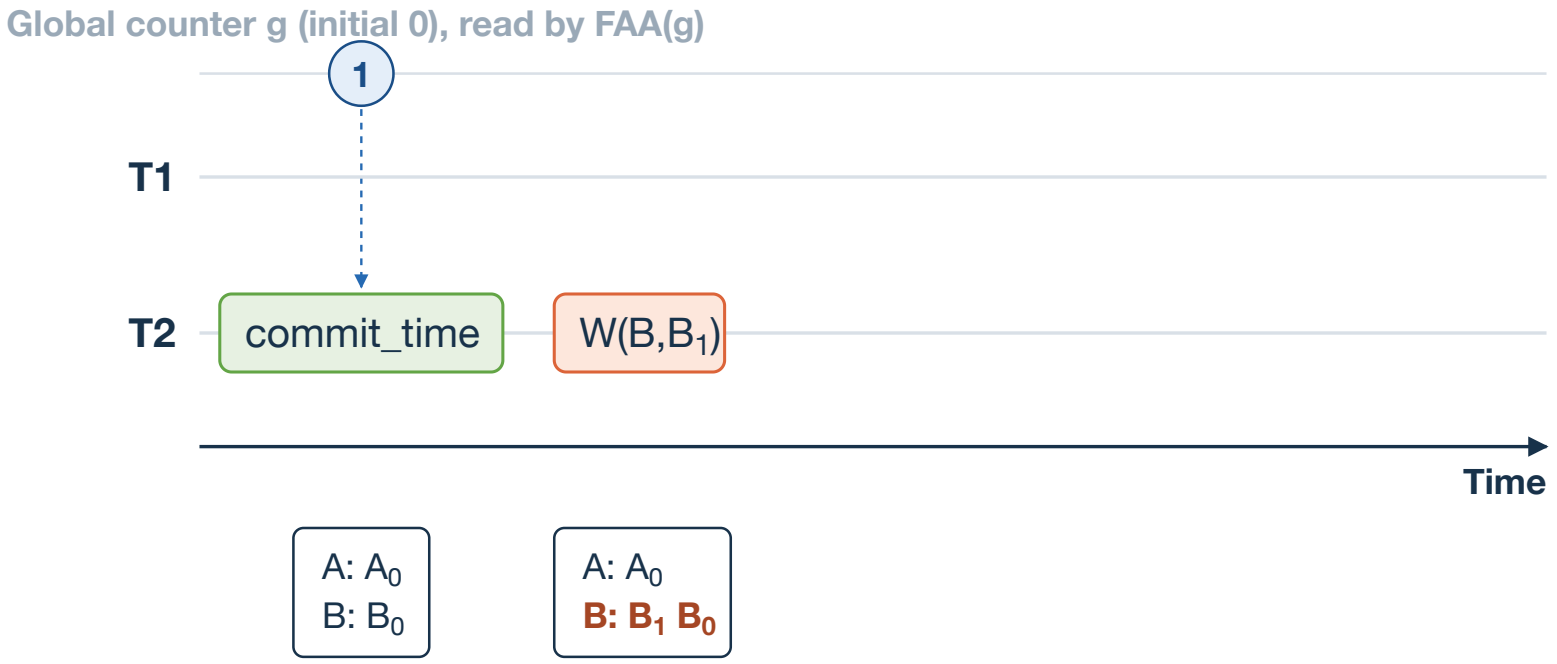
Why incomplete? Partial snapshot (An example)

T1: `Print(A+B)` T2: `B = 2; A = 3`



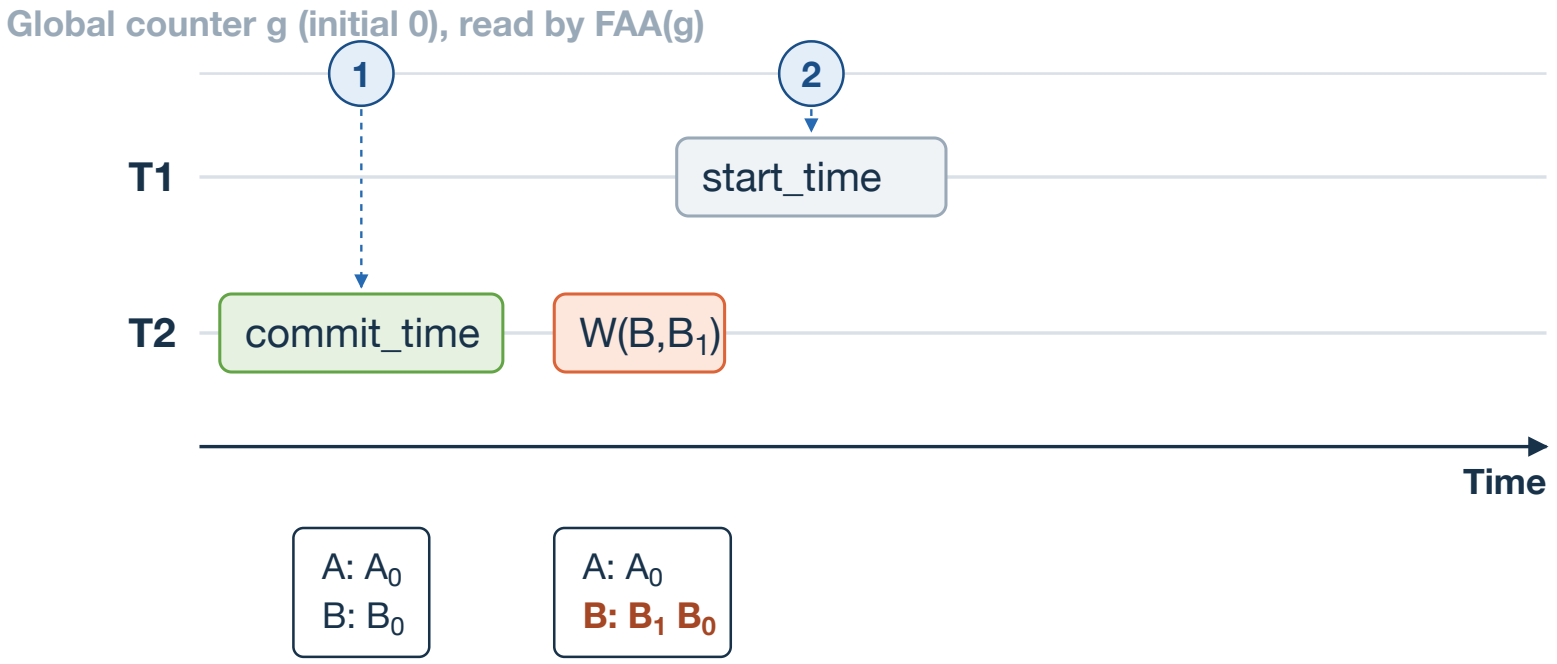
Why incomplete? Partial snapshot (An example)

T1: `Print(A+B)` T2: `B = 2; A = 3`



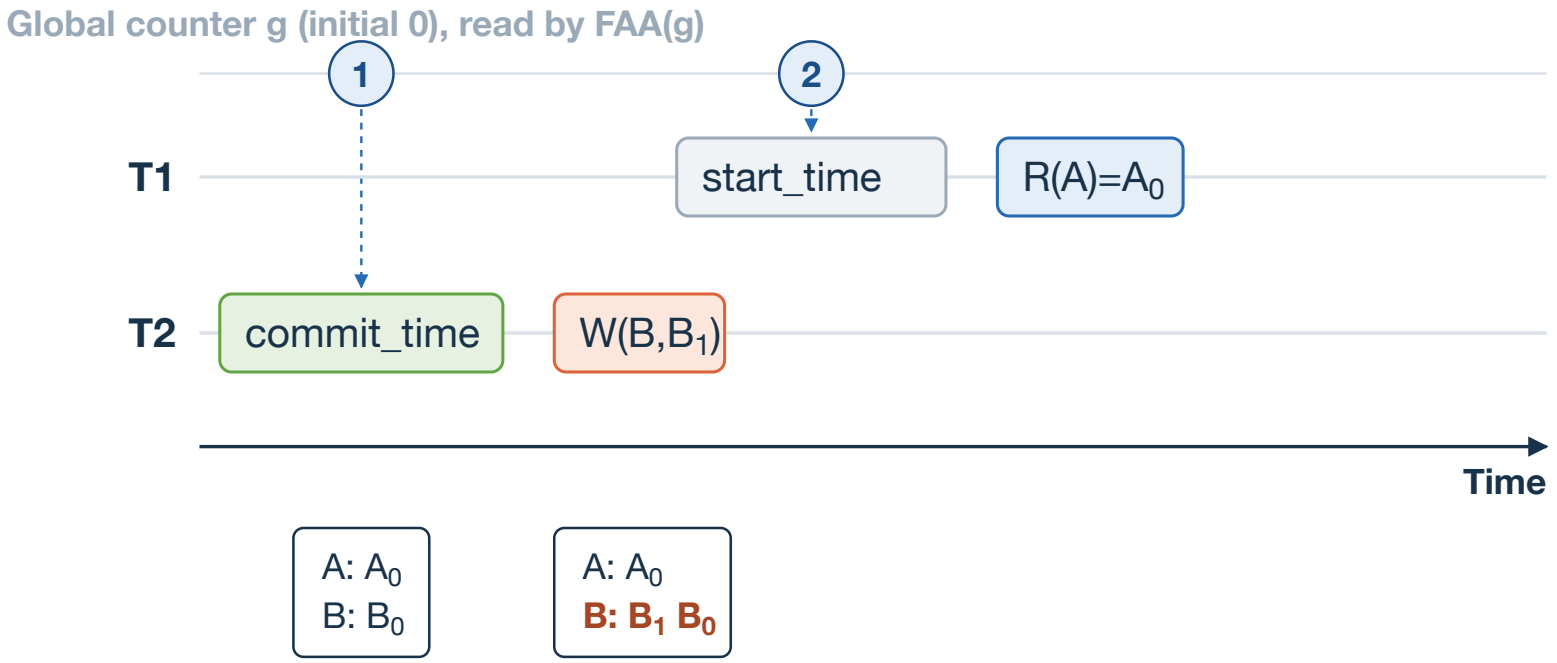
Why incomplete? Partial snapshot (An example)

T1: `Print(A+B)` T2: `B = 2; A = 3`



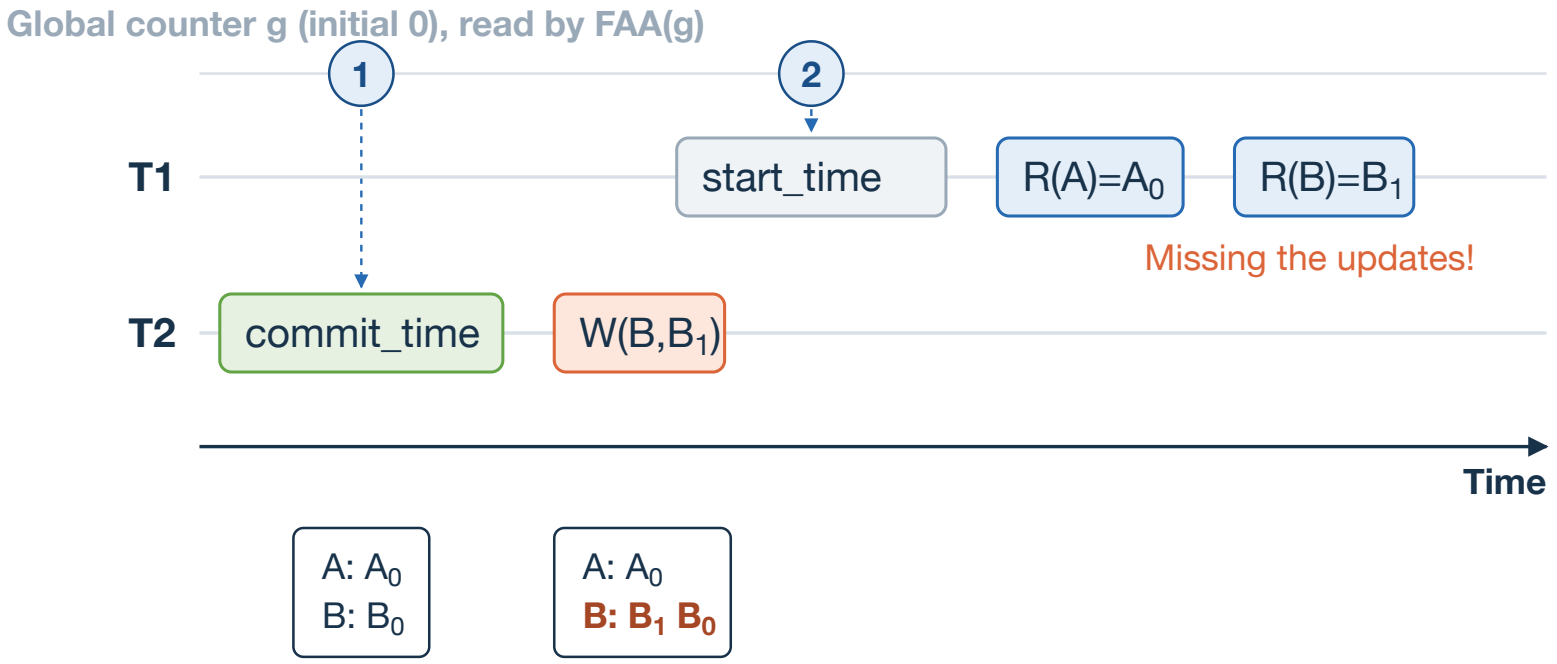
Why incomplete? Partial snapshot (An example)

T1: `Print(A+B)` T2: `B = 2; A = 3`



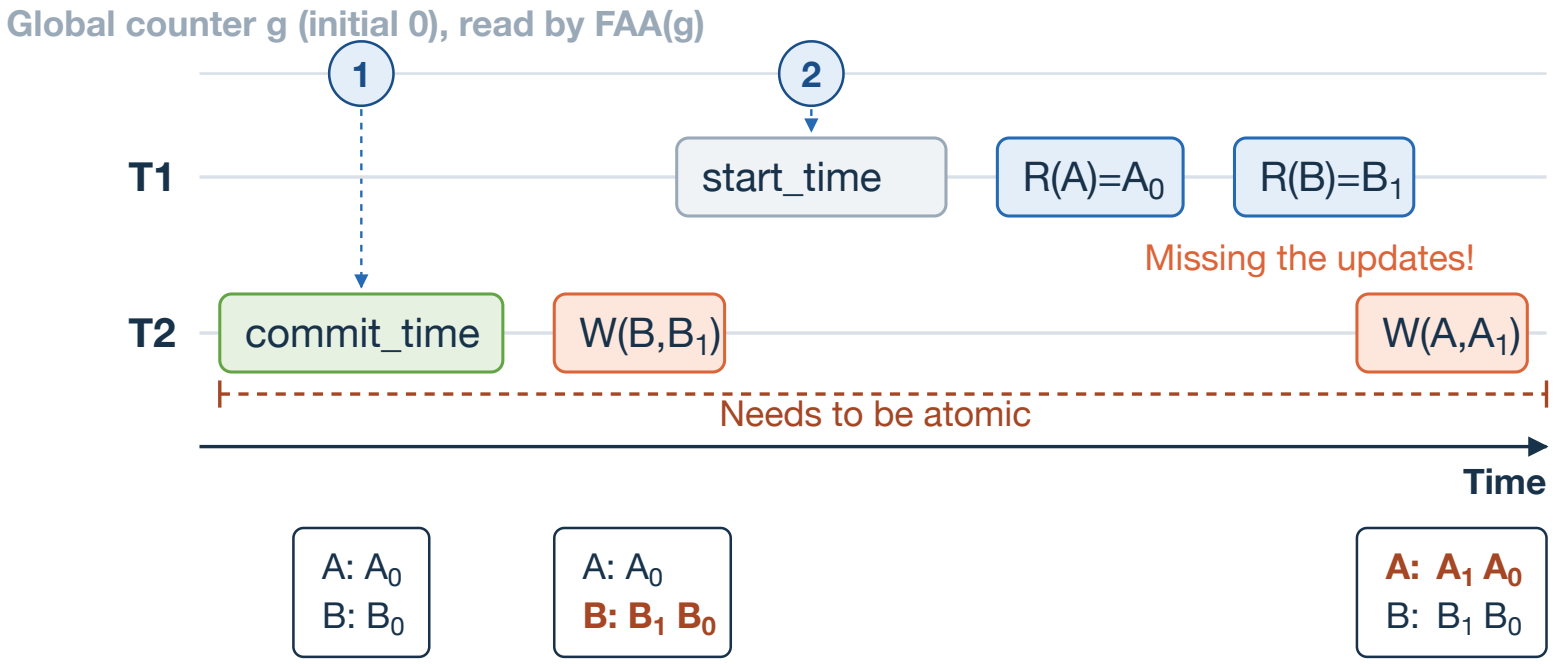
Why incomplete? Partial snapshot (An example)

T1: `Print(A+B)` T2: `B = 2; A = 3`



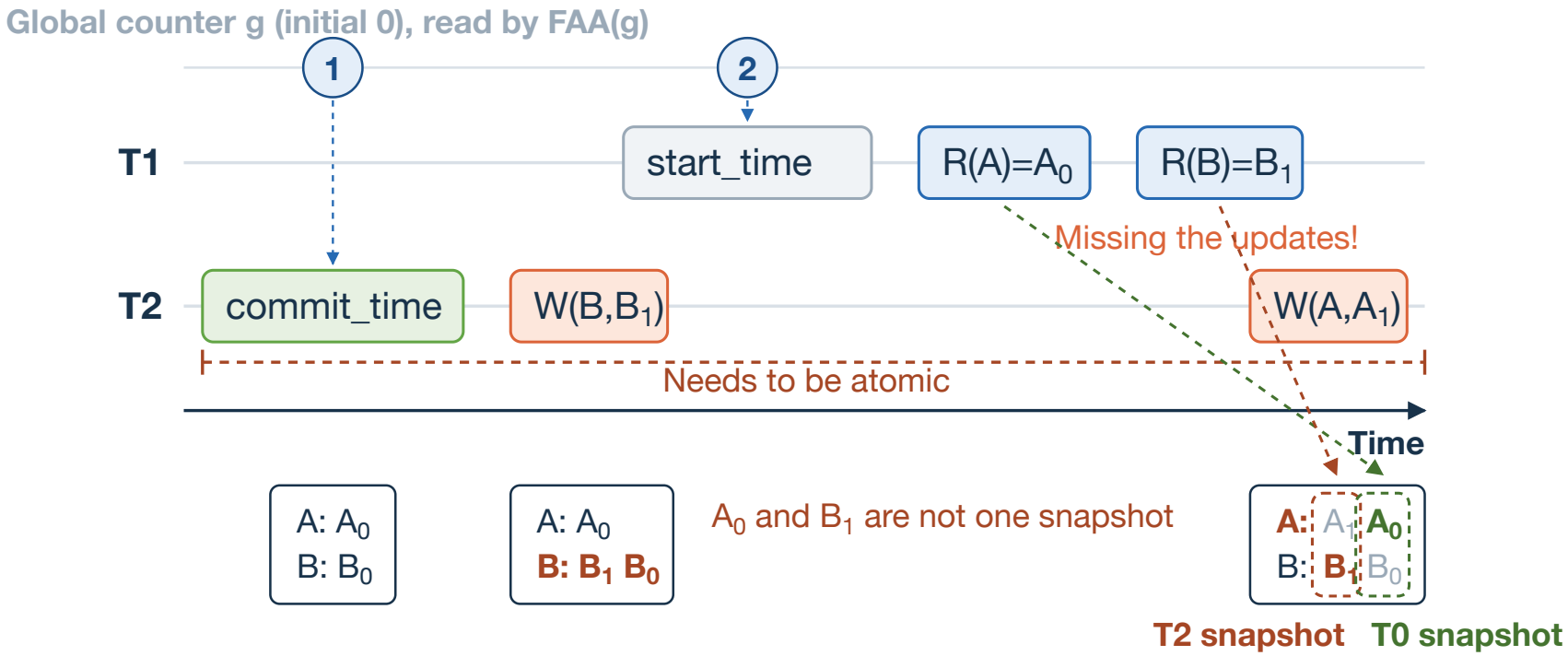
Why incomplete? Partial snapshot (An example)

T1: `Print(A+B)` T2: `B = 2; A = 3`



Why incomplete? Partial snapshot (An example)

T1: `Print(A+B)` T2: `B = 2; A = 3`



We need T2's write to be before-or-after atomic!

Enforcing the write atomicity of reading a snapshot w/ read checks

Observation: If a snapshot is being written, all its records must be locked

- a reader waits on `record.is_locked()` to read the full snapshot

```
Commit(tx):
    for record in tx.write_set:
        lock(record)
    let commit_ts = FAA(global_counter)
    for record in tx.write_set:
        record.insert_new_version(commit_ts, ...)
        unlock(record)
```

```
Get(tx, record):
    while record.is_locked():
        pass
    for version,value in record.sort_version_in_decreasing():
        if version <= tx.start_time:
            return value
```

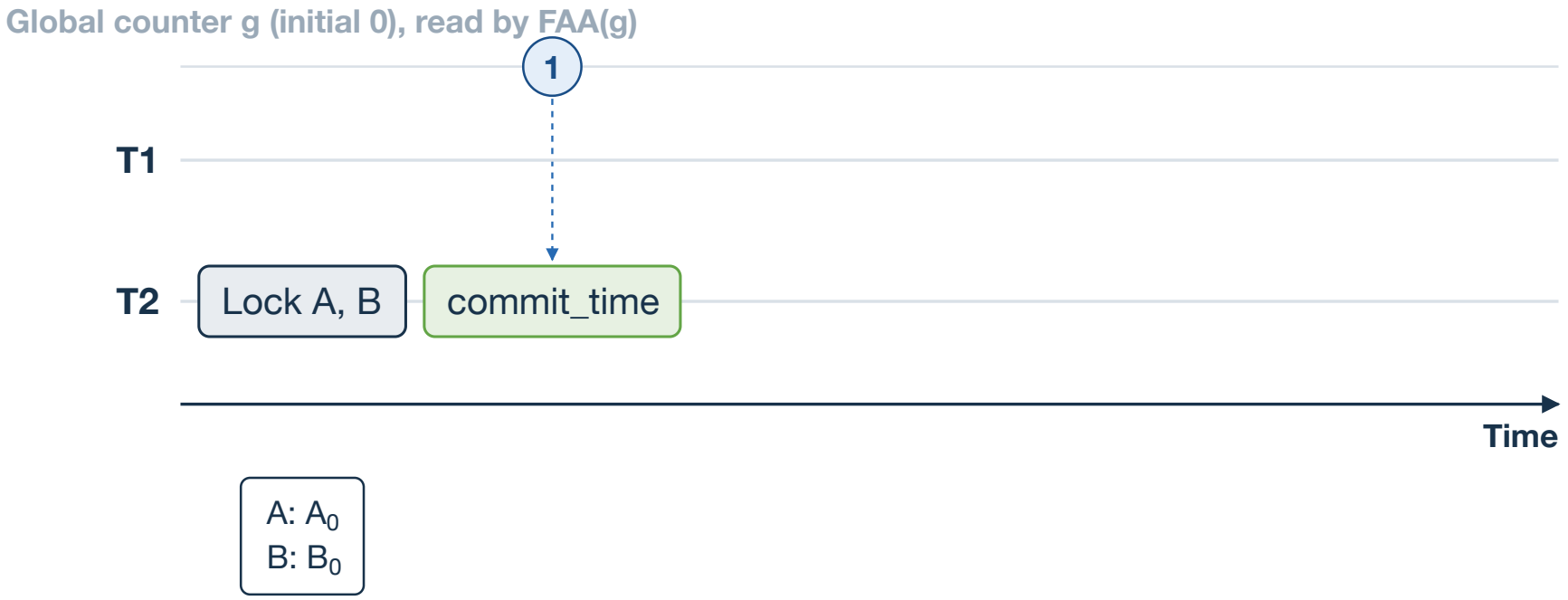
Example revisit

T1: `Print(A+B)` T2: `B = 2; A = 3`



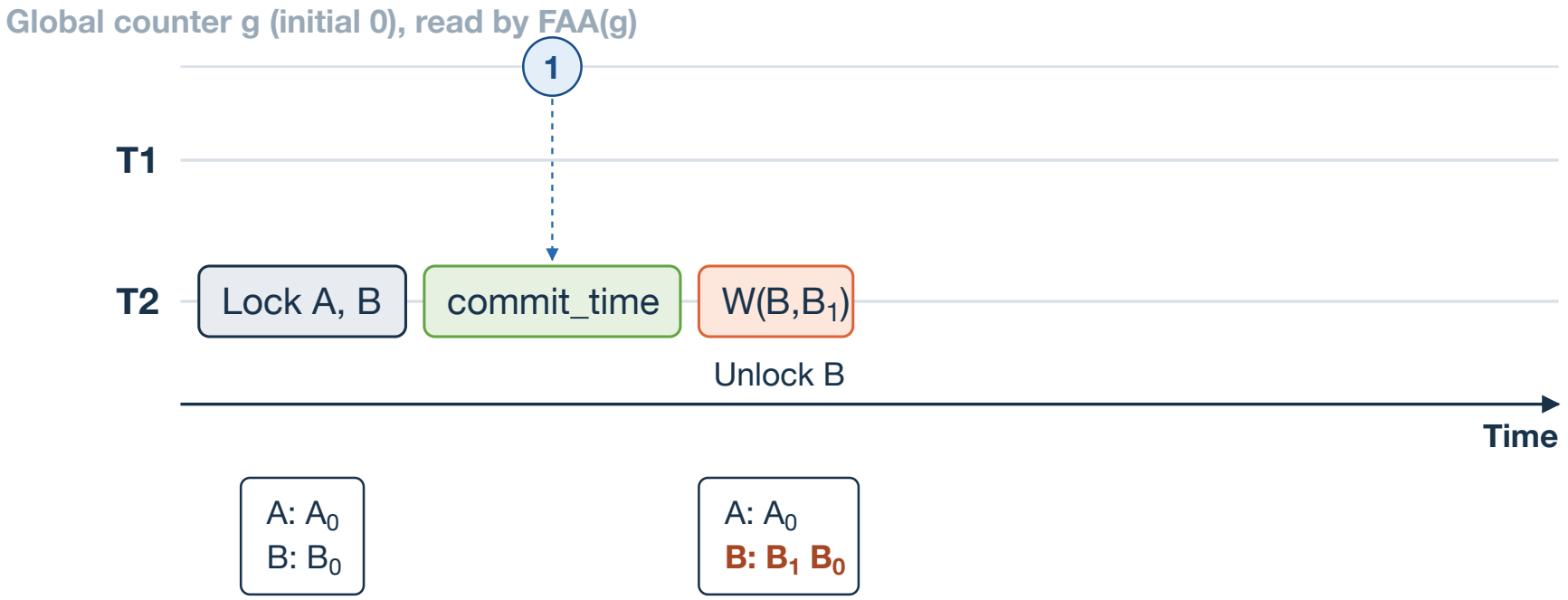
Example revisit

T1: `Print(A+B)` T2: `B = 2; A = 3`



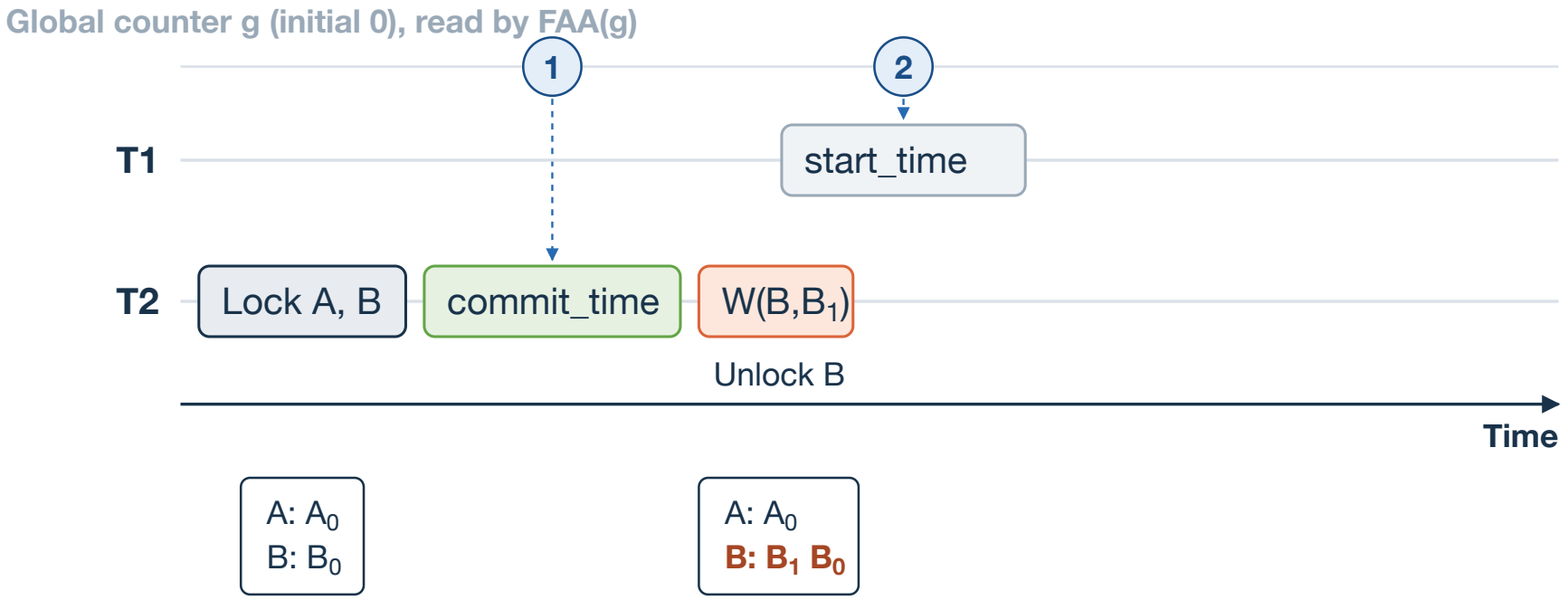
Example revisit

T1: `Print(A+B)` T2: `B = 2; A = 3`



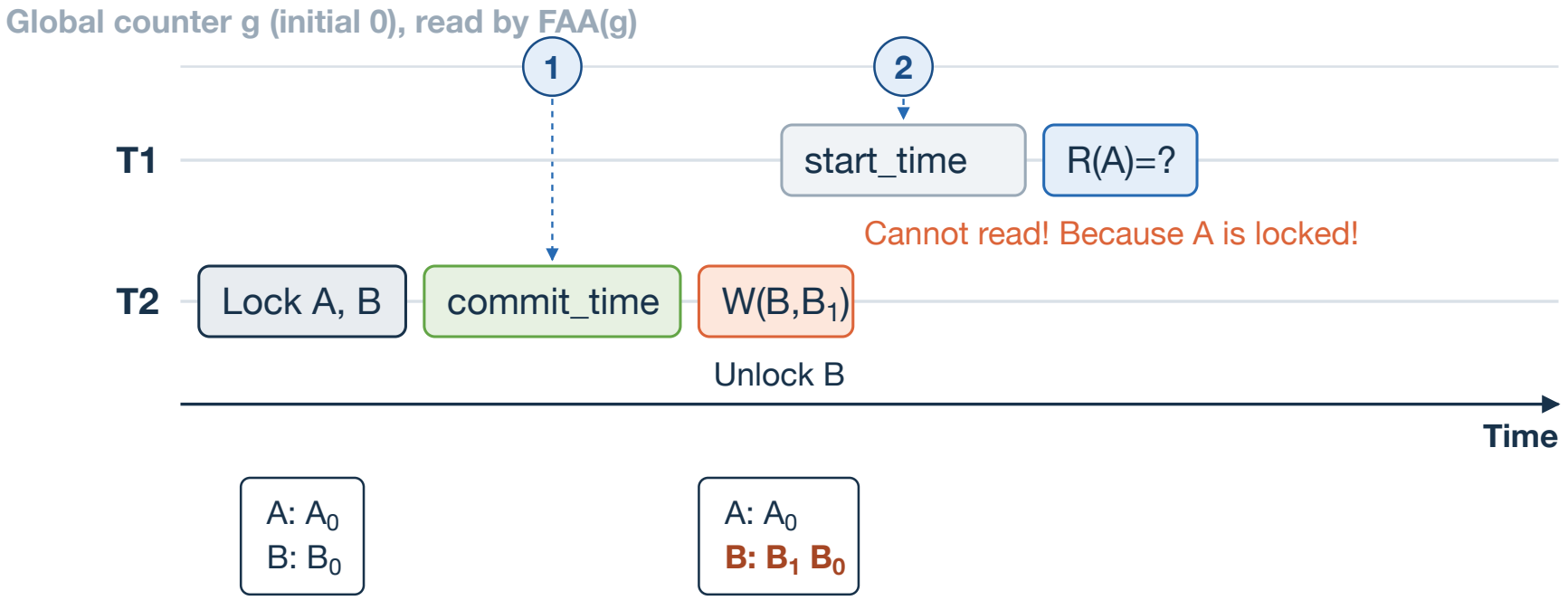
Example revisit

T1: `Print(A+B)` T2: `B = 2; A = 3`



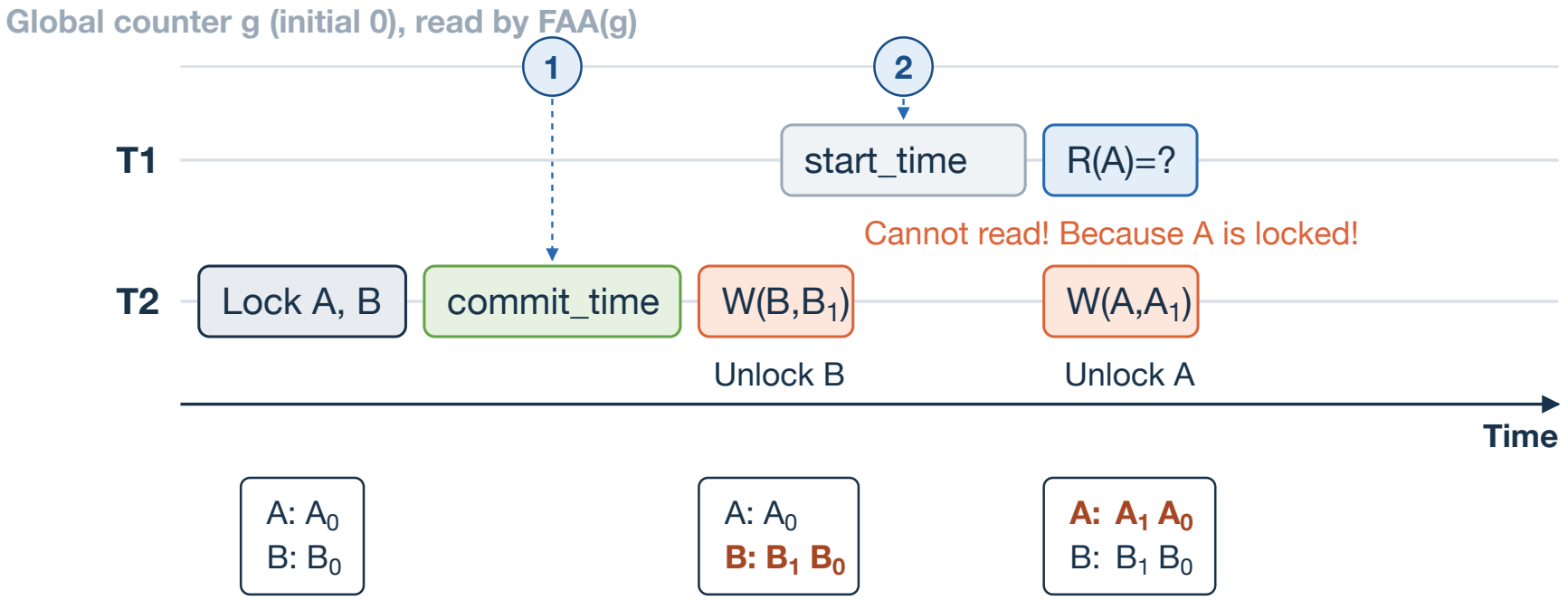
Example revisit

T1: `Print(A+B)` T2: `B = 2; A = 3`



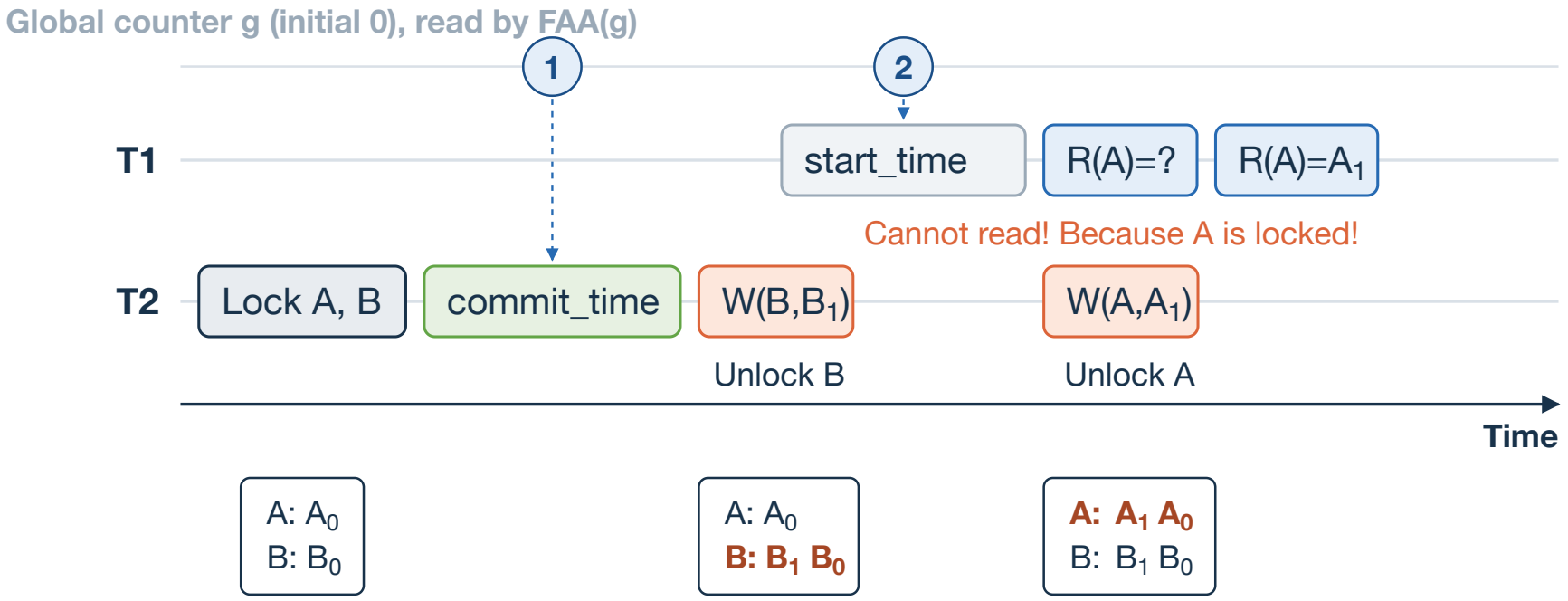
Example revisit

T1: `Print(A+B)` T2: `B = 2; A = 3`



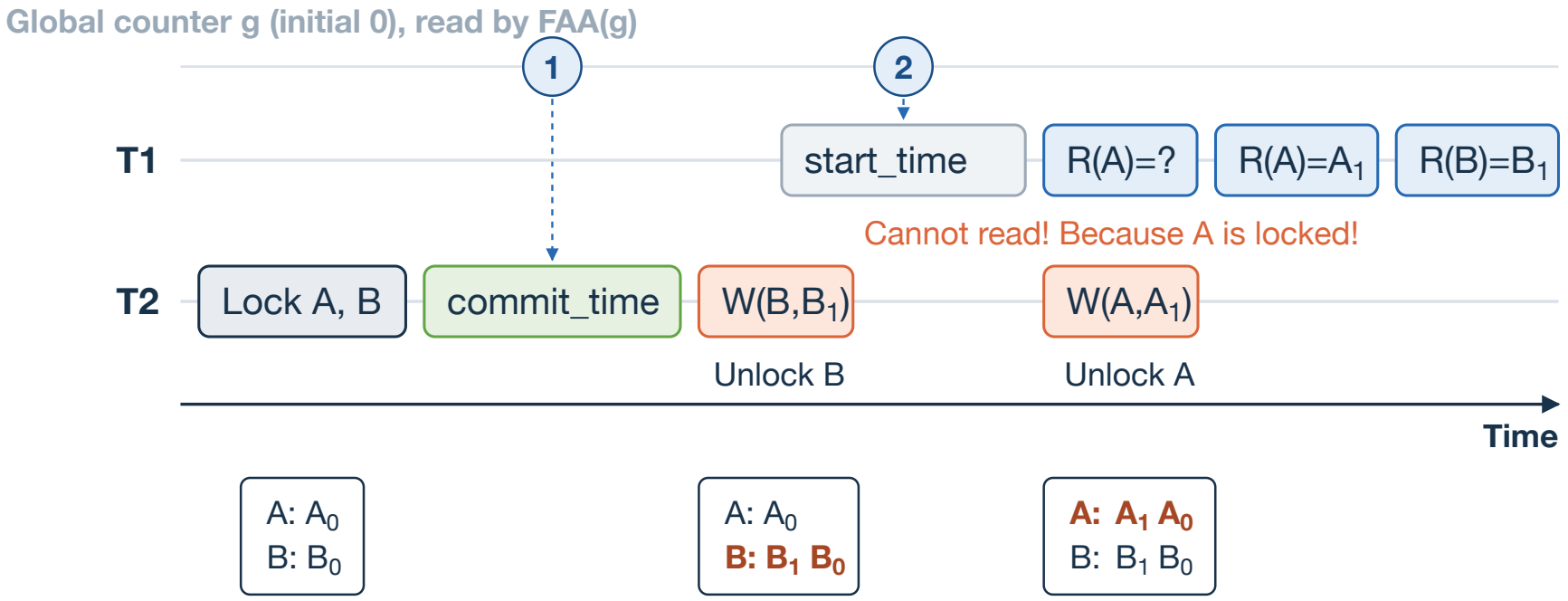
Example revisit

T1: `Print(A+B)` T2: `B = 2; A = 3`



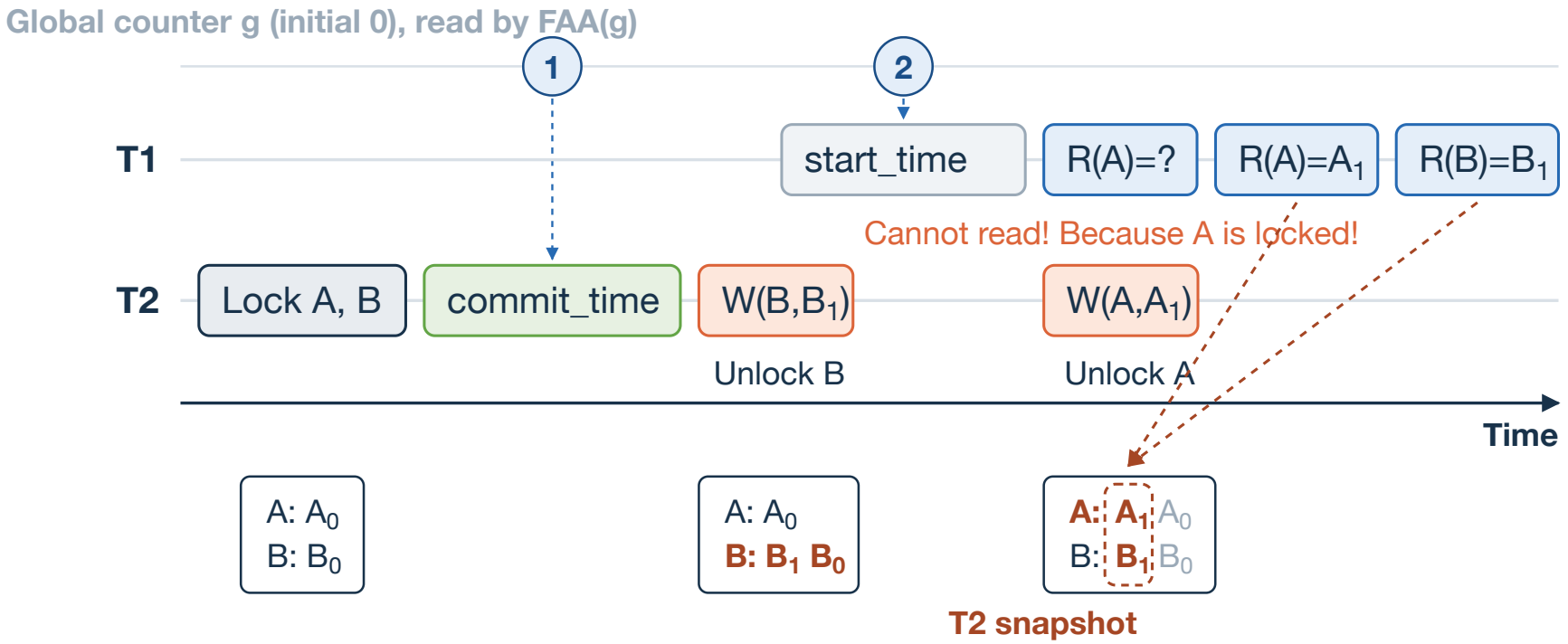
Example revisit

T1: `Print(A+B)` T2: `B = 2; A = 3`



Example revisit

T1: `Print(A+B)` T2: `B = 2; A = 3`



Question: can we reorder getting the commit time and locking?

What about writes?

T1: `A += 1` T2: `A += 1`



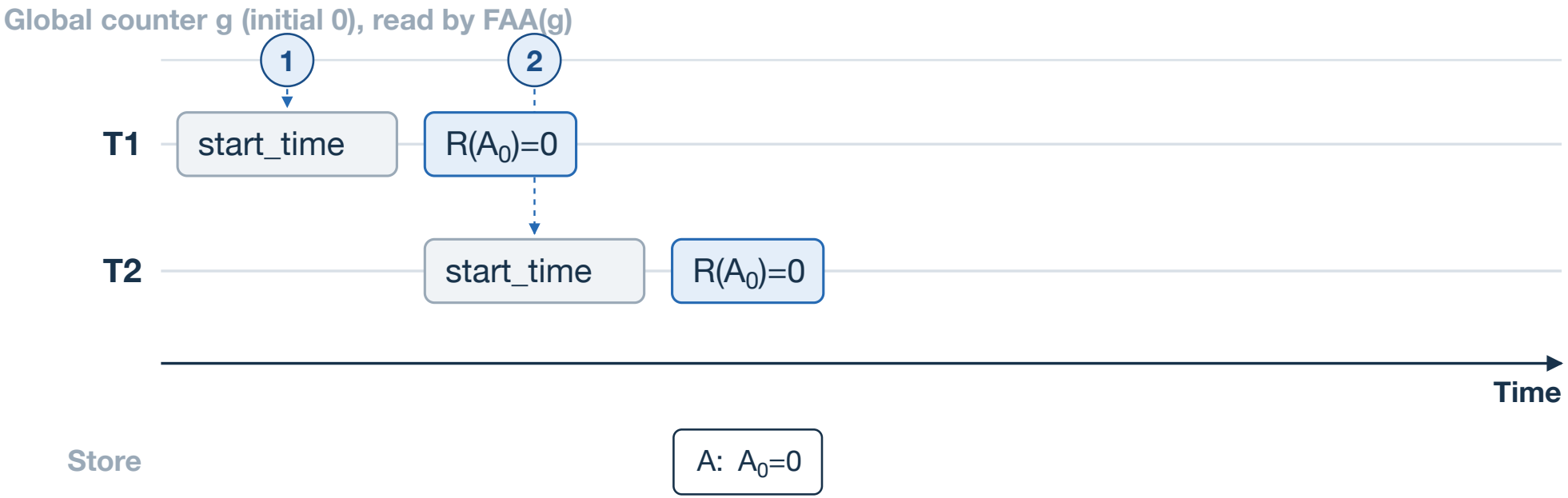
What about writes?

T1: `A += 1` T2: `A += 1`



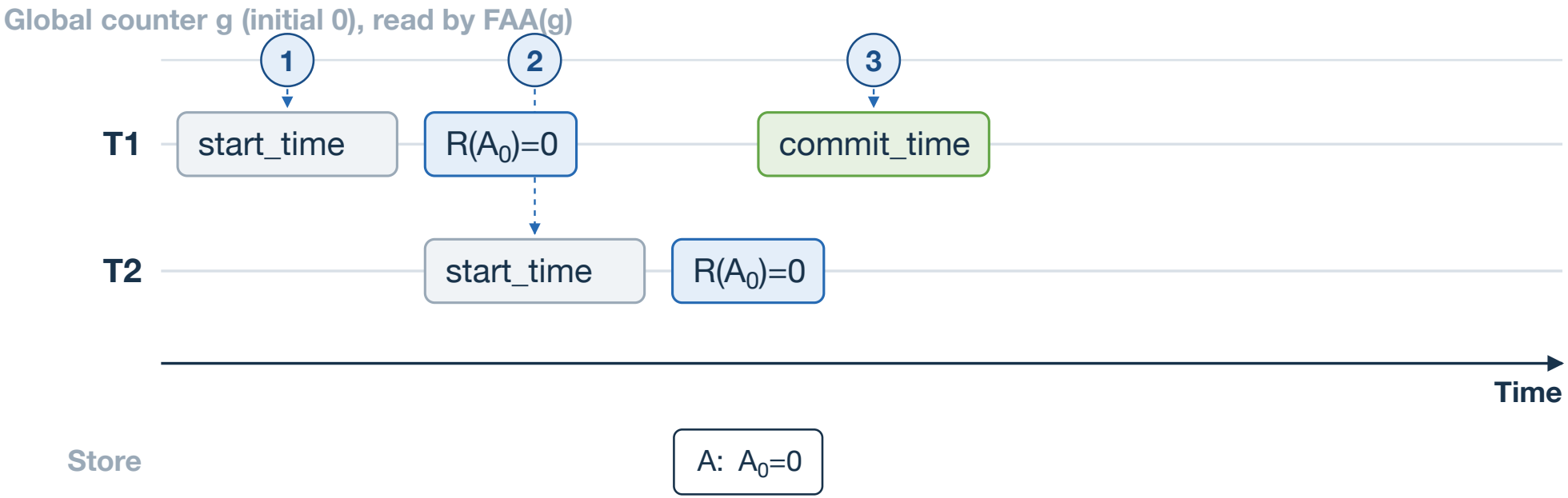
What about writes?

T1: `A += 1` T2: `A += 1`



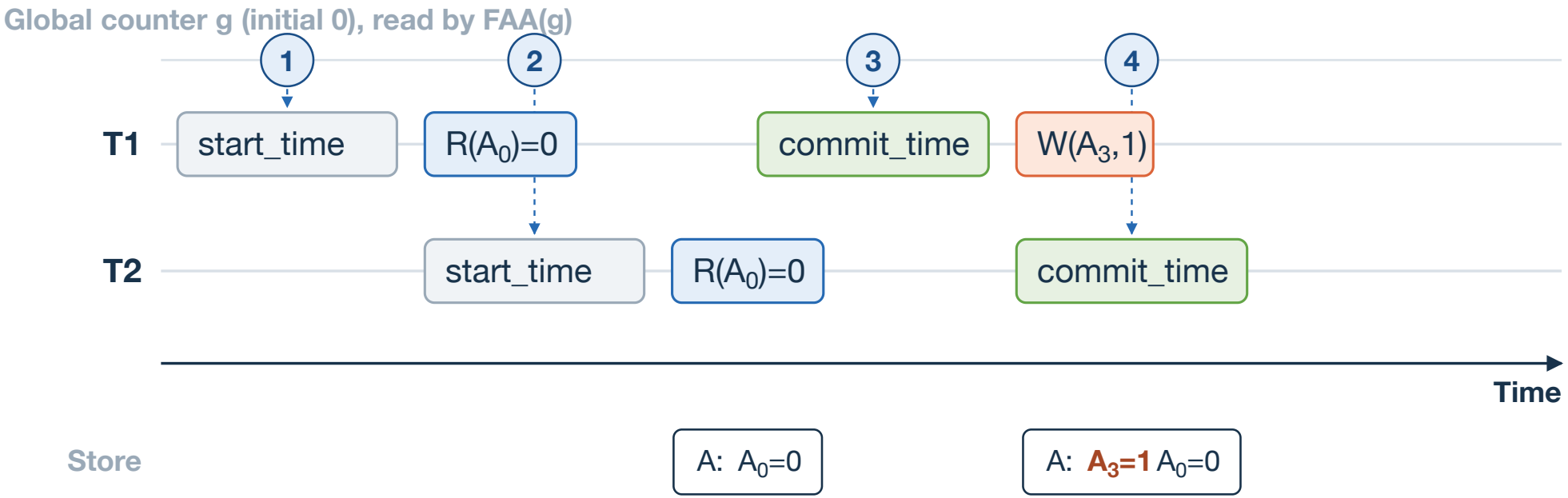
What about writes?

T1: `A += 1` T2: `A += 1`



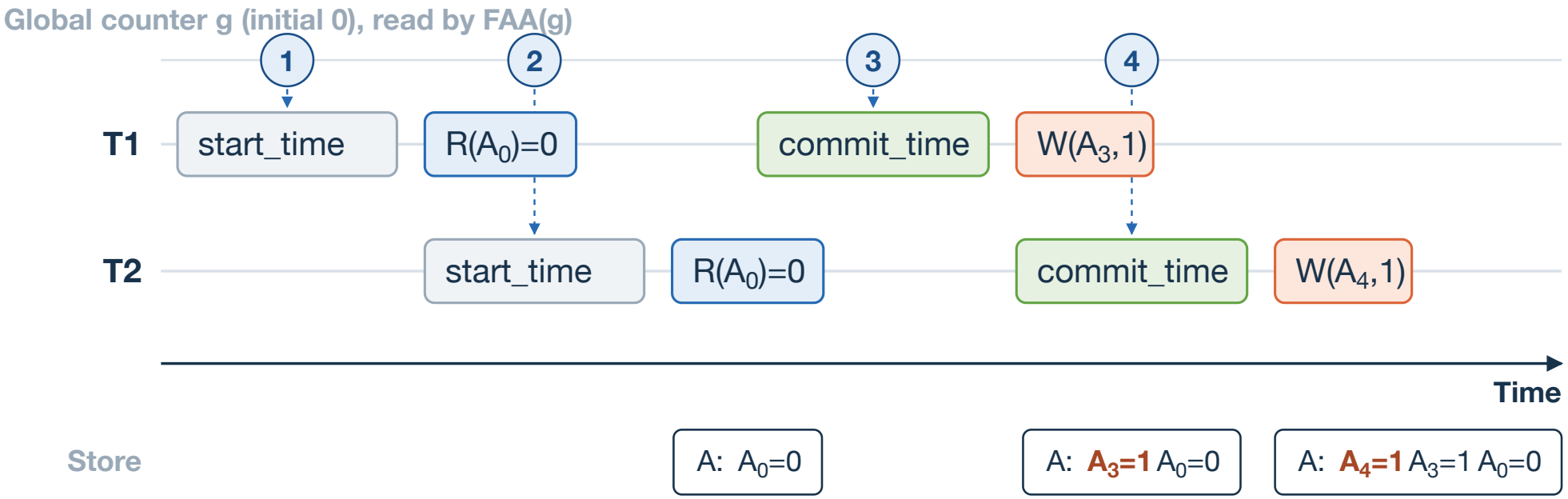
What about writes?

T1: `A += 1` T2: `A += 1`



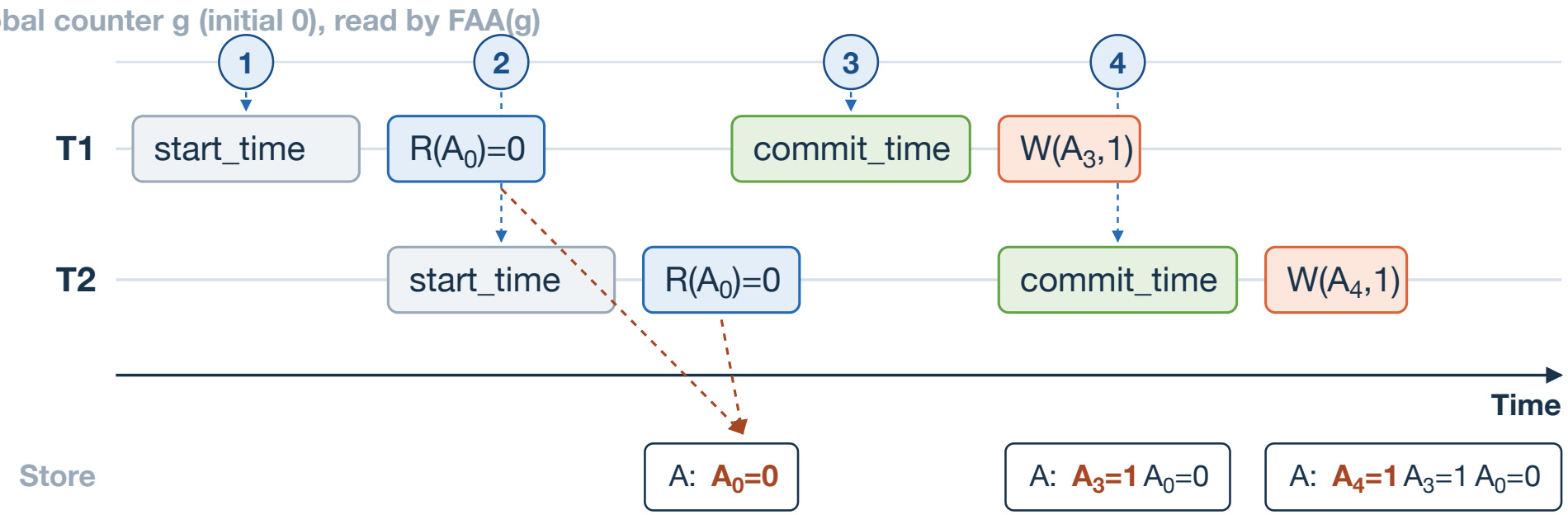
What about writes?

T1: `A += 1` T2: `A += 1`



What about writes?

T1: `A += 1` T2: `A += 1`



Both TXs read A = 0 and both write 1: one update is lost

MVCC ensures no race for reads, but not writes

Race conditions of reads are isolated via snapshots

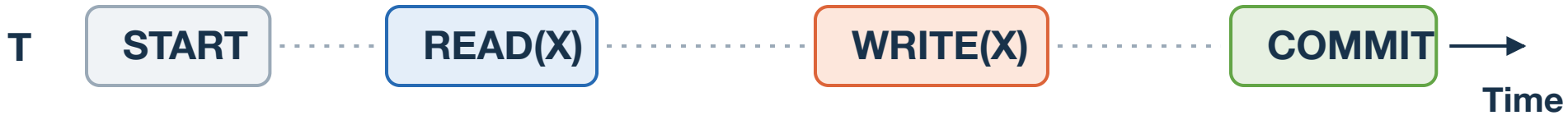
But we still need to rule out race conditions between reads + writes

- We do so by validating the writes at the commit time

The validation is simple:

- During commit time, check whether another TX has installed a new snapshot after the committing TX's start time
- If so, the TX is aborted

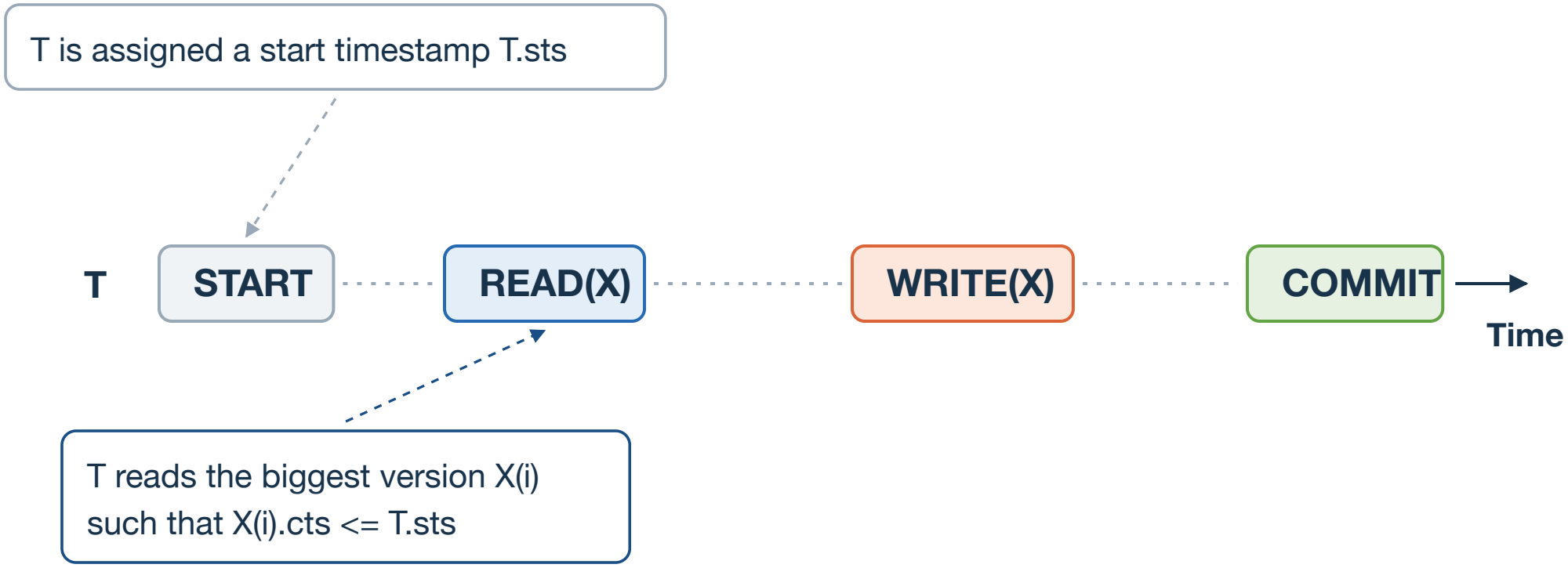
Put it together, MVCC so far



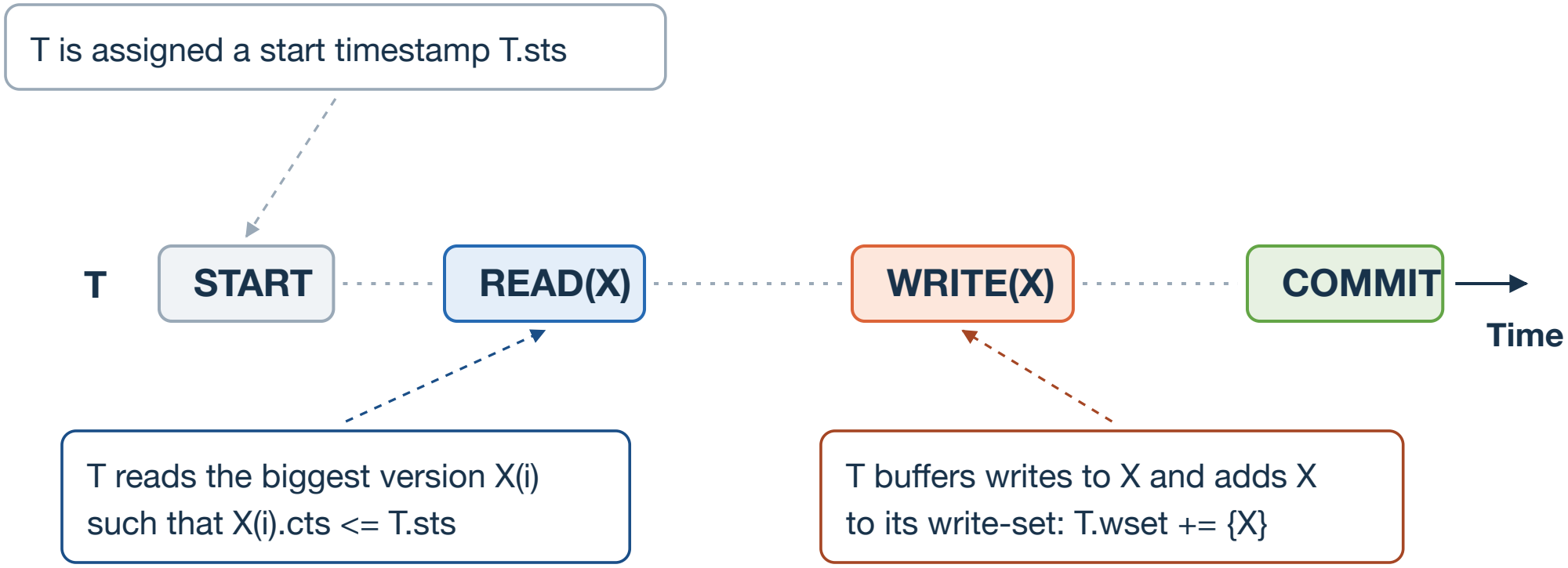
Put it together, MVCC so far



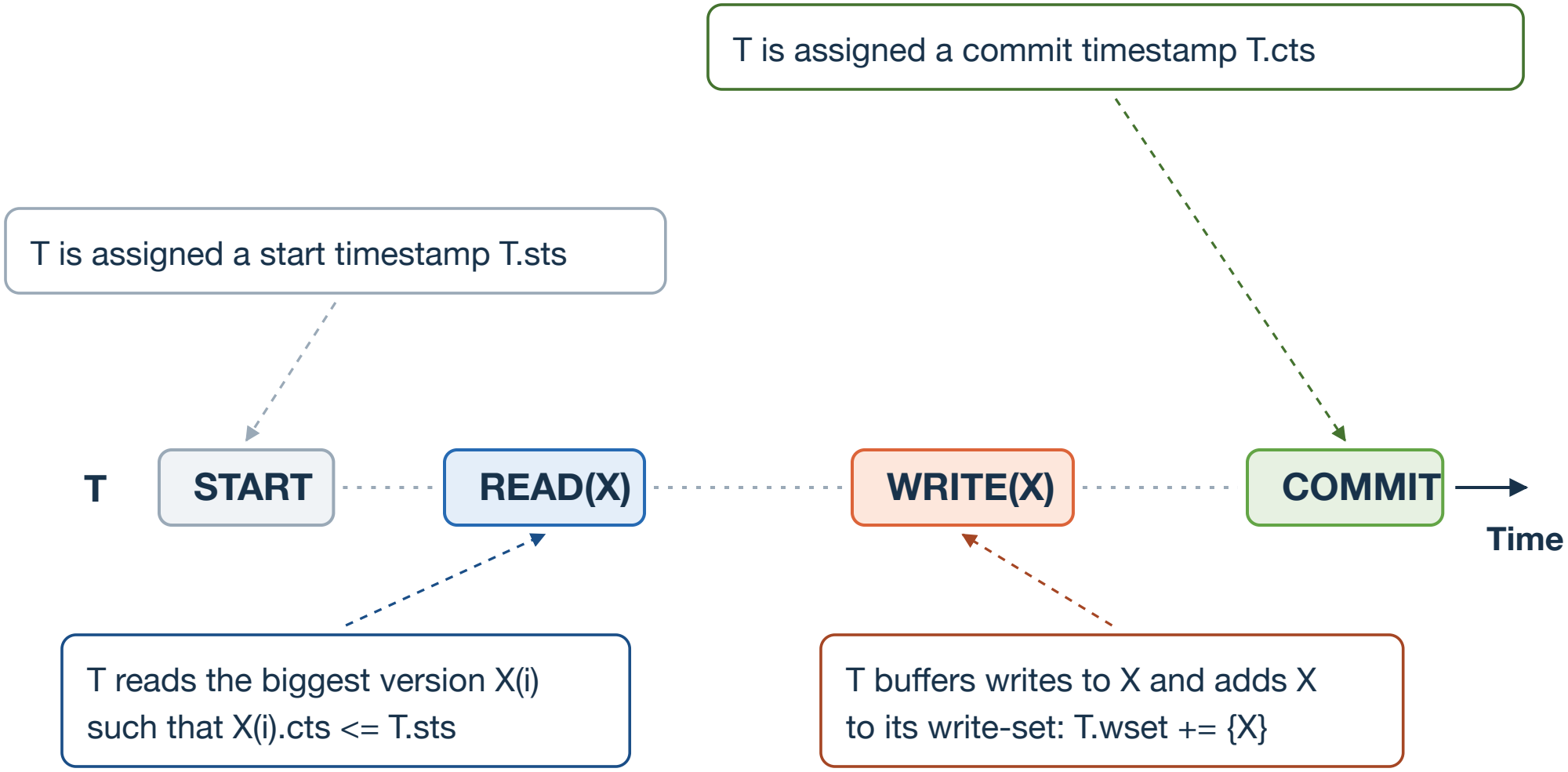
Put it together, MVCC so far



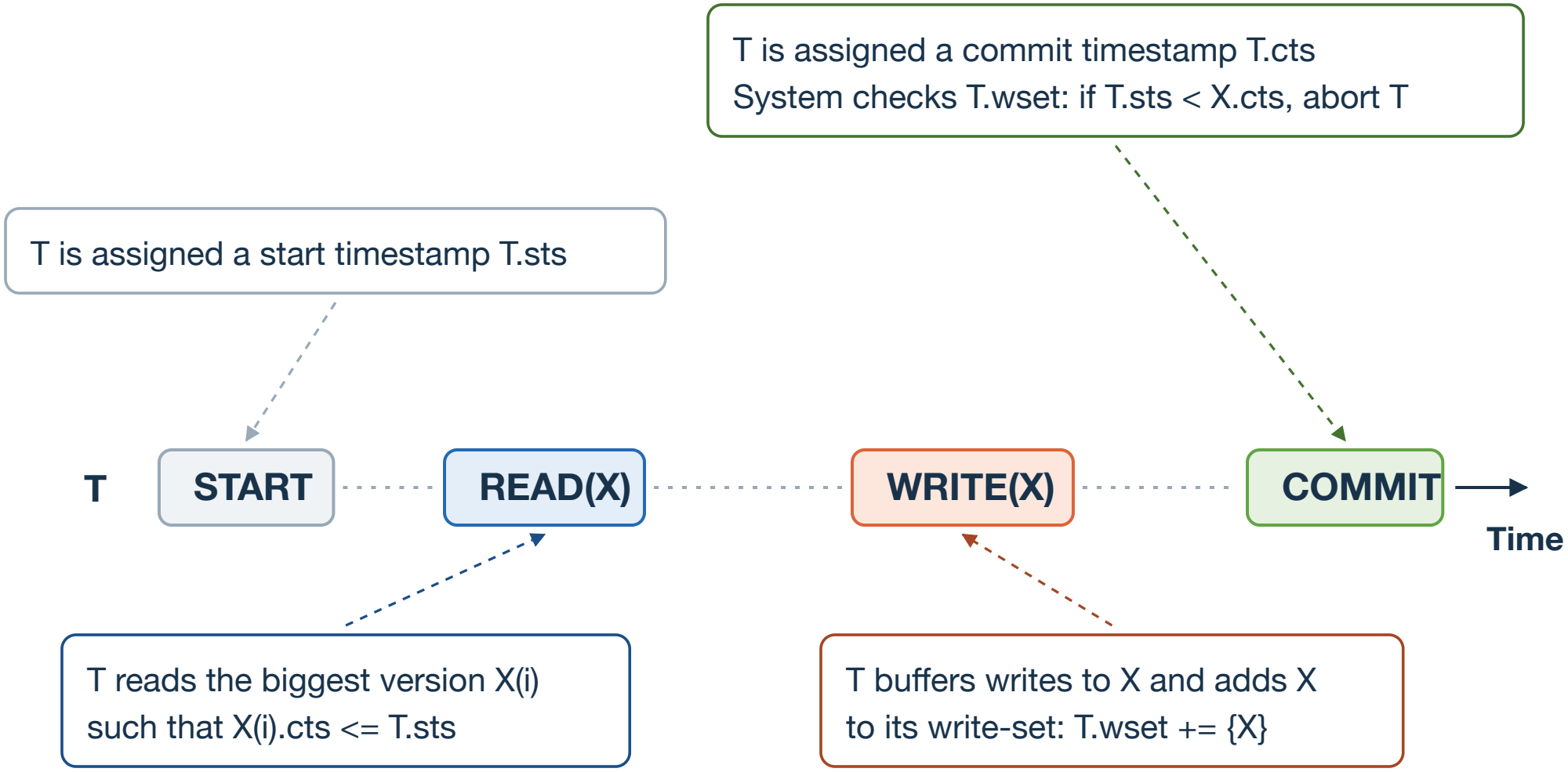
Put it together, MVCC so far



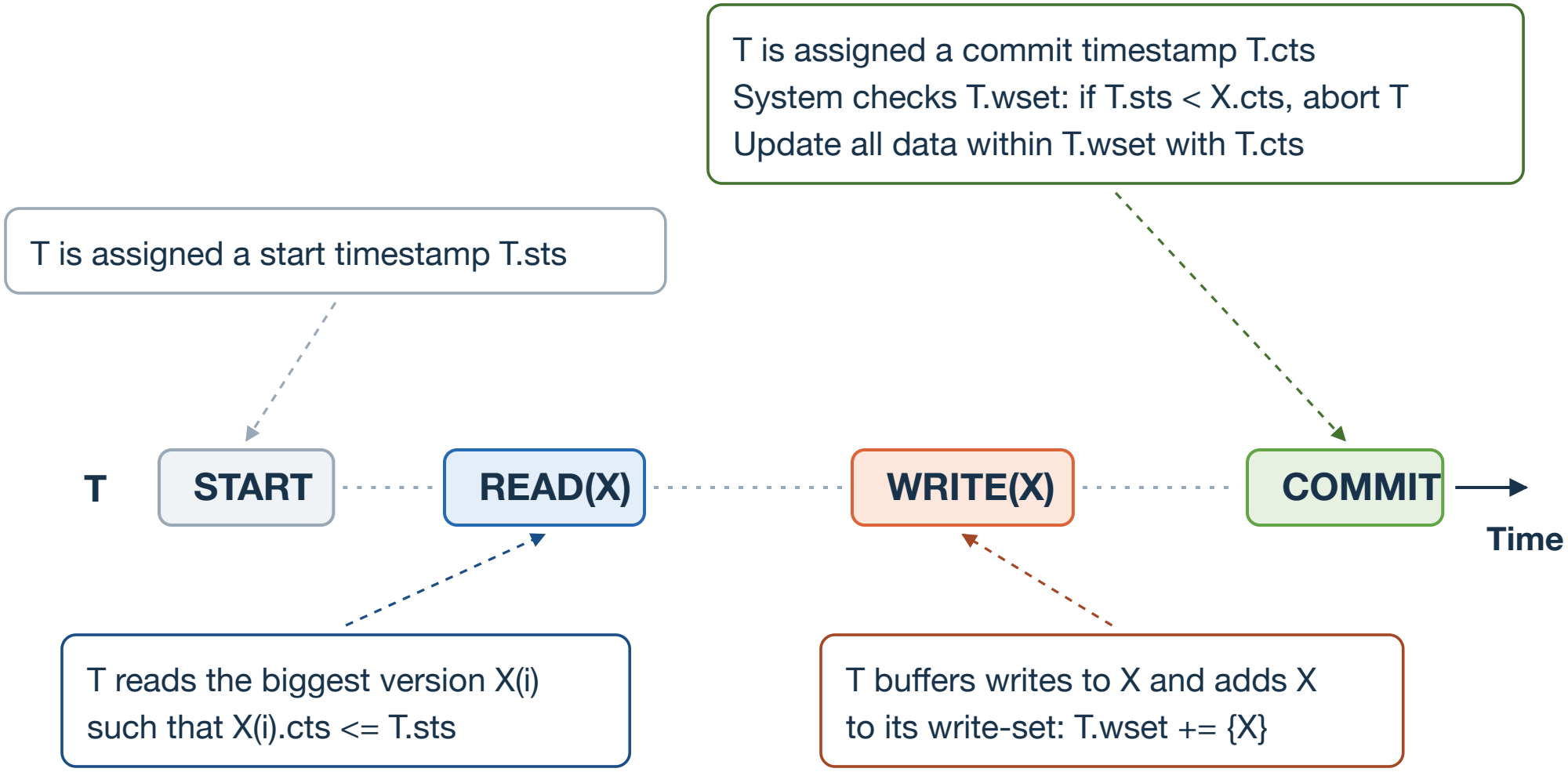
Put it together, MVCC so far



Put it together, MVCC so far



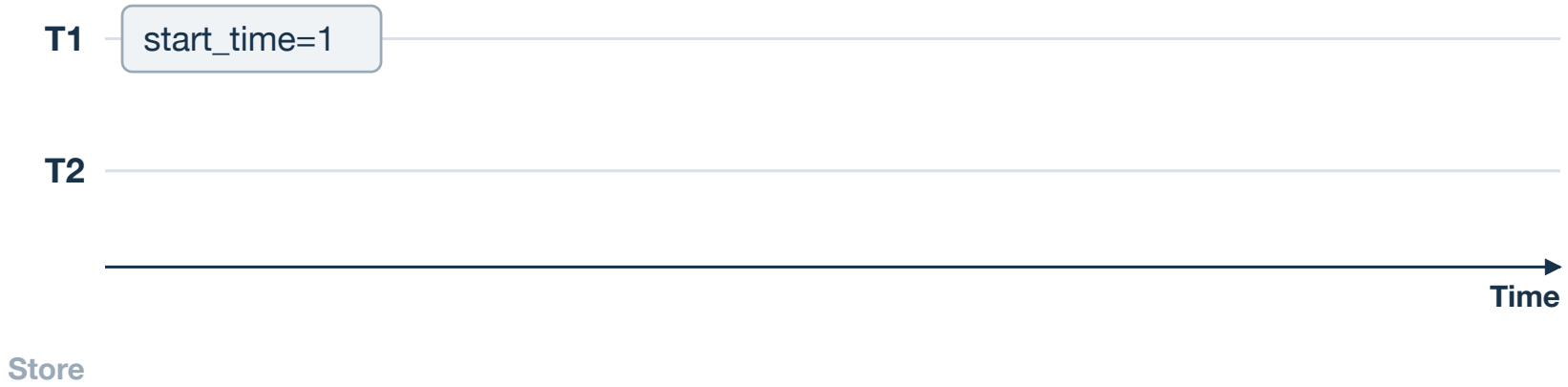
Put it together, MVCC so far



Does our current MVCC method guarantee before-or-after?

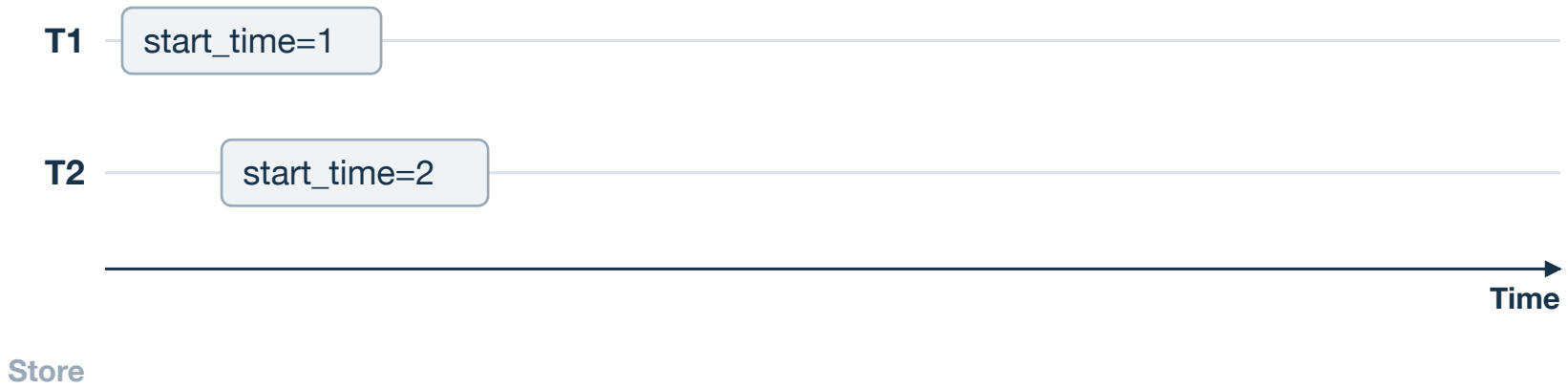
Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



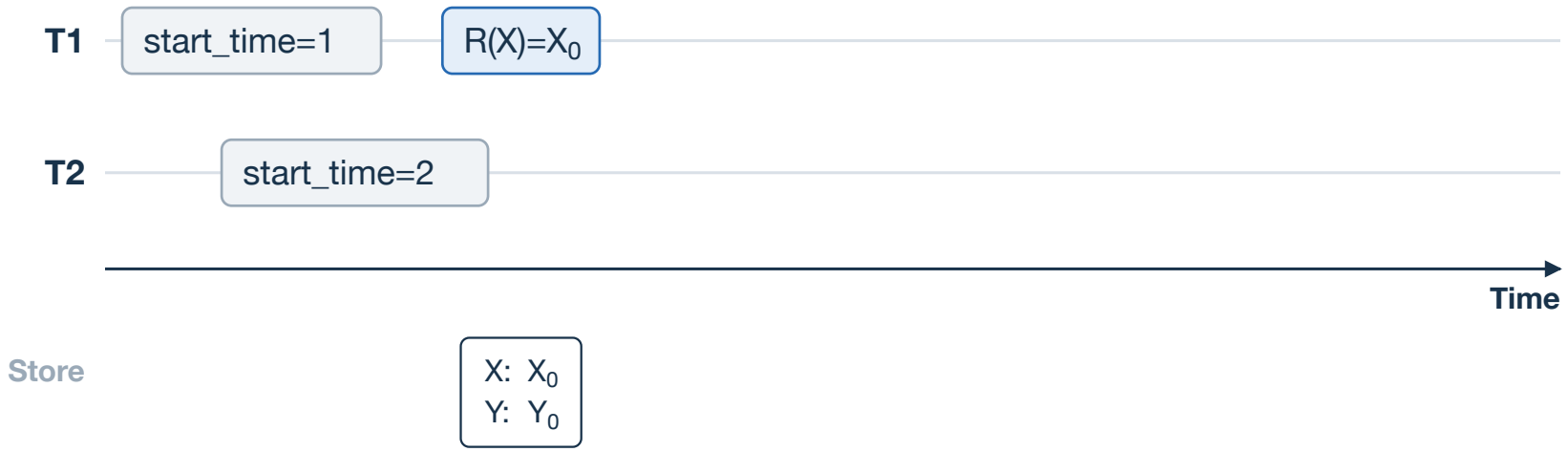
Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



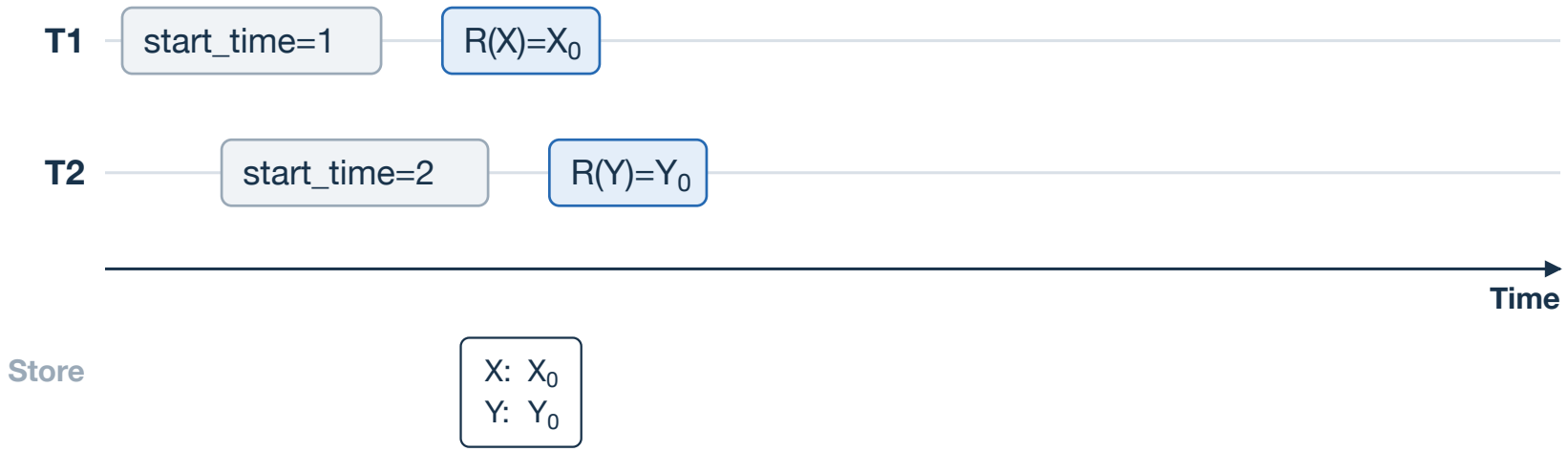
Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



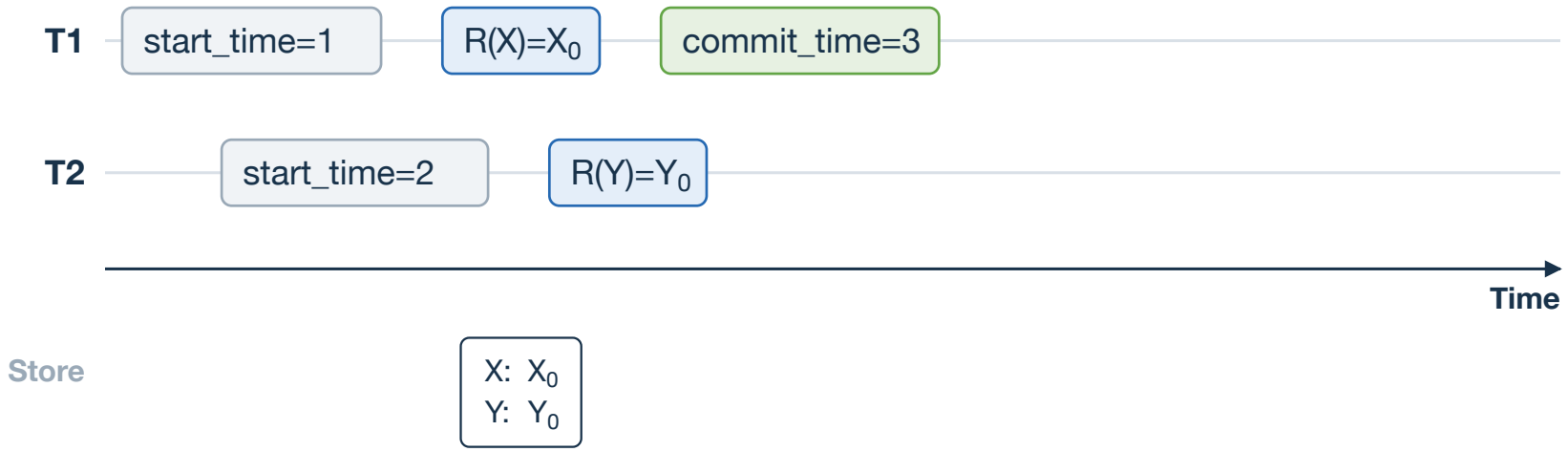
Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



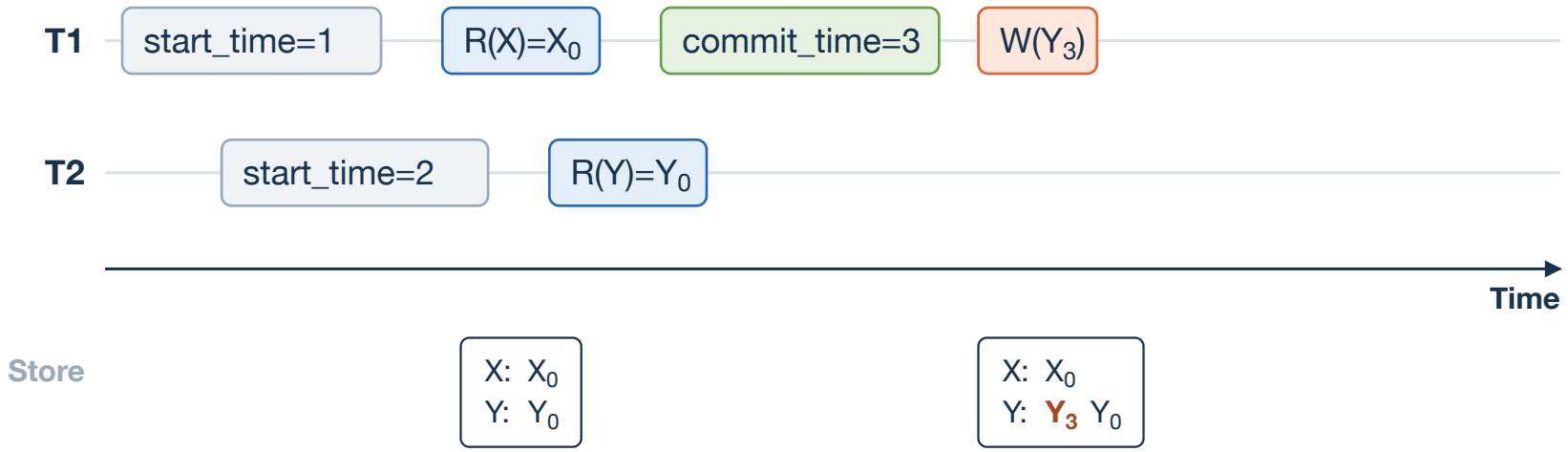
Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



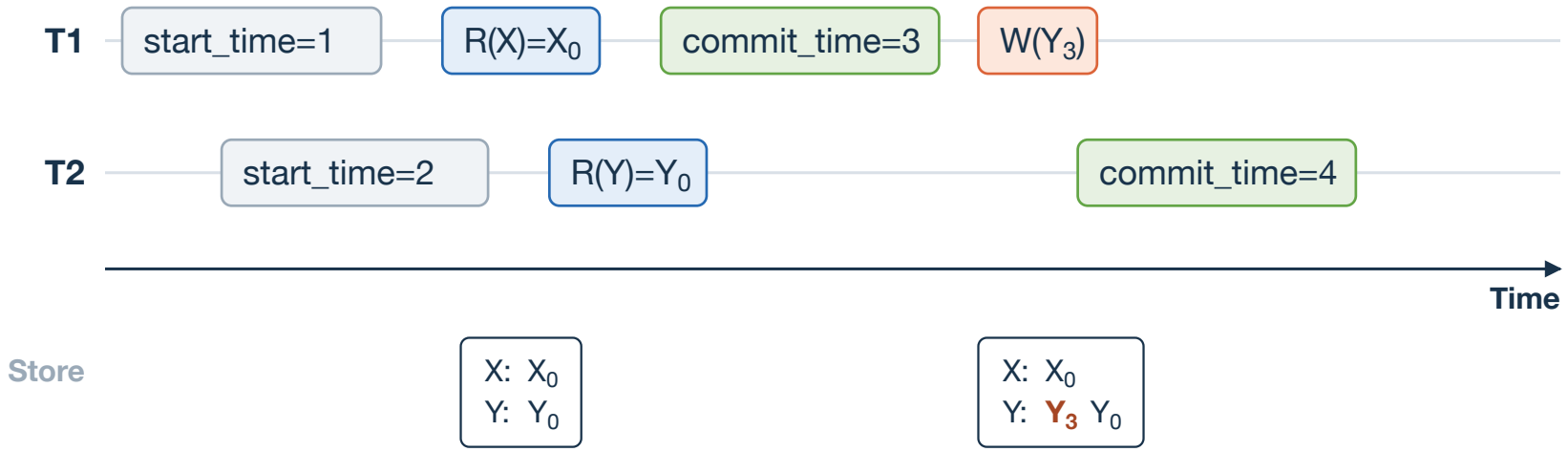
Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



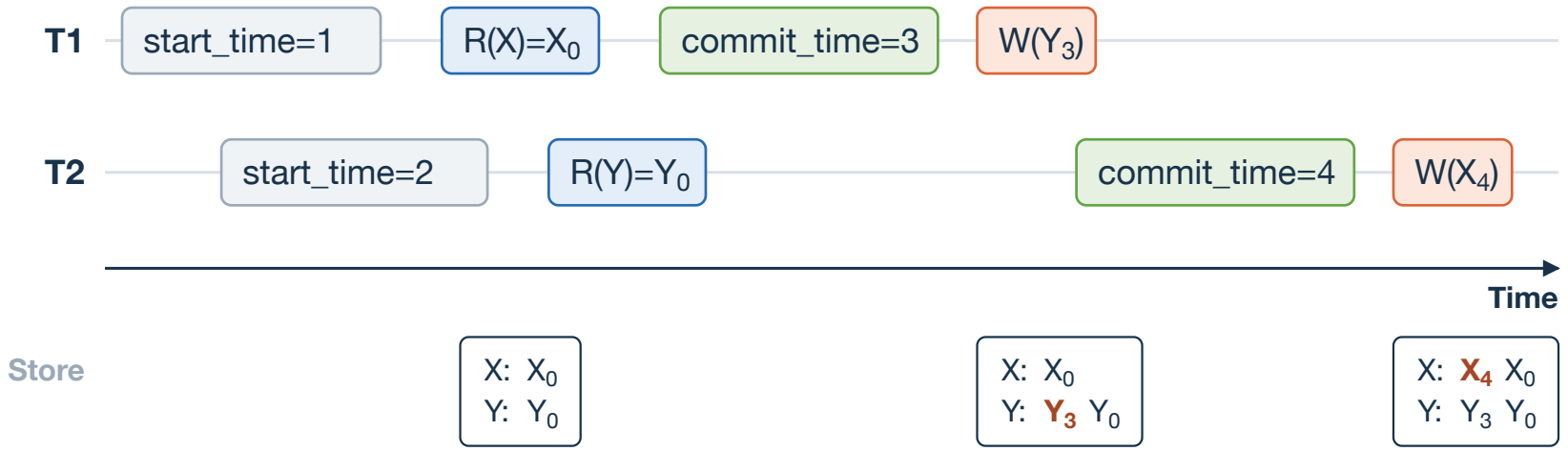
Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



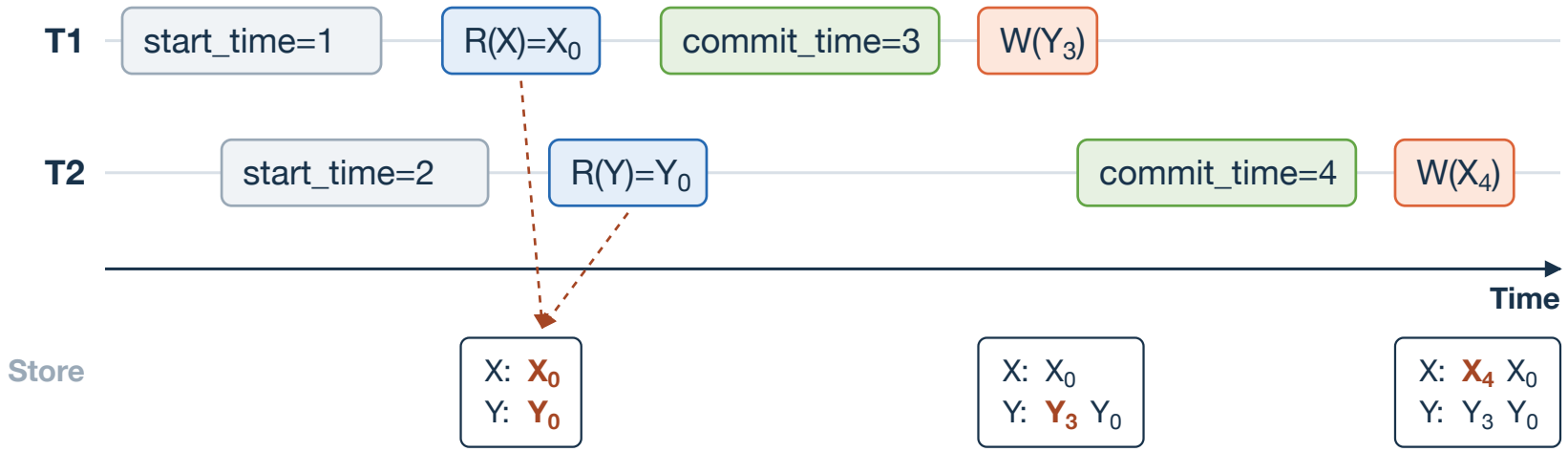
Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



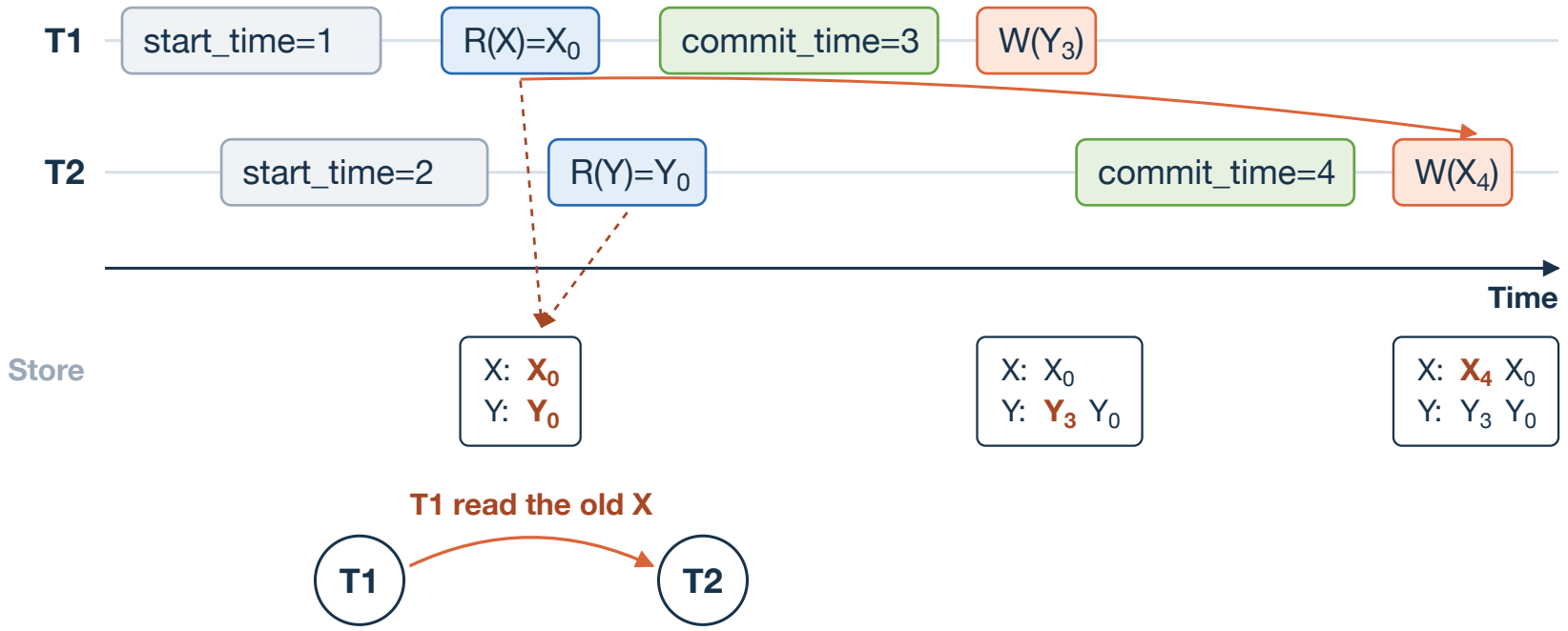
Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



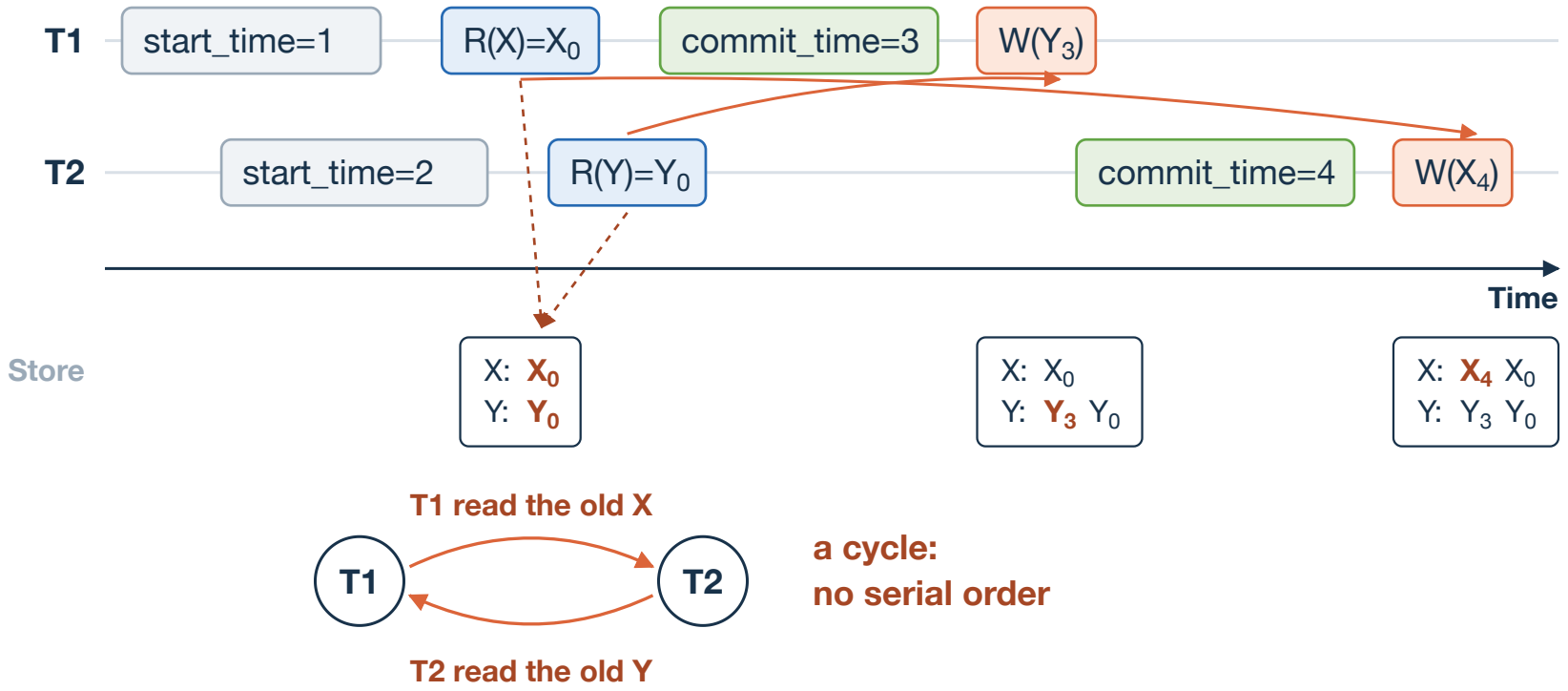
Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



Write skew anomaly

Our current MVCC differs from serializability due to one anomaly



Give a concrete application where this anomaly is undesirable.

Fixing the anomaly

The simplest way is to validate the read set in a read-write TX

- Essentially falls back to OCC for a read-write TX
- But a read-only TX can still enjoy the benefits of MVCC
  - Never aborts & no validations

Usually ignored in practice (Snapshot isolation)

- The MVCC without the read validation is also called snapshot isolation (SI)

Though the idea of MVCC was first proposed in SI, its usage is not restricted to it, e.g., we can have MV-2PL and MV-OCC; & we will see it later

Summary

Three ways to stay before-or-after atomic without holding locks during transaction execution

- **OCC**: run in a private workspace, validate the read set, install the write set
- **HTM (RTM)**: the CPU tracks the read/write sets and validates them for you
- **MVCC**: read a snapshot during transaction execution, so a read-only transaction never validates

参考文献

[1] On Optimistic Methods for Concurrency Control — Kung and Robinson, 1981. ACM TODS 6(2) (P2)

[2] Intel 64 and IA-32 Architectures Software Developer's Manual — Intel (P20)

[3] Numbers Every Programmer Should Know By Year — Colin Scott (P22)

[4] Extracting More Concurrency from Distributed Transactions — Mu et al., 2014. OSDI (P26)

[5] Using Restricted Transactional Memory to Build a Scalable In-Memory Database — Wang et al., 2014. EuroSys (P35)

[6] Speedy Transactions in Multicore In-Memory Databases — Tu et al., 2013. SOSP (P37)

[7] Principles of Computer System Design: An Introduction — Saltzer & Kaashoek, 2009. Morgan Kaufmann (P42)

[8] A Critique of ANSI SQL Isolation Levels — Berenson et al., 1995. SIGMOD (P61)